

Verification Continuum™

Verdi® Power-Aware Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

1. Preface	8
Customer Support	8
Synopsys Statement on Inclusivity and Diversity	9
2. Introduction	10
Overview	10
3. Using Power-Aware Debug	12
Importing CPF/UPF Files into Verdi	12
Importing UPF Files	13
Starting Verdi Power Aware Debug with UFE Flow	14
Power-Aware Debug Panes	14
Power Domain List Pane	15
Filtering Power Aware Objects in the Power Domain Hierarchy Tree	21
Design Browser Pane	23
Filtering Power Elements and Objects in the Design Browser Pane	24
Power State/Mode Table Pane	25
Color Coding in Power State Table	26
Power Source Code Pane	26
Enabling UPF Commands of Inactive Power Domains in the Power Source Code View	28
Power Map Pane	28
Partial Power Map	32
Power-Aware Support in Other Panes	36
Design Browser and Source Code	37
Watch Tab	39
nSchema	40
nWave	41
Viewing Power Objects in nWave	45
Temporal Flow View	51
Tracing Driver/Load on Low Power Design Elements	52

Contents

Power Port	53
Power Port Drivers	53
Power Port Loads	53
Power Net	54
Power Net Drivers	54
Power Net Loads	54
Power Logic Net	55
Power Logic Net Drivers	55
Power Logic Net Loads	55
Power Logic Port	56
Power Logic Port Driver	56
Power Logic Port Load	56
Power Supply Set	56
Power Supply Set Drivers	57
Power Supply Set Loads	57
Power Domain	58
Power Domain Drivers	58
Power Switch	58
Power Switch Drivers	58
Power Switch Loads	58
HDL Signals	58
Drivers	59
Loads	61
Support for Power Pad Cell, Power Switch Cell, and Punch Ports in nSchema	62
Debugging Power Pad Cells in nSchema	62
Debugging Power Switch Instrumented Cells in nSchema	63
Debugging Punch Ports in nSchema	64
Power-Aware Debug Support on Temporal Flow View (TFV) in Transition Mode . . .	65
Supply Set of Power Domain	66
Supply Net/Supply Port	67
Power Switch	67
Isolation	68
Signal Naming	69
Retention	70
SRSN/SPA	70
Boundary Cell	71
Displaying Annotation Values for Power Signals in the Power Source Code Pane	72
Viewing Power Switch Signal Annotation	72
Viewing Supply Set Signal Annotation	72

Contents

Viewing Power State Table Signal Annotation	73
Viewing Power State Group Signal Annotation	74
Viewing and Editing UPF Source Files	74
Viewing UPF files in the FileList View	75
Enabling UPF File Editing Feature in All Flows	77
Adding Edit Option in RMB Menu	78
Tcl Command	78
FilelistViewEditSourceFile	78
Support for the set_repeater UPF Command	79
Debugging Repeater Cells in nTrace	80
New Tcl Commands and Options	84
Debugging Repeater Cells in Power Domain Hierarchy Pane	85
New Tcl Commands and Options	88
Debugging Repeater Cells in nWave	89
Setting Highlight Color for Repeater Signals	89
Displaying In/Out Direction for Repeater Signals in Waveform	90
Applying Repeater Rules	90
Power Masking for Repeater Signals	91
Filtering of Repeater Signals in Get Signals Dialog Box	91
New Tcl Commands and Options	91
Debugging Repeater Cells in nSchema	93
Trace Driver/Load Support for Repeater Cells	94
Trace Driver	95
Trace Load	95
Debugging Repeater Cells in Temporal Flow View	95
Support Pass Through connect_supply_net and connect_logic_net commands as Pass-Through Trace Result	97
Power Aware Trace Driver/Load Support for SPICE Signals	99
Reporting the SPICE Supply Port Connection to UPF Supply Net	101
4. Power-Aware Debug Functions	103
Highlighting Power Domain, Power Objects, and Power Signals	104
Highlighting Power Domain	104
Highlighting Power-Aware Objects	108
Highlighting Power Signals	111
On-Demand Dumping of Low Power Signals into FSDB File in Interactive Debug Mode	114
Power Source Code Pane	115

Contents

Power Domain Hierarchy Pane	115
Signal Pane and Watch Pane	116
Dumping PST and PSG Information	117
Reporting Impacted Signals	117
Listing HDL Signals	120
Listing Power Related X	121
Power Sequence Checking	123
Displaying Isolation Cell Information	126
Debugging VCS Low Power Instrumented Objects in Verdi	129
Use Model	129
Dumping Low Power Debug DB XML File During Simulation	129
Loading the Dump XML Database File in Verdi	130
Viewing Dump XML Database File in Verdi	130
Adding Instrumented Signals to nWave for Different Policies	133
Isolation Signals	133
Retention Signals	135
SRSN Signals	137
SPA Signals	138
Power Aware Tracing for Boundary MUX Cells	139
Support for Boundary MUX Cells in nTrace	139
Power Domain Not Modeled as Boundary MUX	140
Power Domain Modeled as Boundary MUX	141
Limitations and Error Handling	143
Known Issues	144
Support for Boundary MUX Cells in nWave	144
Support for Boundary MUX Cells in nSchema	145
Support for Boundary MUX Cells in Temporal Flow View	147
Power Corruption Model in Temporal Flow View	147
Directly Importing Liberty Files and Displaying Power Cell Information	150
Debug Support for nwell and pwell Supply Functions	151
Viewing nwell and pwell Supply Functions in the Power Domain Hierarchy and Signal List Pane	152
Debugging nwell and pwell Supply Functions in nTrace	154
Adding nwell and pwell Supply Functions to nWave	155
Support for nwell and pwell Supply Functions in nSchema	155
Support for nwell and pwell Supply Functions in TFV	156
Displaying Power State Group in Power State Table Frame	156

Contents

Overview	156
Use Model	157
Supported Features	159
Viewing Supply Net Relationship	159
Supply Net Relationship Frame	160
Supply Net Relationship Section	161
Merged Power State Table Section	163
Viewing the State Names and Supply Values of a Supply Net	167
Dumping Low Power Debug DB XML File During Compilation In UFE Flow	170
Use Model	170
5. Demo	171
6. Appendix A: Power Map Validation	182

1

Preface

Verdi® Automated Debug Platform supports power-aware debug. The standard Verdi debug capability can be used to debug power issues that are detected by these visualizations. This guide describes various scenarios for using Power-Aware Debug functionality in the Verdi Automated Debug Platform.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:_...:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to

identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

2

Introduction

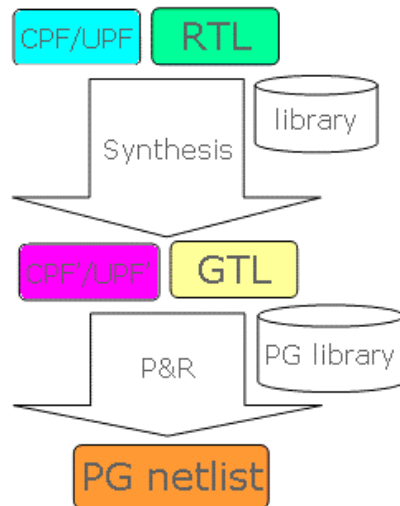
Overview

The power architecture in today's designs is getting increasingly complex, and multiple power domains with various power modes require a thorough verification method. Some technologies are provided to define and verify the complex power domains. Among these technologies, standard power formats, Unified Power Format (UPF - IEEE 1801-2009 standard), and power-aware simulations are widely used to ensure that low power designs still produce a high confidence chip.

In order to support the standard power formats, and also to visualize the simulation results, the Verdi® Automated Debug Platform supports power-aware debug. The standard Verdi debug capability can be used to debug power issues that are detected by these visualizations.

When the RTL design is converted to a gate-level design, part of the power intent defined in the UPF power files is synthesized either by a synthesis tool or by the designers manually. For example, isolation or level-shifter rules are implemented by inserting isolation or level-shifter cells during conversion from RTL to gate-level design. Along with the conversion, the content of the UPF files should be refined by the tool or designers respectively to reflect the changes. For the previously mentioned example, the related isolation/level-shifter power rules should be removed from the UPF files since they are replaced by the isolation/level-shifter cells respectively (for the flow, see [Figure 1](#)).

Figure 1 UPF Refinement



The refined UPF (UPF') files and the gate-level design need to be verified to confirm equal functionality to the original UPF files and the RTL design. The Verdi platform provides exactly the same power-aware debug capability as in the RTL stage, so that you can debug inconsistencies (between RTL and gate-level) in the same Verdi environment.

Additionally, the Verdi platform can recognize and highlight power cells that are unique to the gate-level. The Verdi platform can import the power cell information from the Synopsys Liberty files (for usage details, see the [Directly Importing Liberty Files and Displaying Power Cell Information](#) section) and UPF files (power cell commands). The power cells can be highlighted that helps you to compare the inserted power cell and its related logic with the mapping part in the RTL design. For example, you can inspect whether the location and direction of isolation, switch, or level shifter cells is consistent with the related power rules defined in the RTL stage.

3

Using Power-Aware Debug

The following sections describe the various scenarios for using Power-Aware Debug functionality in the Verdi® Automated Debug Platform:

- [Importing CPF/UPF Files into Verdi](#)
- [Power-Aware Debug Panes](#)
- [Power-Aware Support in Other Panes](#)
- [Tracing Driver/Load on Low Power Design Elements](#)
- [Support for Power Pad Cell, Power Switch Cell, and Punch Ports in nSchema](#)
- [Power-Aware Debug Support on Temporal Flow View \(TFV\) in Transition Mode](#)
- [Displaying Annotation Values for Power Signals in the Power Source Code Pane](#)
- [Viewing and Editing UPF Source Files](#)
- [Support for the set_repeater UPF Command](#)
- [Support Pass Through connect_supply_net and connect_logic_net commands as Pass-Through Trace Result](#)
- [Power Aware Trace Driver/Load Support for SPICE Signals](#)
- [Reporting the SPICE Supply Port Connection to UPF Supply Net](#)

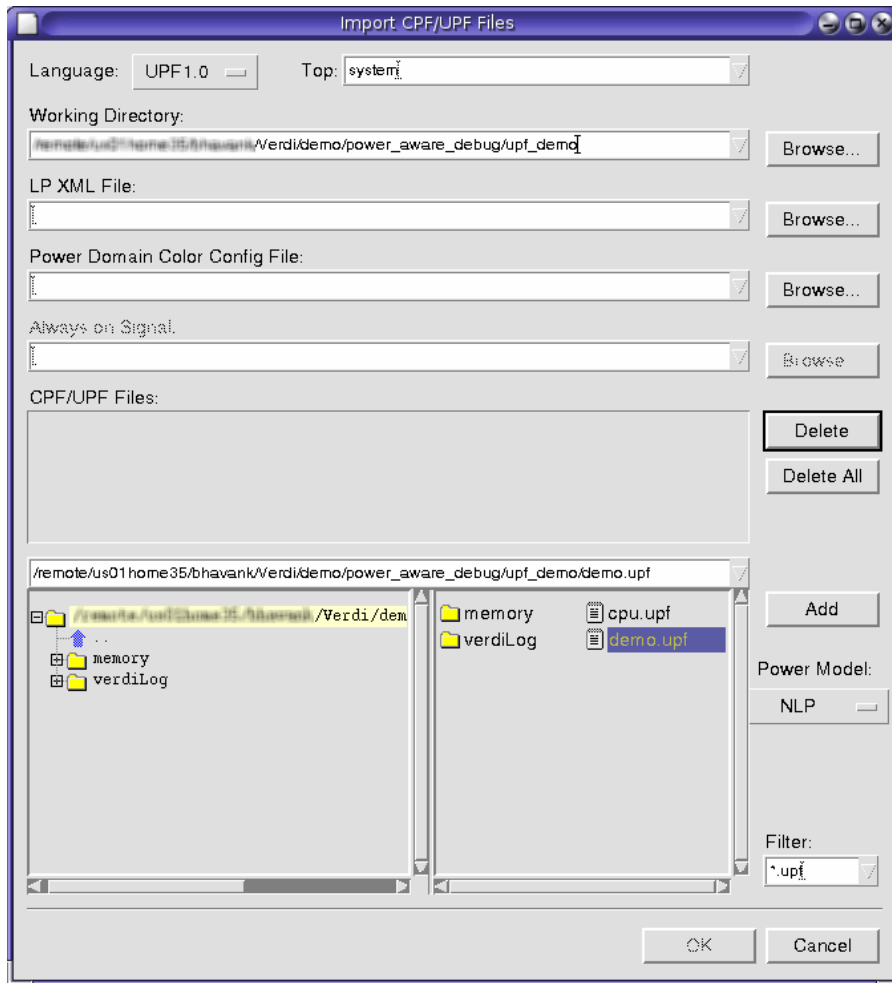
Importing CPF/UPF Files into Verdi

After the design is loaded into the Verdi platform, use the **File -> Import CPF/UPF Files** menu options in the *main* window to open the *Import CPF/UPF Files* form. The *Import CPF/UPF Files* form opens up as illustrated in [Figure 2](#).

Note:

Power license is required when a CPF/UPF file is loaded.

Figure 2 Import CPF/UPF Files Form



Verdi platform supports *UPF1.0* and *UPF 2.0* power formats.

Importing UPF Files

Select *UPF 1.0* or *UPF 2.0* option in the **Language** field and select the UPF files to load into the Verdi platform.

Alternatively, use the `-upf1.0` or `-upf2.0` command-line option to import the UPF files, and use the `-upfTop` option to specify the top scope for the imported power specification.

For more information, see the *Language Support and Compile/Import Methods* chapter in *Verdi and Siloti Command Reference*.

Starting Verdi Power Aware Debug with UFE Flow

You must specify the `-kdb` option at compilation/elaboration and the `+fsdb+power` option at runtime to use Verdi Power Aware capabilities with VCS under the UFE flow.

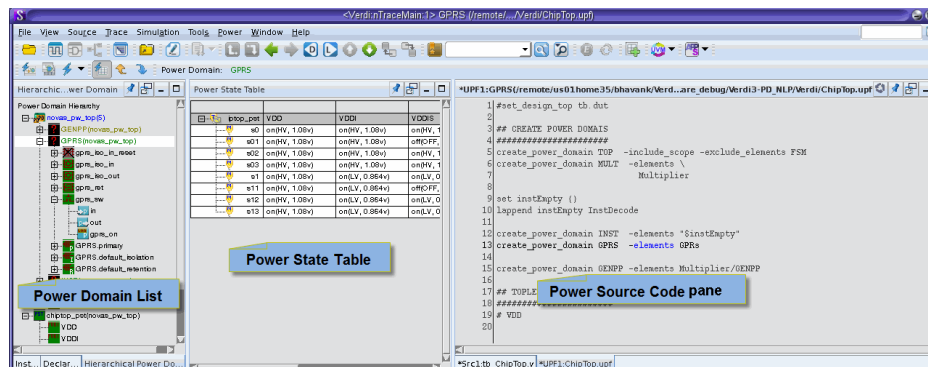
Example:

```
% vcs -sverilog -f run.f -upf demo.upf -  
debug_access+all -kdb -power_top system  
% simv +fsdb+power +fsdbfile+power.fsdb  
% verdi -ssf power.fsdb
```

Power-Aware Debug Panes

The Power-Aware Debug support is integrated with the main *nTrace* window. The power-related panes are opened in the *main* window after importing the UPF power file into the Verdi platform. Also, you can select the **Power Debug Mode** work mode from the *Welcome* page or select the **Window -> Power Debug Mode** menu option to open the power-related panes. Figure 3 illustrates an example of Power-Aware Debug panes.

Figure 3 Power-Aware Debug Panes



There are four specific Power-Aware Debug panes:

- Power Domain List
- Power State/Mode Table
- Power Source Code
- Power Map

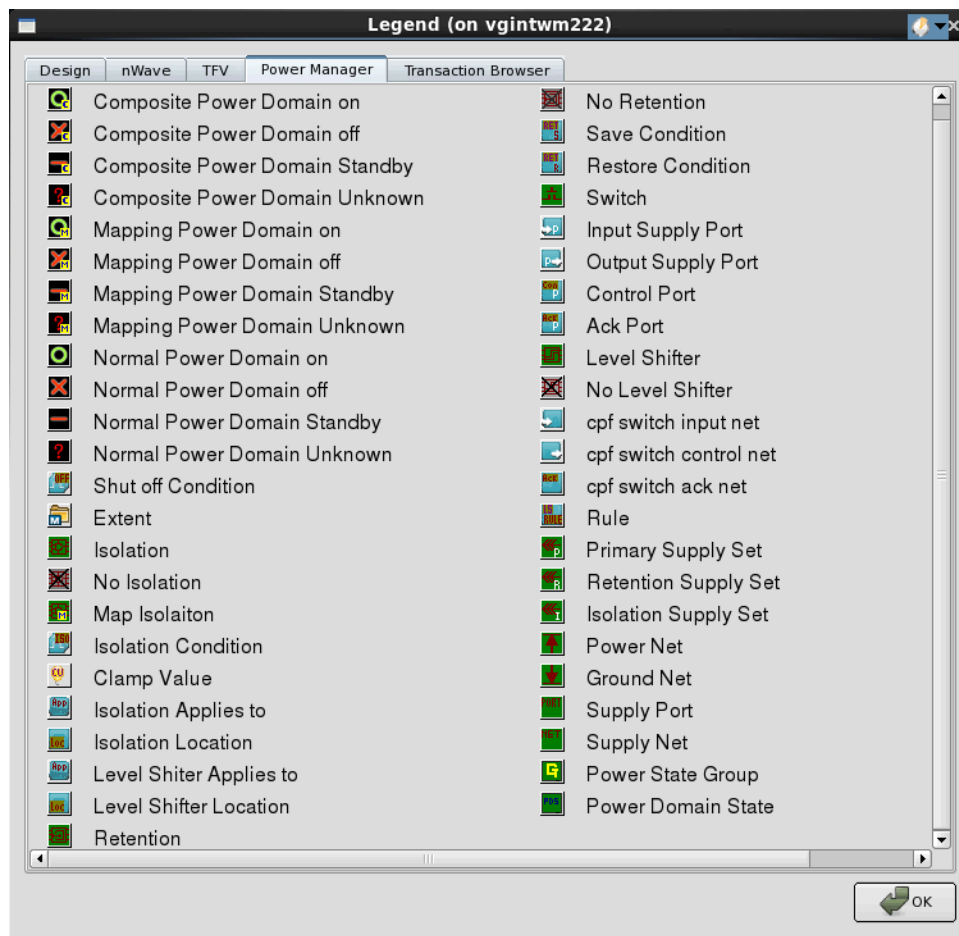
In addition, the *Source Code* pane, *Temporal Flow View* pane, *nSchema* window, and *nWave* window also support Power-Aware Debug.

Power Domain List Pane

The *Power Domain List* pane displays all the power domains defined in the design. This pane is docked to the same pane location as the *Design Browser* as a new tab. Each domain is marked with a different color. For details regarding assigning colors for different power domains and power objects, see the [Highlighting Power Domain, Power Objects, and Power Signals](#) section.

The power states for all power domains are represented by different icons. For example, the  icon represents the *Normal Power Domain On* and the  icon represents the *Normal Power Domain Off*. Use the **Help -> Legend** option and select the **Power Manager** tab to check all the icons. [Figure 4](#) shows the power icons in the *Legend* form.

Figure 4 Power Manager Legend Form



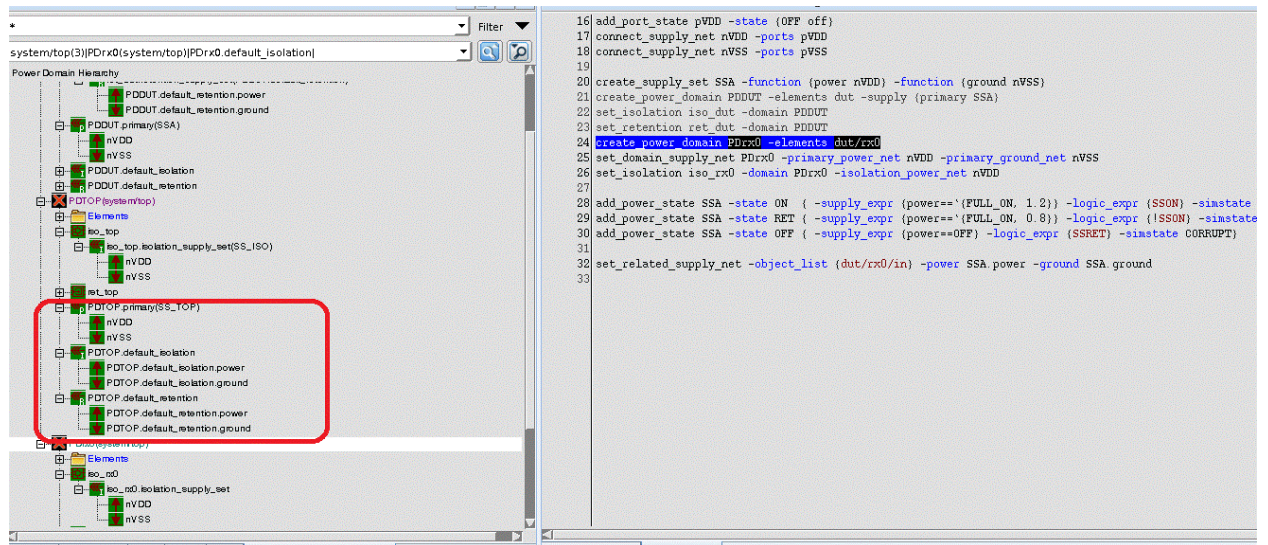
All power definitions for the selected power domain are shown in a structural way so that the related definitions can be easily found. If you load an FSDb file and click the **Source**

-> **Active Annotation** menu option, the domain status is displayed dynamically by *on*, *off*, *standby* or *unknown* icons according to the time changed in *Cursor Time* field on the toolbar.

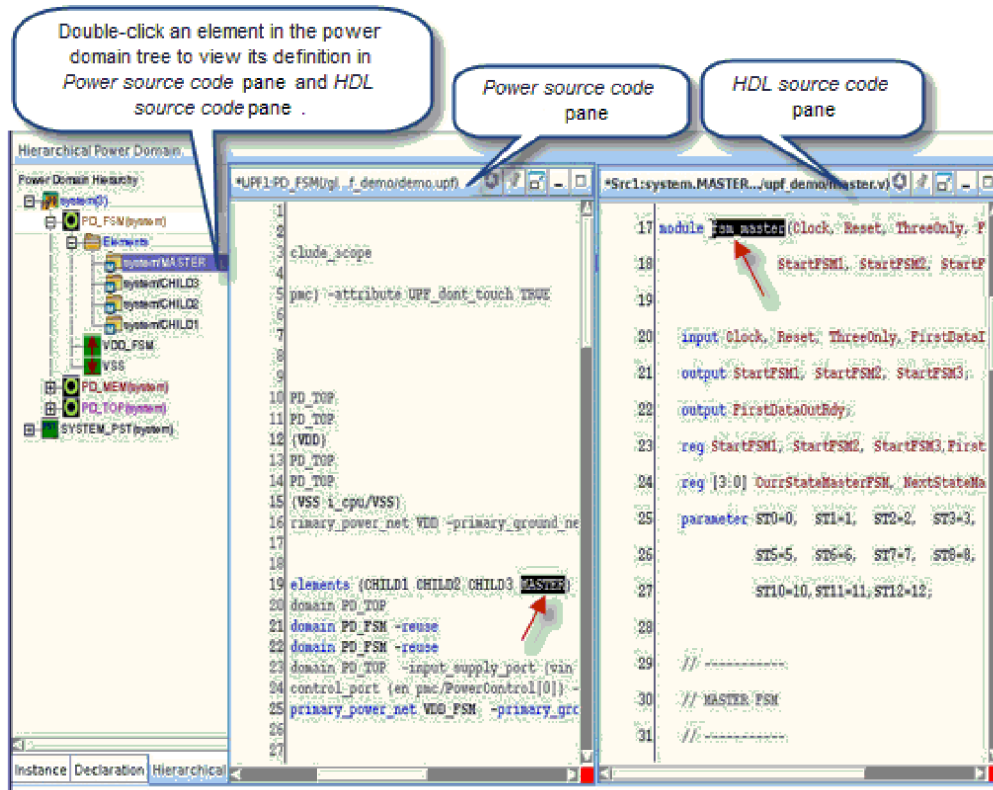
Some of the features for *Power Domain List* pane are as follows:

- The primary/default_isolation/default_retention supply set nodes appear as children nodes for a power domain tree node.
- If a supply set node is associated with a supply set, the name of the node appears as the *supply_set_name(associate_supply_set_name)* string format.

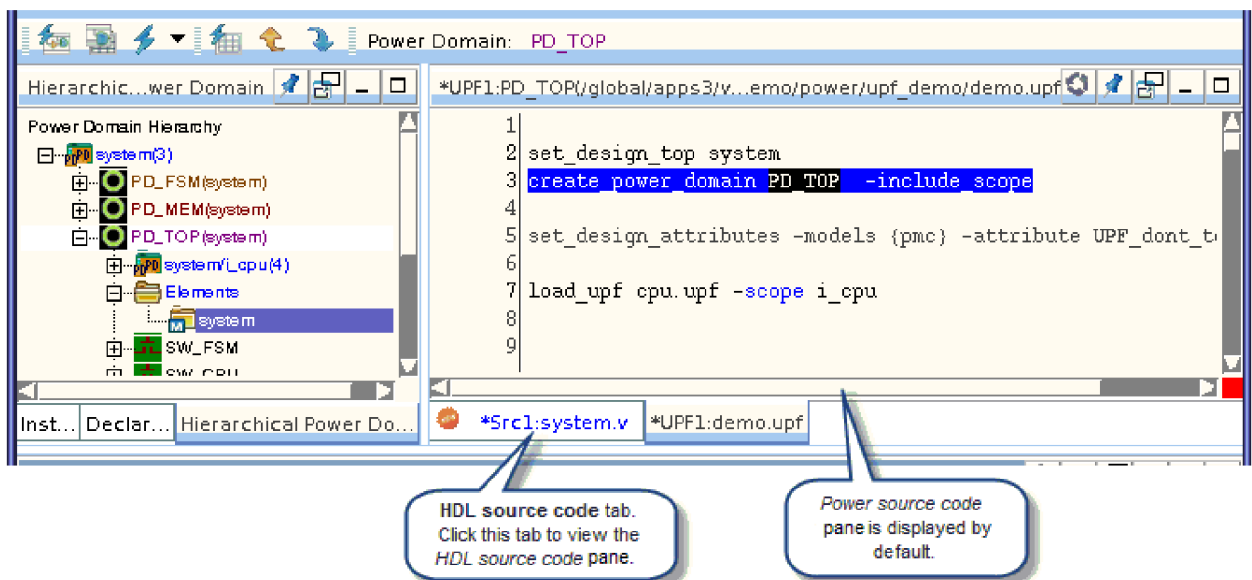
This is also illustrated in the following figure:



- You can double-click an element in the *Power Domain List* pane to view its definition in both *Power Source Code* pane and **HDL source code** pane, as shown in the following figure:

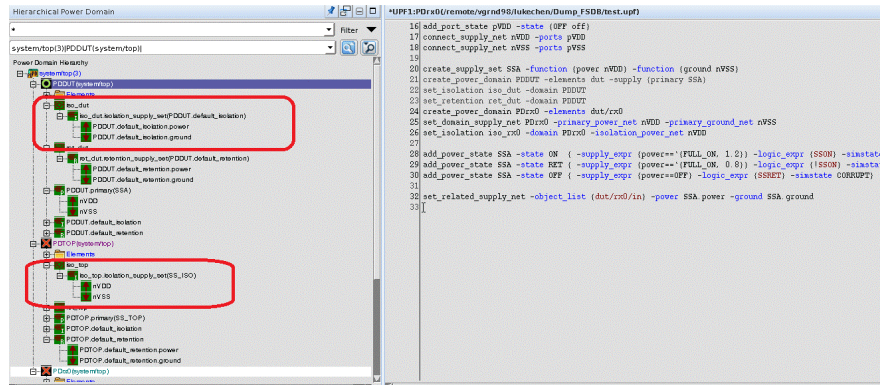


The *Power source code* pane is displayed by default. Click the **HDL source code** tab, as shown in the following figure, to view the *HDL source code* pane.



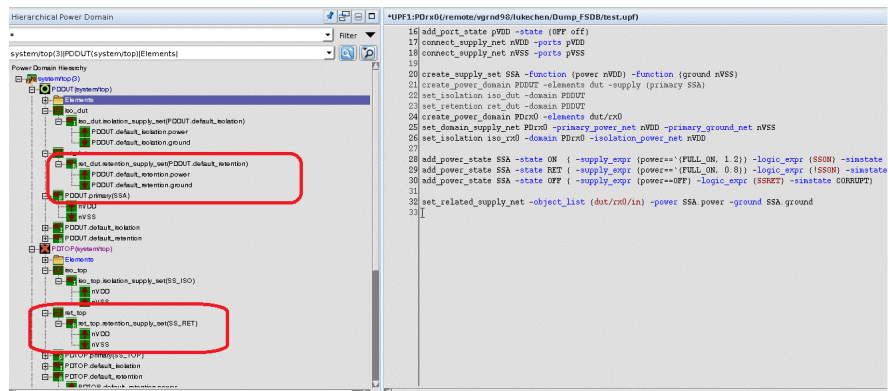
- By default, the isolation supply set node appears as the children node for an isolation policy node.

For example, if the supply set is associated with some other supply set *SS_ISO*, the associated supply set appears as *SS_ISO*, else, associated supply set appears as *domain_name.default_isolation*. The supply set appears as the *policy_name.isolation_supply_set(associate_supply_set_name)* string format. This is also illustrated in the following figure.



- The retention supply set node appears as the children node for a retention policy node, by default.

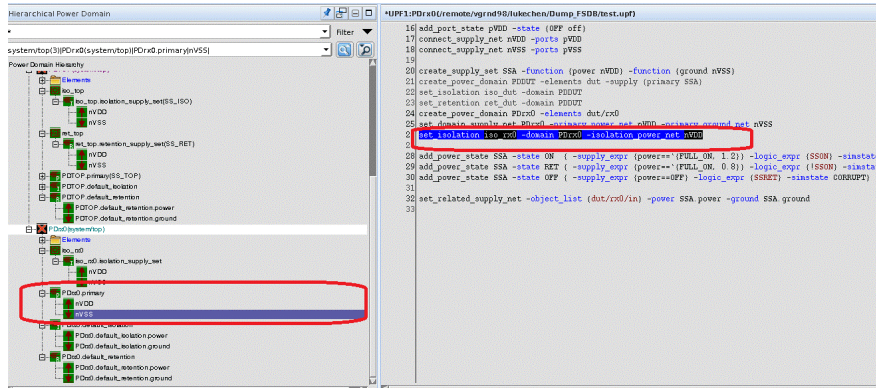
For example, if the supply set is associated with some other supply set *SS_RET*, the associated supply set appears as *SS_RET*, else, associated supply set appears as *domain_name.default_retention*. The supply set appears as the *policy_name.retention_supply_set(associate_supply_set_name)* string format. This is also illustrated in the following figure.



- The power/ground children node for a supply set node appears as *supply_set_name.power* or *supply_set_name.ground* by default. If the power/ground children nodes are overwritten by a supply net object, the node name appears as the overwritten supply net object.

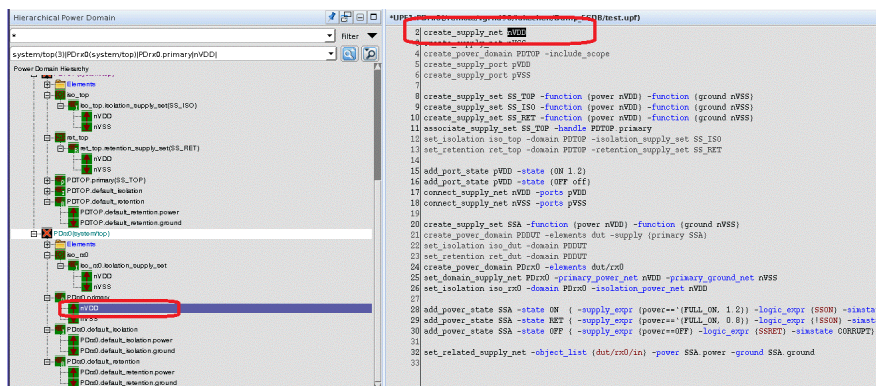
For example, for a supply set node *set1*, the child power net node appears as *set1.power* by default, and the child ground net node appears as *set1.ground* by default. If *set1.power* is overwritten by *VDD*, the child power net node appears as *VDD*. If *set1.ground* is overwritten by *VSS*, the child ground net node appears as *VSS*.

- For the supply set of a certain policy, if one of its power/ground is overwritten by a supply net, the other one is overwritten by its power domain's primary power/ground. This is also illustrated in the following figure.

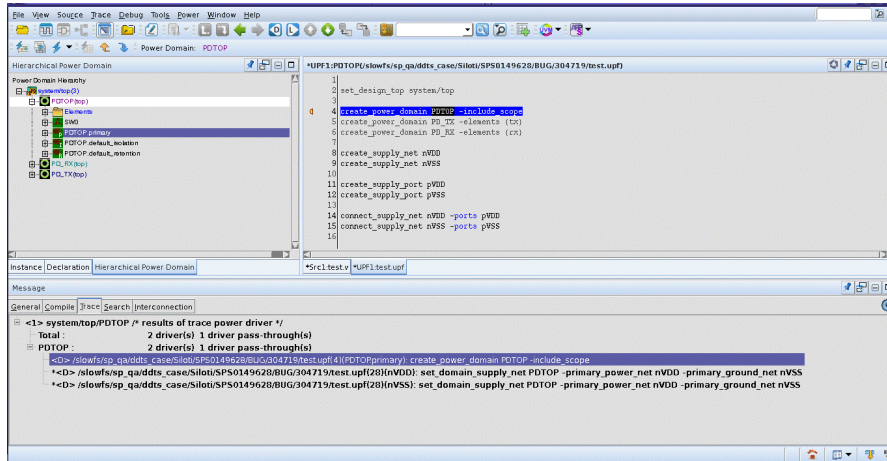


- If a supply set node is double-clicked, the power source code is synced to its declaration. If the supply set node is implicitly created by the `create_power_domain` command, the source code is synced to the `create_power_domain` command.
- If a supply net node under supply set node is double-clicked, the power source code is synced to its declaration. If it is implicitly created by the `create_supply_set` command, the source code is synced to the `create_supply_set` command. If it is implicitly created by the `create_power_domain` command, the source code is synced to the `create_power_domain` command.

This is also illustrated in the following figure.



- While performing trace driver action for a power domain object, its primary supply set and the primary supply set's power/ground net is listed as the driver result, by default. This is also illustrated in the following figure.

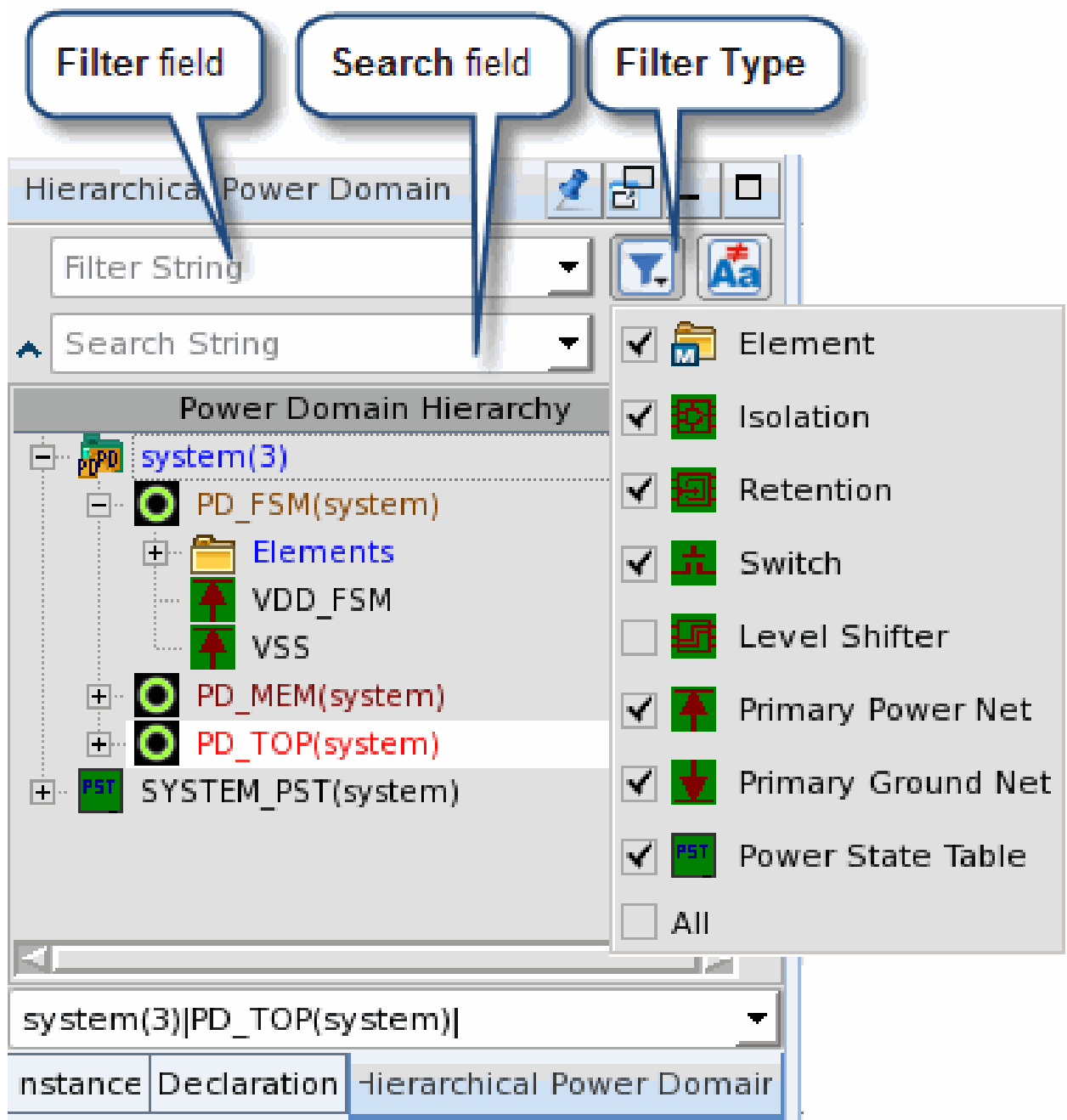


Filtering Power Aware Objects in the Power Domain Hierarchy Tree

You can filter power aware object types such as Elements, Isolation, Retention, Switches, Level Shifters, Primary Power Net, Primary Ground Net, and Power State Tables in the *Power Domain Tree* pane based on the object types mentioned in [Figure 5](#).

To filter the power objects, enter a string in the **Filter** field, select the filter type by clicking the **Filter Type** icon  on the right, and press the **Enter** key. Tree nodes (including their parent nodes) matching the input string and the filter type has their power domain nodes displayed in the *Power Domain Tree* pane.

Figure 5 Filter Types and Search in the Power Domain Tree

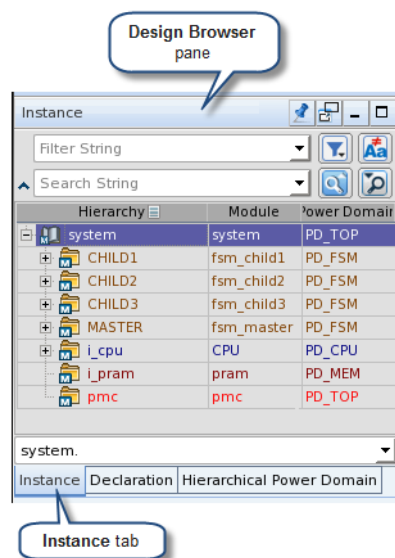


All the object types, except Level Shifters, are displayed by default in the Power Domain Hierarchy. To view Level Shifters in the Power Domain Hierarchy, select the **Level Shifter** option from the *Filter Type* list.

Design Browser Pane

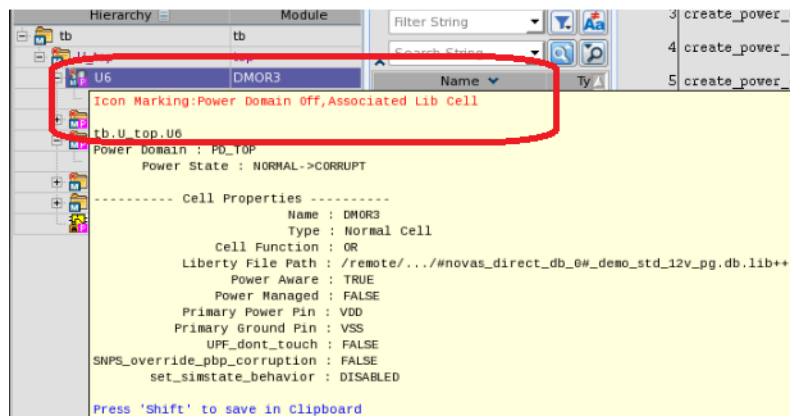
The *Design Browser* pane provides design hierarchy information. Click the *Instance* tab as shown in the following figure to view the *Design Browser* pane:

Figure 6 *Design Browser Pane*



Place the mouse pointer on an instance element to view tooltip with the relevant power domain and power state information. The icon marking information of an instance element is highlighted in red and is displayed on top of the tooltip as shown in the following figure:

Figure 7 *Viewing Power Domain Information in the Design Browser Pane*



The following type of icon marking information is supported:

Icon Marking: Power Domain Off, X-propagation, Associated Lib Cell

The icon marking format is as follows:

Icon Marking: Mark1....

For an instance element with multiple marking information, the markings are separated with "," as follows:

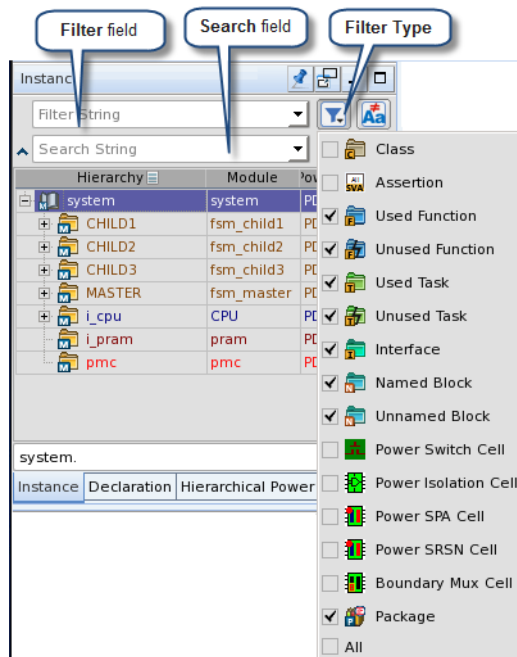
Icon Marking: Mark1, Mark2....

Filtering Power Elements and Objects in the Design Browser Pane

You can filter power elements and objects such as Class, Assertion, Used Function, Unused Function, Used Task, Unused Task, Interface, Named Block, Unnamed Block, Power Switch Cell, Power Isolation Cell, Power SPA Cell, Power SRSN Cell, Boundary Mux Cell, and Package in the *Design Browser* pane.

To filter the power elements and objects, enter a string in the **Filter** field, select the filter type by clicking the **Filter Type** icon  on the right, and press the **Enter** key.

Figure 8 Filtering Power Elements and Objects in the Design Browser Pane



The power switch names in the instance tree are suffixed with (SNSP_UPF_UniqPowerSwitch).

The **Display Module Name** option in the *Preferences* form (**Source Code -> Design Tree**) allows you to control the display of the power switch cell names in the instance tree.

Power State/Mode Table Pane

The *Power State/Mode Table* pane shows the power state/mode table of the imported design. This pane is docked to the same area as the *signal list* pane as a new tab. The table is enabled by default, if there is power mode/power state information (refer to the Table below) in the imported UPF/CPF file.

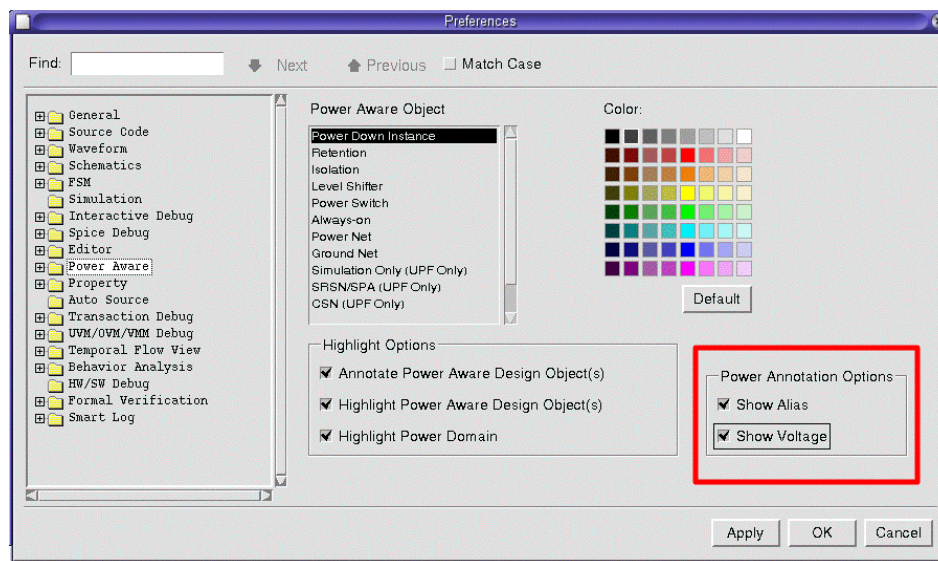
If the *Power State/Mode Table* pane is not opened (or has been closed), invoke the following menu option to open it:

- **Power -> Show Power State Table** (if UPF file is loaded)

Power Standard Format	Command
UPF	add_port_state/add_power_state add_pst_state create_pst

If the **Source -> Active Annotation** option is turned *on*, the active state is highlighted. To view the voltage values in the Power State Table, the **Power Annotation Options** in *Power Aware* tab of the *Preferences* form (invoked with **Tools -> Preferences**) must be turned *on*. Refer to the preference setting displayed in [Figure 9](#).

Figure 9 Power Annotation Options



Color Coding in Power State Table

- A row highlighted with an orange background color signifies that the signal is in active power mode.
- A row highlighted with red background color signifies that Verdi cannot find the combination of power status to meet the predefined *Power State Table* in UPF.
- Multiple rows are highlighted because these modes are differentiated only in voltage. Such modes cannot be distinguished in the Verdi platform because voltage value is not recorded in the FSDB file (simulator dependency).
- Text in red color signifies that Verdi cannot find the exact value of a signal from FSDB. For example, as illustrated in [Figure 10](#), the font color for *on(HV, 1.08v)* is red, which signifies that the value of *VDDMS* is not the exact value.

Figure 10 Color coding in Power State Table

	dutchipstn_pst	VDD(HV, 1.08v)	VDDI(HV, 1.08v)	VDDIS(HV, 1.08v)	VDD3(HV, 1.08v)	VDDG3(HV, 1.08v)	VDDM(HV, 1.08v)->(#undefined, 1,	VDDMS(HV, 1.08v)->(#undefined, 1,
s0	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)
s01	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	off(OFF, 0v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)
s02	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	off(OFF, 0v)	on(HV, 1.08v)	on(HV, 1.08v)
s03	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)	off(OFF, 0v)
s1	on(HV, 1.08v)	on(LV, 0.964v)	on(LV, 0.964v)	on(LV, 0.964v)	on(LV, 0.964v)	on(LV, 0.964v)	on(HV, 1.08v)	on(HV, 1.08v)
s11	on(HV, 1.08v)	on(LV, 0.964v)	off(OFF, 0v)	on(LV, 0.964v)	on(LV, 0.964v)	on(LV, 0.964v)	on(HV, 1.08v)	on(HV, 1.08v)
s12	on(HV, 1.08v)	on(LV, 0.964v)	on(LV, 0.964v)	on(LV, 0.964v)	off(OFF, 0v)	on(HV, 1.08v)	on(HV, 1.08v)	on(HV, 1.08v)
s13	on(HV, 1.08v)	on(LV, 0.964v)	on(LV, 0.964v)	on(LV, 0.964v)	on(LV, 0.964v)	on(HV, 1.08v)	on(HV, 1.08v)	off(OFF, 0v)
illegal	FULL_ON(HV, 1.08v)	FULL_ON(HV, 1.08v)	FULL_ON(HV, 1.08v)	FULL_ON(HV, 1.08v)	FULL_ON(HV, 1.08v)	FULL_ON(HV, 1.08v)	FULL_ON(HV, 1.08v)-FULL_ON	FULL_ON(HV, 1.08v)-FULL_ON

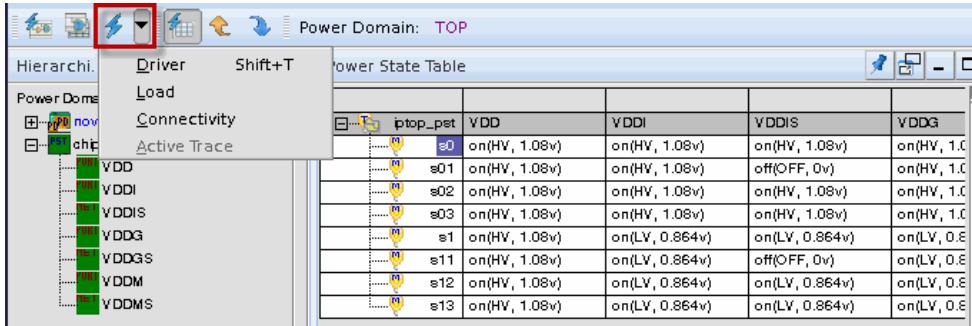
Power Source Code Pane

The *source code* pane of the *main* window shows the source code of the imported UPF power file in a separate tab. The content is changed accordingly when double-clicking different items in the *Power Domain List* pane. Enabling the **Source -> Active Annotation** menu option annotates the FSDB value for signals under the source code, if the FSDB file is loaded into the Verdi platform.

Tracing capabilities including tracing HDL driver/load and tracing power driver/load are also available in the power source code pane:

- To trace the HDL driver/load, click the **Trace -> Trace Driver/Trace Load** option on the toolbar. The traced results are displayed in the *design source code* pane.
- To trace the power driver/load/connectivity, select a signal in the *power source code* pane, right click, and select the **Trace Power Driver/Trace Power Load/Trace Power**

Connectivity option. Or, select a signal in the *power source code* pane, click the **Trace Power** icon on the toolbar, and select **Driver**, **Load**, or **Connectivity** option. Refer to the following figure:



The cursor jumps to the corresponding source code, and the traced results are displayed in the **Trace** tab of the *Message* pane.

Definitions of the power driver/load are as follows:

- **Power Driver:** These commands impact the HDL signal. For example, if an HDL signal is defined in a retention rule command, it implies that when the associated power domain is turned *off*, the HDL signal becomes a retention signal (impacted). Therefore, this retention rule is one of the traced results of the HDL signal's Power Driver.

The Power Driver trace results include the following:

UPF

- `create_power_domain`: if the HDL signal belongs to the power domain.
- `set_isolation`: if the HDL signal is the isolation signal of the command.
- `set_retention`: if the HDL signal is the retention signal of the command.
- `set_level_shifter`: if the HDL signal is the level shifter signal of the command.

- **Power Load:** These commands are impacted by the HDL signal. For example, if the HDL signal is defined in a shut off condition, the signal turns off the associated power domain when the value matches the condition. This implies the power domain is impacted by the HDL signal. Therefore, this shut off condition is one of the traced results of the HDL signal's Power Load.

The Power Load trace results include the following when the HDL signal is contained in the options:

UPF

- `create_power_switch -control_port`
- `set_isolation_control -isolation_signal`

- `set_retention_control -restore_signal`
- `set_retention_control -save_signal`

Enabling UPF Commands of Inactive Power Domains in the Power Source Code View

Verdi allows you to debug the UPF commands of inactive power domains in the *Power Source Code View*. Debug functions like RMB, viewing annotations, and drag-and-drop are supported on these commands.

The **Show Previous/Show Next** command searches the commands in the previous/next line by file in the *Power Source Code View*.

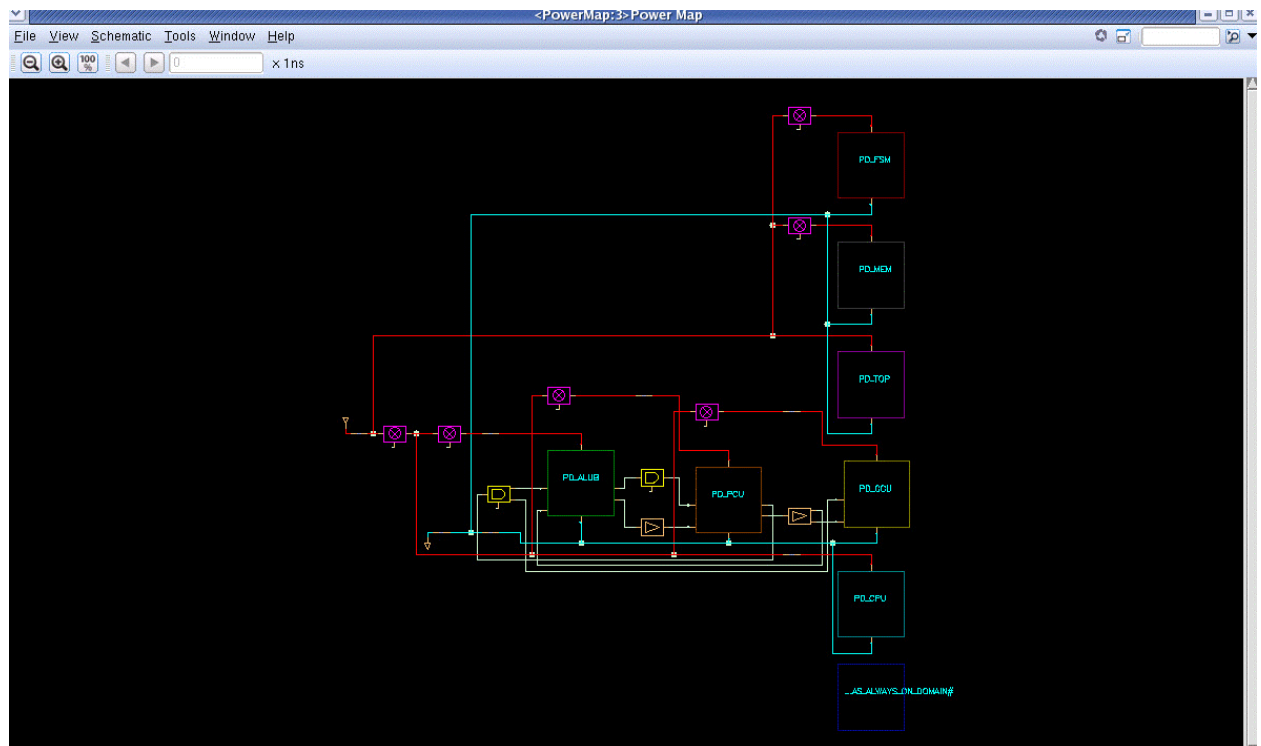
Power Map Pane

The *Power Map* pane provides the capability to visualize power intent in a flattened schematic view. The schematic view shows the structure of the power design (see

[Figure 11](#)), where the  icon represents an *Isolation* command, the  icon

represents a *Level-shifter* command, and the  icon represents a power switch cell. The *Power Map* pane provides support to highlight power domains. For details, see the [Highlighting Power Domain, Power Objects, and Power Signals](#) section.

Figure 11 Power Map Pane

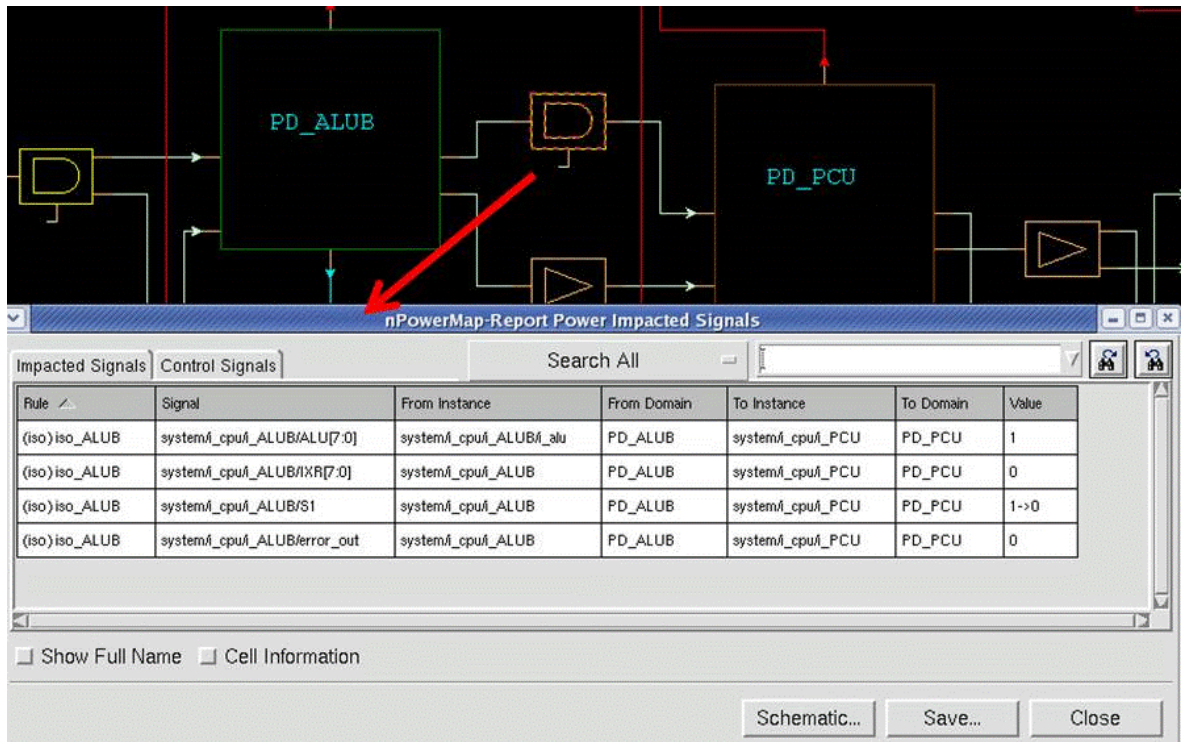


To invoke the full power mapping view, select the **Power -> New Power Map -> Full Power Map** menu option from the *main* window or the **Tools -> Power Map** menu option from the *nSchema* window to invoke the *Power Map* pane (UPF should be imported). The *Power Map* pane is docked to the same area as the source code pane as a new tab.

Browse through the window to understand the structure of the power design. Drag any of the icons like isolation, level-shifter, or switch cell (, , and ) to the *power source code* pane to view the definition of the related rules. Similarly, drag any power domain from the *Power Map* pane to other power related panes to view the power domain's definition. If the *Power Map* pane is overlapped with the *power source code* pane, you cannot drag the cell icons to the *power source code* pane. You must move either of the panes to dock to another location, so that it is possible to drag-and-drop between the panes.

To check the related signals of isolation or level-shifter rule, select the related icon, then select the **Schematic -> Report Selected Impacted Signals** option to open a form summarizing the impacted signals of the selected rule. For example, see [Figure 12](#). Alternatively, select the **Schematic -> Report Total Impacted Signals** option to open a form summarizing the total impacted signals.

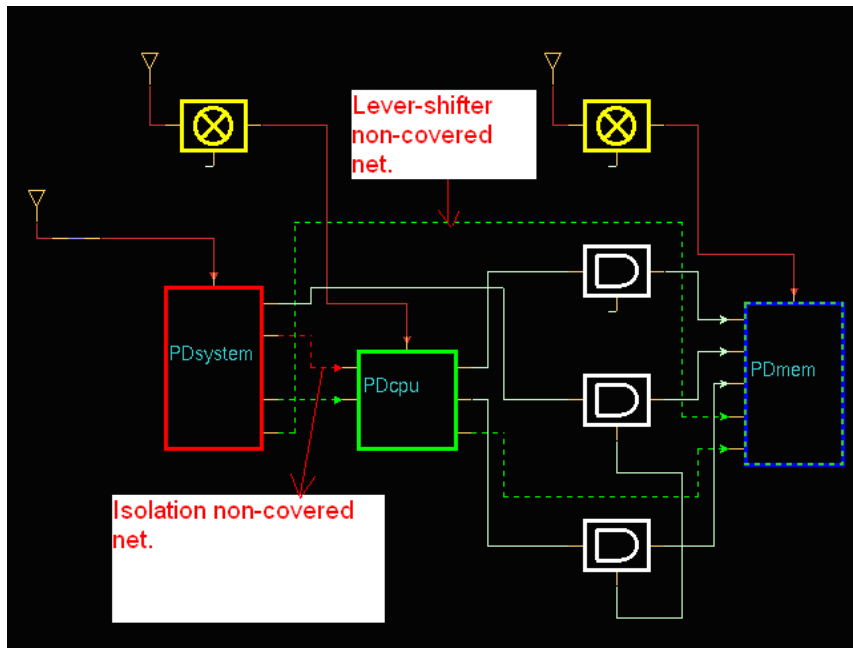
Figure 12 Impacted Power Signals of the Isolation Rule



The *Power Map* pane can validate all signals in between power domains to be guarded with isolation rules if a possibility exists (see [Appendix A: Power Map Validation](#)) that the domain a signal is coming from (*from* domain) is *off* while the domain the signal is going to (*to* domain) is not. In such a case, unexpected results could propagate through such signals from the *from* domain to the *to* domain. Similarly, signals are suggested to be guarded with level-shifter rules if the signals' *from* and *to* domains are both *on/standby* states, but the domains' voltages are different.

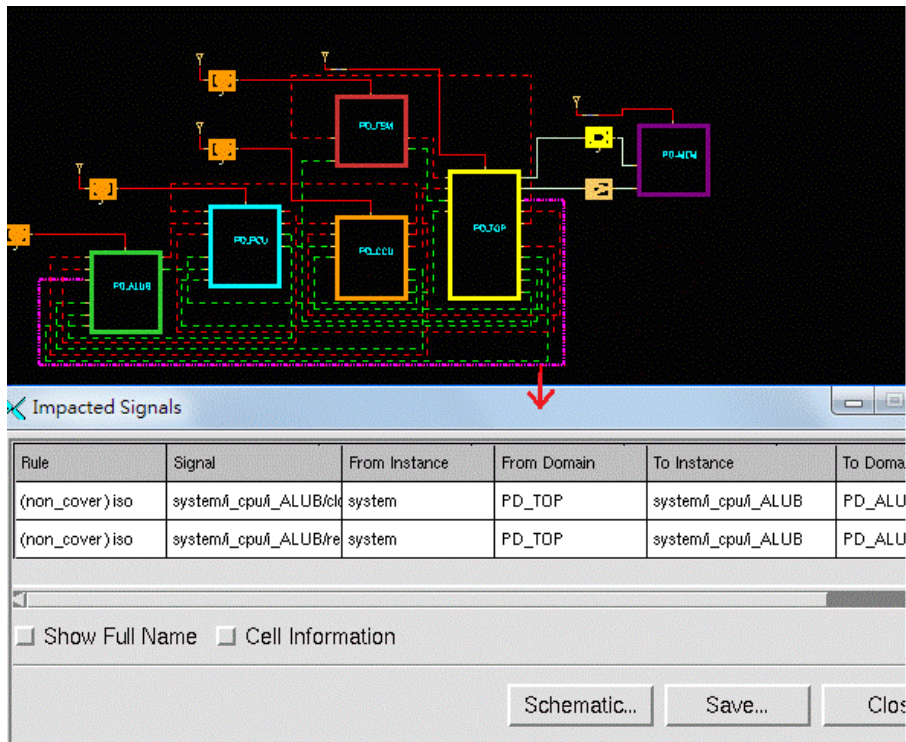
The checking is performed when the *Power Map* pane is first invoked or when the related checking options (**View -> Non-covered Signals -> Isolation Signals/Level-shifter Signals**) are turned *on*. The results are displayed as non-covered nets (shown by dotted lines). A non-covered net contains all non-covered signals on the path from domain X to domain Y. The red dotted lines represent the non-covered isolation nets (it is suggested to apply isolation rules); the green dotted lines represent the non-covered level-shifter nets (it is suggested to apply level-shifter rules). Refer to [Figure 13](#).

Figure 13 *Non-Covered Signals Shown by Dotted Lines*



Select any of the non-covered nets and select the **Schematic -> Report Selected Impacted Signals** option to see all the non-covered signals belonging to the selected non-covered net (see [Figure 14](#)). Or select the **Schematic -> Report Total Impacted Signals** option to view all impacted signals for the whole design.

Figure 14 Impacted Power Signals of Non-Covered Isolation Net



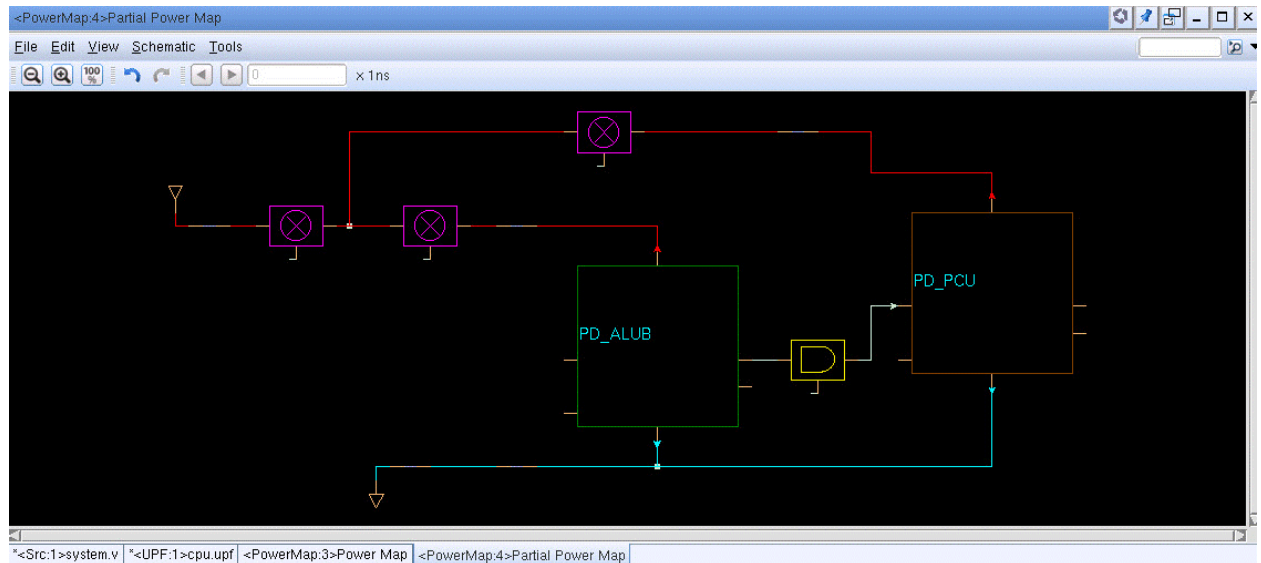
Double check the non-covered signals to decide whether you need to apply isolation or level shifter rules accordingly (they may be missed during the power design). To turn off the automatic non-covered nets checking, select the **View -> Non-covered Signals -> Isolation Signals/Level-shifted Signals** option to toggle them off.

For details regarding the partial *Power Map* pane, see the [Partial Power Map](#) section. For complete details about the usage of the *Power Map* pane, see the *Power Aware Debug* chapter in *Verdi and Siloti Command Reference*.

Partial Power Map

You can observe and trace the power network in the partial power mapping view for specific modules. To invoke partial power mapping view, select power domain, power switch, power net, isolation and level-shifter cells (including rule-cover and cell-cover cells) from the *power source code* pane or the *Power Map* pane, and use the **Power -> New Power Map -> Partial Power Map** menu option from the *main* window or the **Tools -> Open Partial Power Map** option from the existing *Power Map* pane (as illustrated in [Figure 15](#)).

Figure 15 *Partial Power Map Pane*



After invoking the *Partial Power Map* pane, right-click a power domain and select **Show Power Network for Selected Domain(s)** option to display the power network connection. Power network goes through the power switch and power port, and ends at the original power net. For example, refer to [Figure 16](#), right-click the *PD_PCU* power domain and select the **Show Power Network for Selected Domain(s)** option. The partial *Power Map* pane shows that the *VDD* power supply net goes through two power switch cells and connects to the *PD_PCU* power domain. Refer to [Figure 17](#) to view the results for *PD_ALUB* power domain using this option. *PD_ALUB* and *PD_PCU* power domains have the same source of power supply and power ground.

Figure 16 Show PD_PCU Power Network

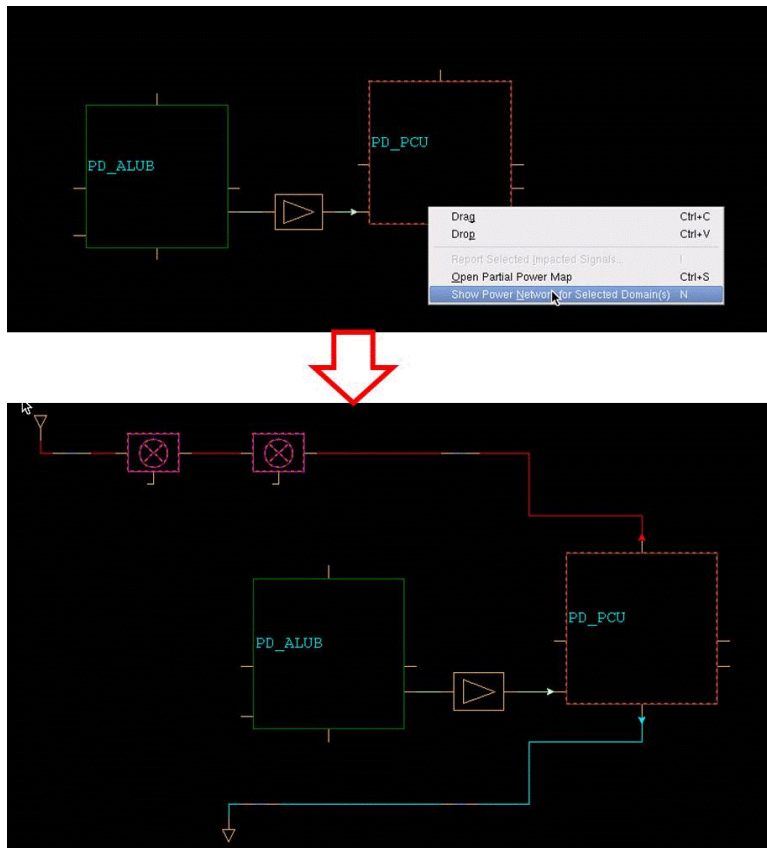
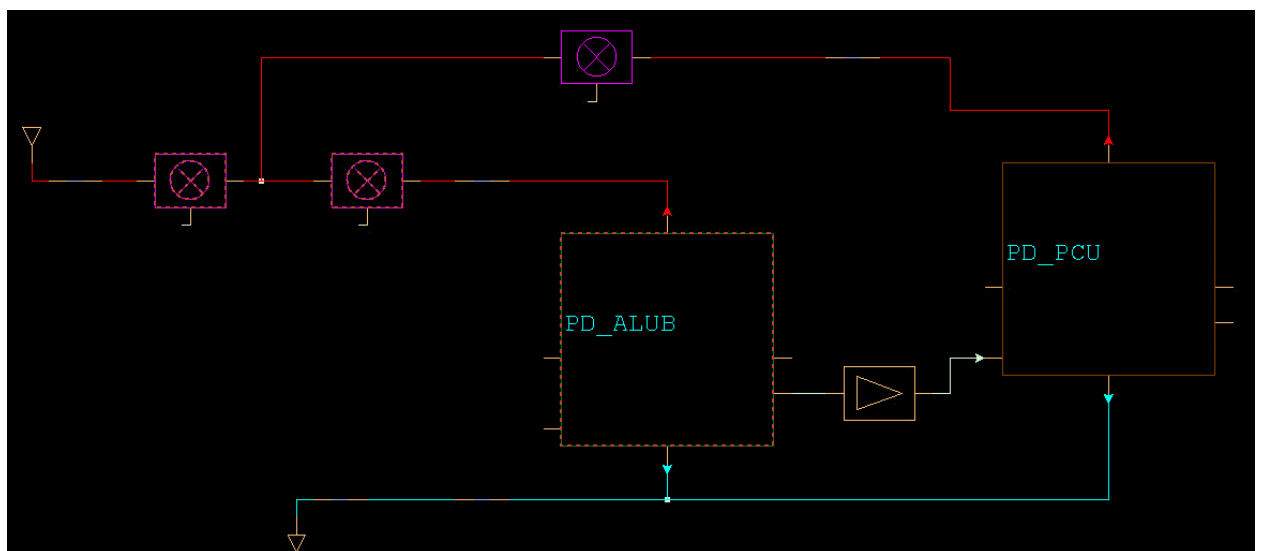


Figure 17 Show Power Network

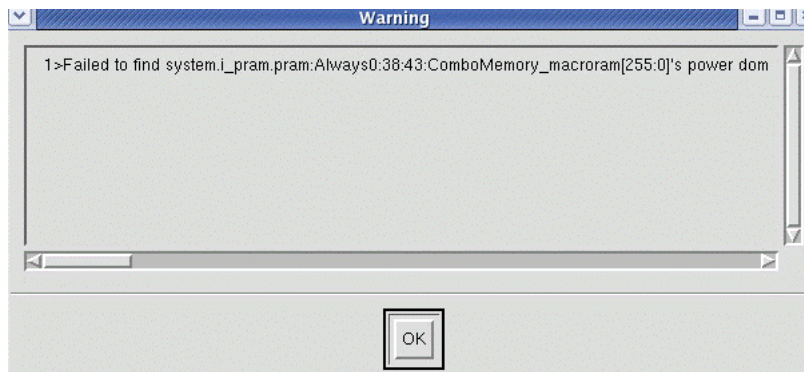


After viewing the power network, the *Undo* option can be selected by clicking the  icon in the toolbar to return the operation back to previous step. The **Redo** option can be selected by clicking the  button to redo the operation.

Power cells can be dragged and dropped to the existing *Partial Power Map* pane. The following additional options are available to perform various actions in *Power Map* pane:

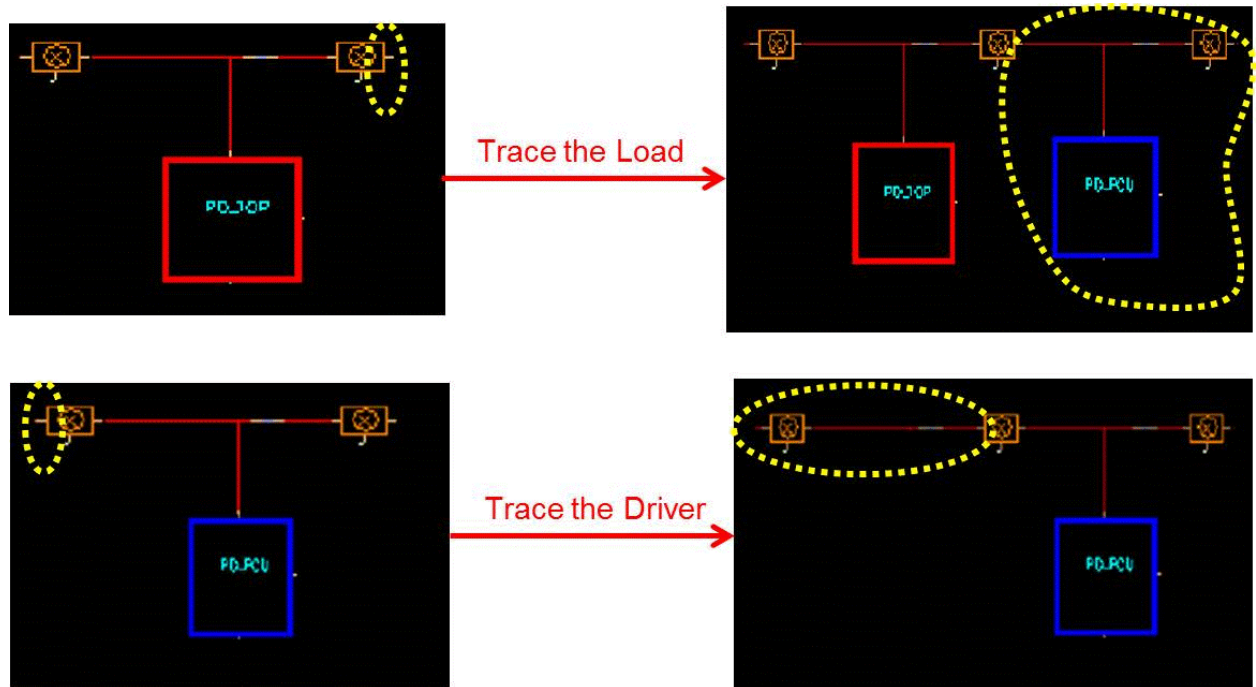
- Power Switch commands: Adds the switch and its domains to the current *Power Map* pane and highlights the switch cell.
- Isolation/Level-shifter rule commands: Adds the target and source power domains of the rule to the current *Power Map* pane and highlights all relative Isolation/Level-shifter cells extracted from this rule.
- Power domain: Adds this power domain supply network to the current *Power Map* pane and highlights the power domain cell.
- Power port: Finds the port in the current *Power Map* pane.
- Power supply net: Adds the connected power domain and power switch to the current *Power Map* pane and highlights the power supply cell.
- HDL instance: Finds the power domain that the instance belongs to. If the power domain exists in the *Partial Power Map* pane, it is highlighted; otherwise, a warning message is displayed to indicate that the power domains cannot be found.

Figure 18



- HDL signal: Finds the impacted Isolation/Level-shifter rule-cover cell or power domain. If the impacted cell or power domain does not exist, a warning message is displayed.
- Others: If the dropped power object cannot extract or trace any results in the *Partial Power Map* pane, a warning message is displayed to indicate that adding the power object to the *Partial Power Map* pane failed.

To trace the power switch, double-click the input pin or output pin. The loading result is displayed for the output pin and the driver result is displayed for the input pin. Tracing passes through the power port and ends at a power domain or a power switch.



Note:

All the existing *Partial Power Map* panes are closed when the **File -> Reload Design** or **File -> Reload CPF/UPF Files** menu option is used. **Save Session** and **Load Session** functions do not support *Power Map* panes.

Power-Aware Support in Other Panes

The following sections explain the support for power-aware debug in other panes:

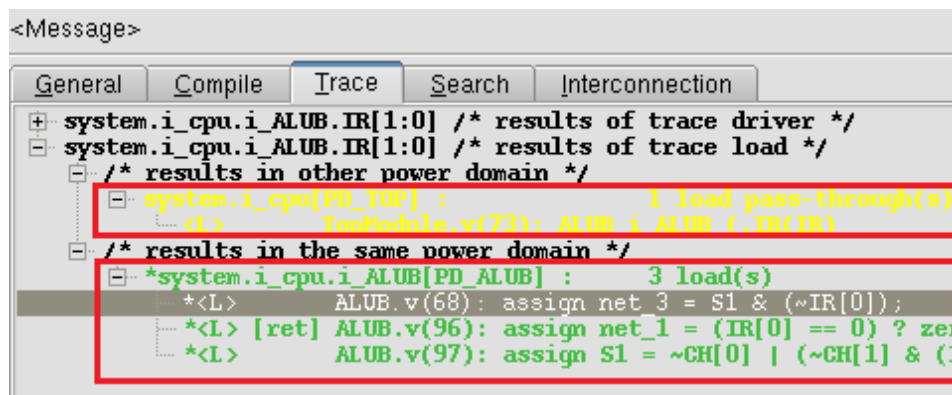
- [Design Browser and Source Code](#)
- [Watch Tab](#)
- [nSchema](#)
- [nWave](#)
- [Temporal Flow View](#)

Design Browser and Source Code

When tracing drivers or loads in *nTrace*, some additional messages are added to the *Trace* tab of the *Message* pane to indicate whether the result belongs to the same power domain or a different one. The results are colored to match its power domain settings, as illustrated in [Figure 19](#). Also, in the source code pane indicator area, all pieces of the result that may be impacted by the power states are highlighted with power impacted symbols. The meaning of these symbols is as follows:

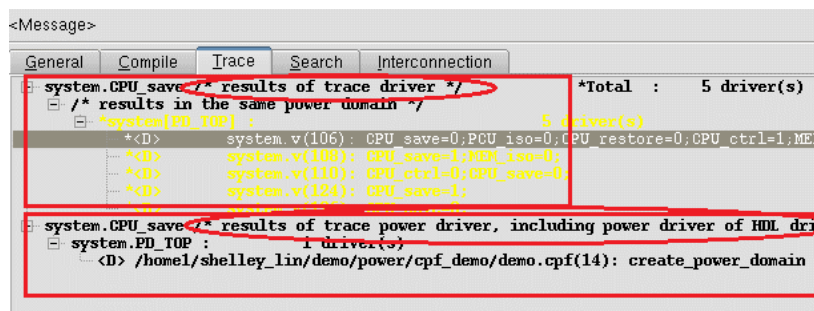
-  signifies that the driver result (HDL signal) could be impacted by a power state.
-  signifies that the driver/load result (HDL signal) could be impacted by a power state.
-  signifies that the load result (HDL signal) could be impacted by a power state.

Figure 19 Additional Messages in the Message Pane



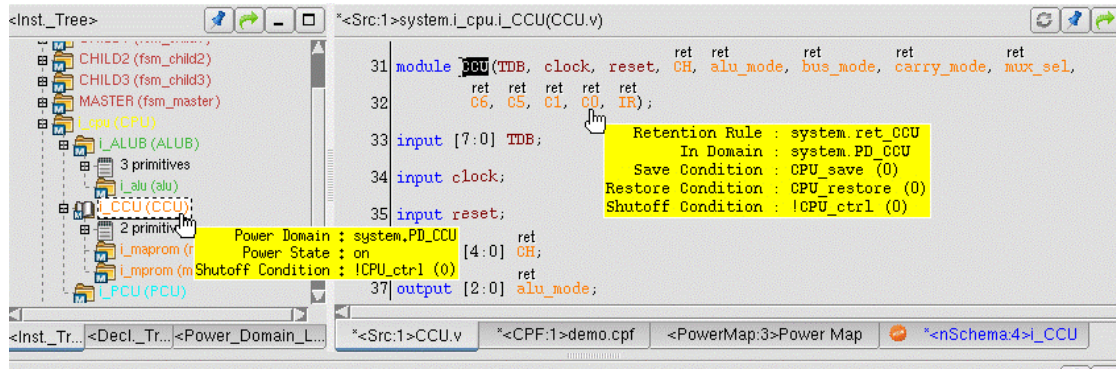
You can turn on the **Trace across Power Design** toggle option (**Trace -> Trace across Power Design**) to enable all trace options (such as, **Trace -> Driver, Load, Fan-in**) to include trace results from the power design (see [Figure 20](#)).

Figure 20 Trace Across Power Design



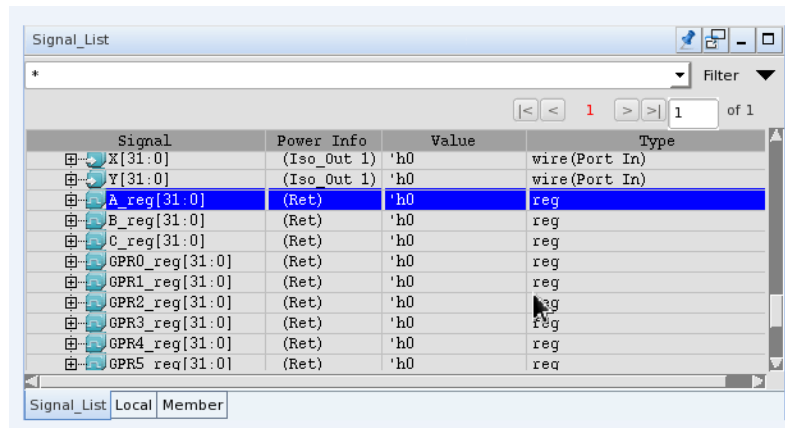
A tip with power information is also supported in the *main* window. When the mouse cursor is placed over a node in the *Design Browser* pane, the node's domain name and power state value is displayed in the yellow tip. When the mouse cursor is placed over a retention/isolation/level-shifted signal in the *Design Source Code* pane, the retention/isolation/level-shifted rule is displayed in the yellow tip, as illustrated in [Figure 21](#).

Figure 21 Power Domain and Retention Signal Tip in nTrace



Select the **View -> Signal List** menu option in the *main* window to display the *Signal List* pane and view the signal's power-related information, like power signal type, and clamp value in the *Power Info* column of the *Signal List* pane. Refer to [Figure 22](#).

Figure 22 Show Power Information in Signal List

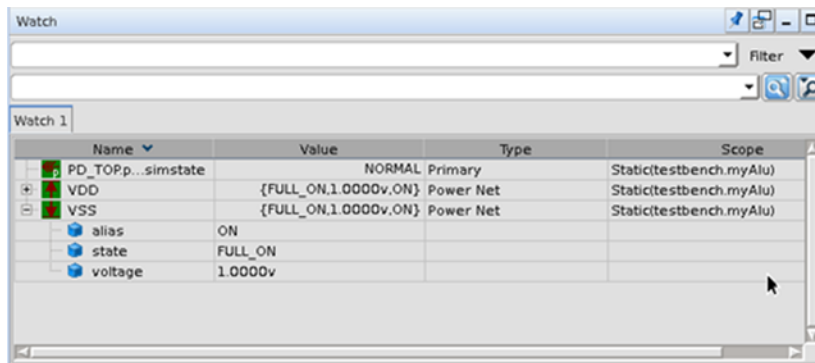


In addition, both the *Design Browser* pane and the *Source Code* pane support highlighting power domains and objects. For details, see the [Highlighting Power Domain, Power Objects, and Power Signals](#) section.

Watch Tab

You can use *Watch* tab to debug power signals. Verdi allows you to observe power signals in the *Watch* tab as shown in the following figure. You can add power signals and scopes to the *Watch* tab and observe the state and value changes as the simulation time changes.

Figure 23 Power Signals in the Watch Tab



- The **Name** field displays the hierarchical name of a signal. If the node is expandable, the children nodes display the short name.
- The **Value** field displays the value as *{state, alias, voltage}*, if *alias* and *voltage* are available.
- The **Type** field displays the signal type (for example, power domain, supply set, and so on).
- The **Scope** field displays the scope of the signal.

The following methods are provided to add a power signal to the *Watch* tab:

- Use the **Add to Watches** right-click option in the *Source Code* pane and *Power Domain Hierarchy* pane.
- Other components can drag-and-drop power signals to the *Watch* tab, if they have power signals.

The following power signals are supported:

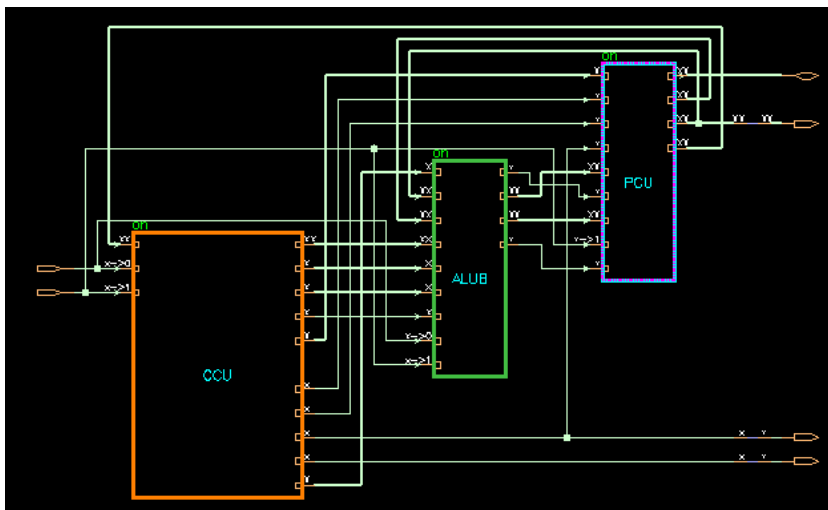
```
POWER_DOMAIN, POWER_STATE_TABLE, POWER_NET, POWER_PORT, POWER_SUPPLY_SET,
POWER_INSTR_SIGNAL, POWER_SWITCH, LOGIC_NET, LOGIC_PORT,
POWER_STATE_GROUP
```

nSchema

The *nSchema* window supports highlighting power domains. For details, see the [Highlighting Power Domain, Power Objects, and Power Signals](#) section. The highlight can also be turned *on/off* with the **View -> Highlight Power Aware Object -> Power Domain Color** and **View -> Highlight Power Aware Object -> Power Aware Object Color** options in the *nSchema* window.

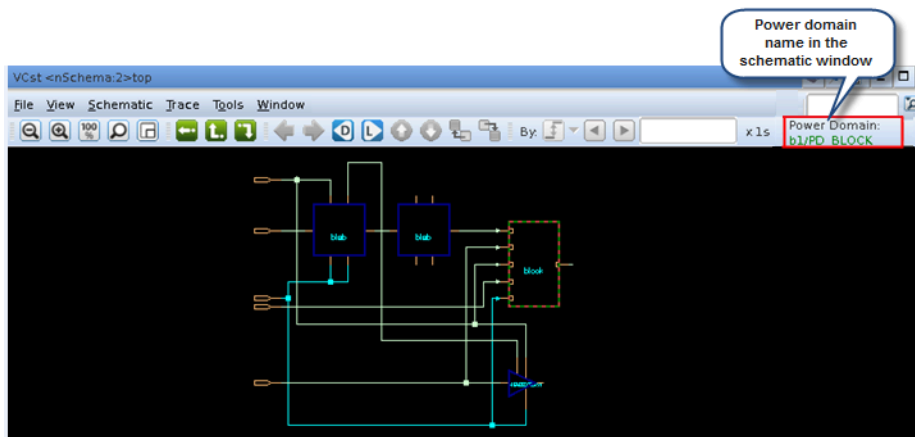
The power state is annotated on top of each instance after turning *on* the **Schematic -> Active Annotation** option, and the instance with an *off* state is grayed out, as illustrated in [Figure 24](#).

Figure 24 Power State Annotation in nSchema



Verdi displays root supply name along with the Power Domain name in the *nSchema* window, as illustrated in [Figure 25](#).

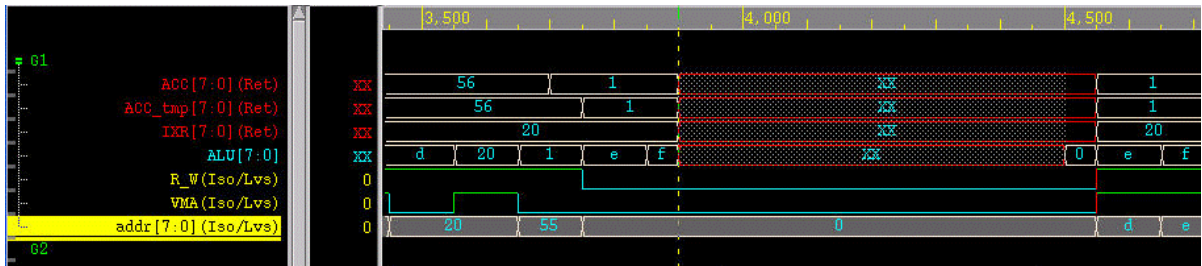
Figure 25 Power Domain Name in nSchema



nWave

Turn on the **View -> Power -> Mask Power Off** option in the *nWave* window to mask out the liberty-affected, *set_related_supply_net* (SRSN) and power-off range for HDL signals in the waveform view. Figure 26 shows an example for the power-off mask capabilities.

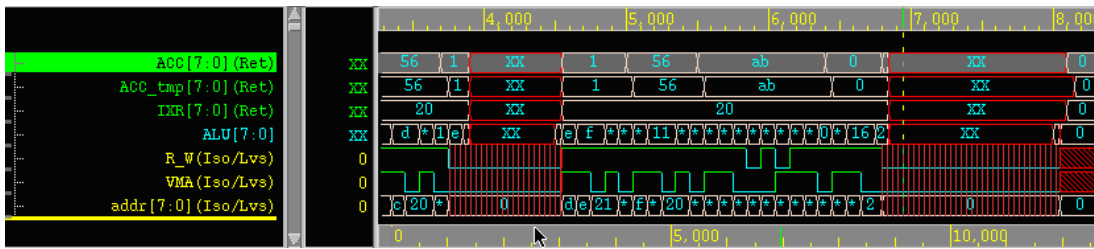
Figure 26 Power-Off Mask in nWave



Turn on the **View -> Power -> Mask Vacuous Range** option to mask out the *vacuous range* in the waveform view. *Vacuous Range* refers to the X range which is affected by the related power domains being switched off. The X value within the vacuous range is correct. However, the X value outside the vacuous range could be a bug. Use the **Mask Vacuous Range** option to mask all vacuous ranges in *nWave* and allow the non-vacuous range X to be easily found.

Turn on **View -> Power -> Mask Isolation** option to mask out the isolation range in the waveform view. Figure 27 illustrates an example for the power isolation mask capability.

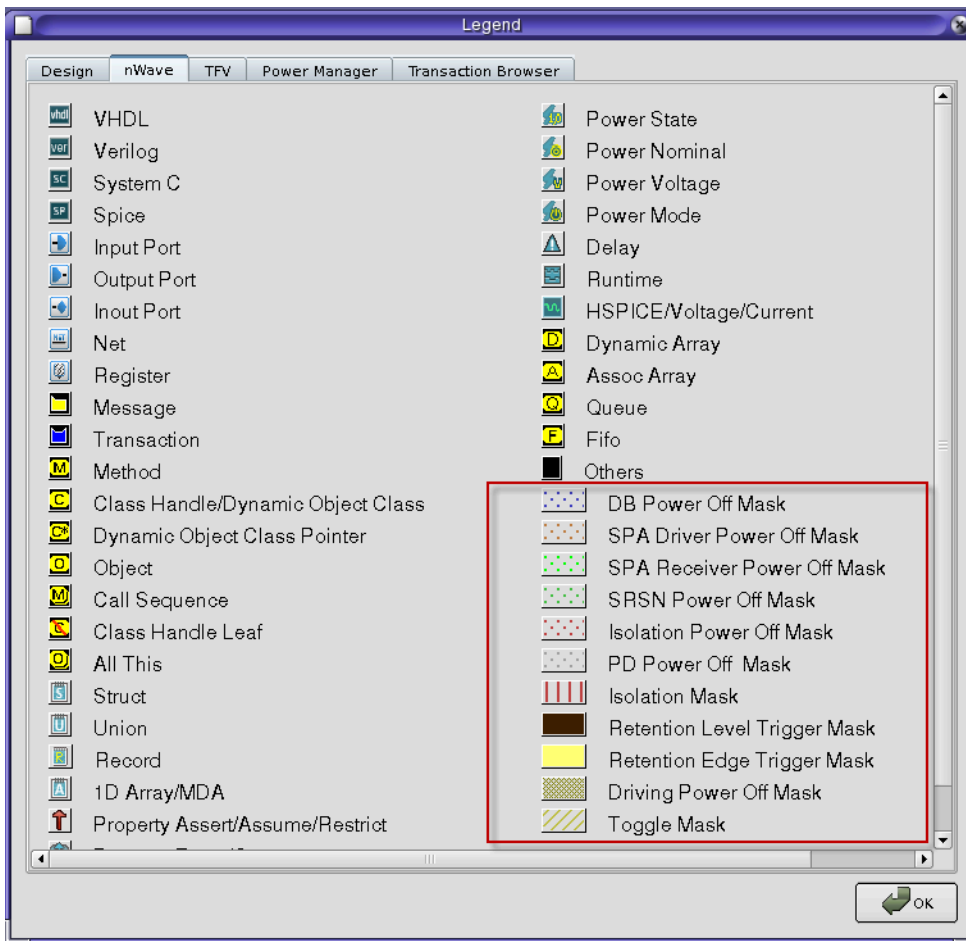
Figure 27 Mask Isolation in nWave



Turn on the **View -> Power -> Mask Toggle** option to mask out the range where the scope is already in a power off corruption state and the input signals have value changes.

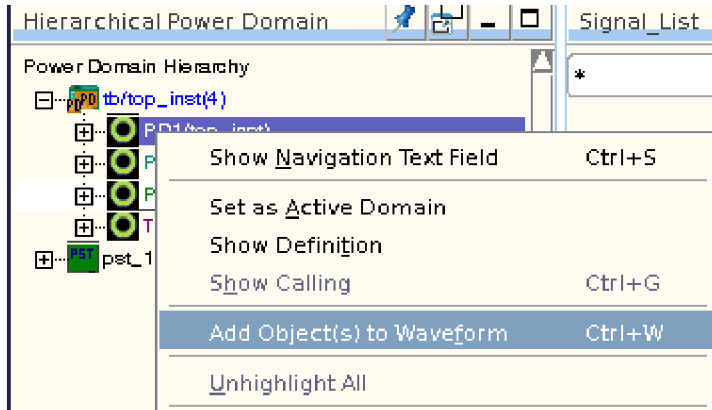
To view the power shading (power masking) information, use the **Help -> Legend** option and select the *nWave* tab. Figure 28 shows the power shading information in the *Legend* form.

Figure 28 nWave Legend Form



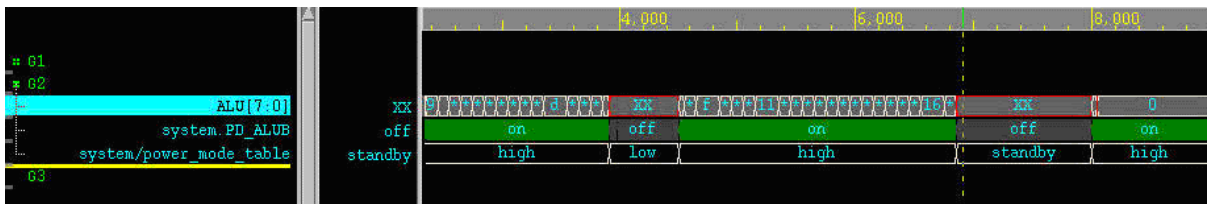
You can right-click a scope/instance node in the *Power Domain List* pane and select the **Add Object(s) to Waveform** option, as shown in the following figure, to add related signal list to the waveform view.

Figure 29 Adding Objects to Waveform



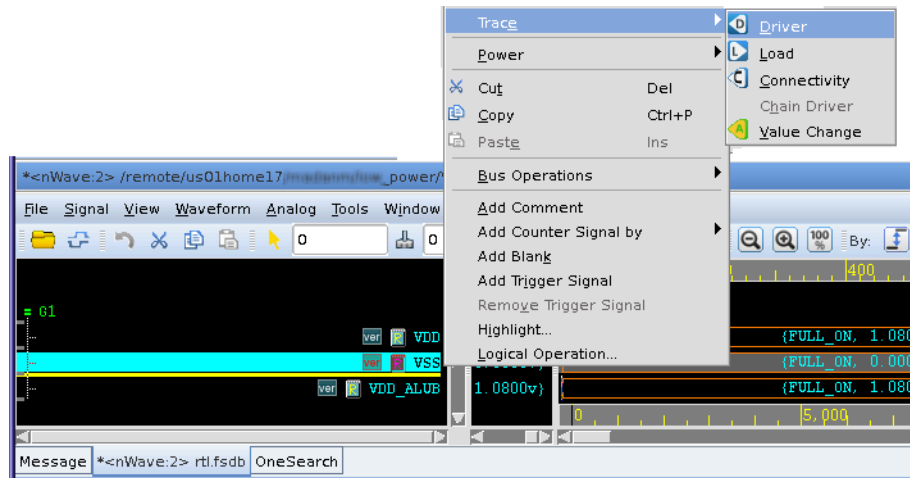
Tracing capabilities for power domains are provided in the waveform view for power-aware signals. For example, right-click a signal in the *nWave* signal pane, and invoke the **Power -> Add Power Domain** option to add the power domain of the selected signal. You can also right-click a signal (or domain) and invoke the **Power -> Add Power Mode** option to add the power mode under the selected signal. Figure 30 illustrates an example for these two tracing capabilities.

Figure 30 Trace Power Domain and Power Mode



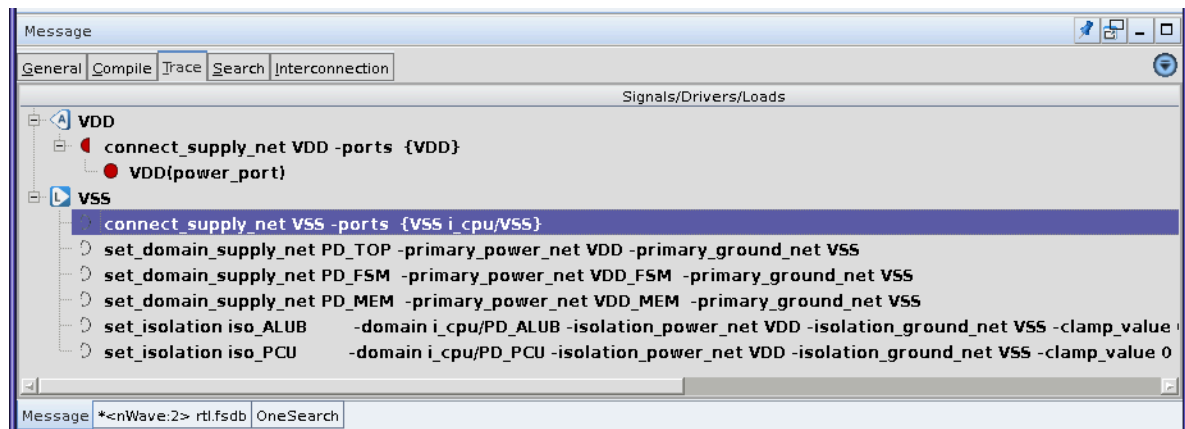
To trace the driver/load of a power net in *nWave*, right-click a power net and select **Trace** -> **Driver/Load**, as shown in the following figure:

Figure 31 Trace Power Net



The traced results are displayed in the **Trace** tab of the *Message* frame, as shown in the following figure:

Figure 32 Trace Results in the Trace Tab



To trace an *unknown* in power domain, place the cursor at the unknown value in waveform pane and right-click and select the **Trace Unknown Power State** option.

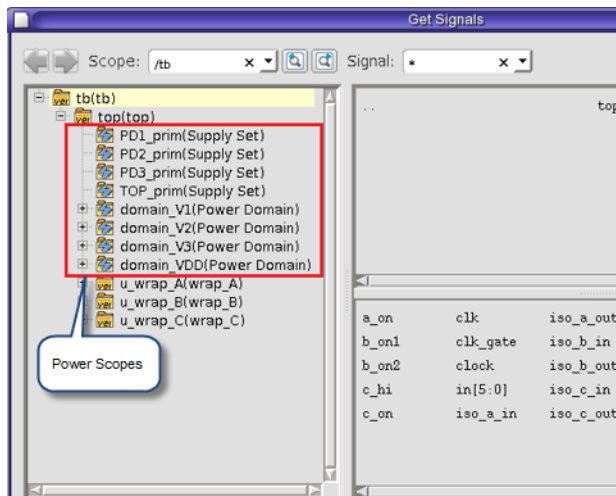
Tracing capabilities are also provided using the **Power** right-click option and selecting the appropriate command for tracing power-aware signals like Retention, Isolation, and Level-Shifter signals. For details, see the *Power Aware Debug* chapter in *Verdi and Siloti Command Reference*.

In addition, the *nWave* window supports highlighting of power domains and objects. For details, see the [Highlighting Power Domain, Power Objects, and Power Signals](#) section.

Viewing Power Objects in nWave

Verdi dumps power objects under the design scope. You can view the power objects in the *nWave Get Signals* form under the design scope. This feature is also supported with the standalone *nWave* window.

Power objects could be signal or power scope. Power scopes are represented with the  icon, as shown in the following figure:



Viewing Power Scopes in the Get Signals Form

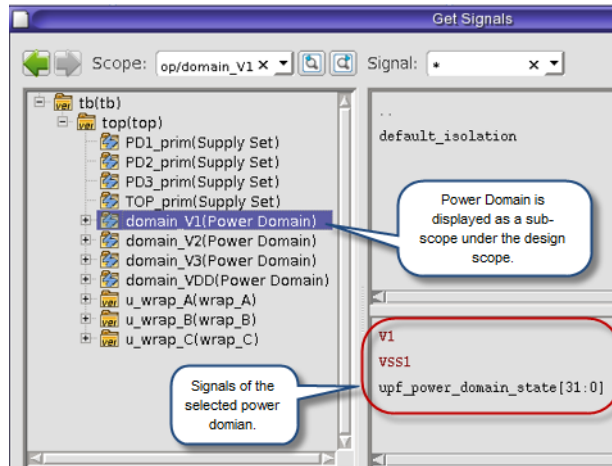
The signal enhancements are described in the following sections:

- [Power Domain](#)
- [Supply Net](#)
- [Supply Set](#)
- [Power Switch](#)
- [Logic Net](#)
- [Power Mode Table](#)
- [Power State Group](#)

Power Domain

The power domain signal is displayed as a sub-scope under the design scope in the *Get Signals* form, as shown in the following figure:

Figure 33 Power Domain Signals

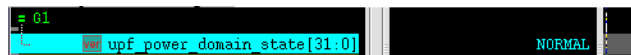


It is named by domain name, and displays the following signals:

- Power domain state: `upf_power_domain_state`
- Power/ground net: `v1/vss1`

Verdi adds the `upf_power_domain_state` signal instead of the power domain signal when you drag-and-drop a power domain from the Power Manager to the waveform, as shown in the following figure.

Figure 34 `upf_power_domain_state` signal in the Waveform

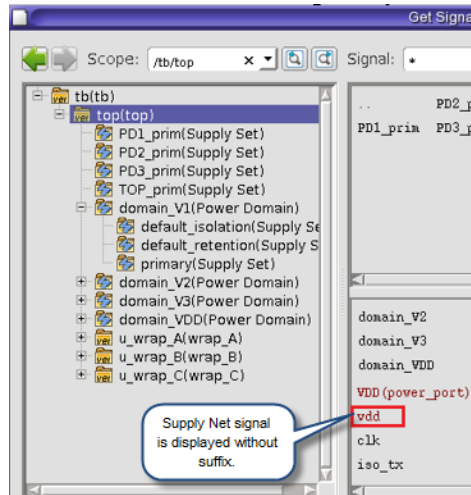


Right-click on waveform for `upf_power_domain_state` allows you to do “Trace Unknown/Error Power State” if there is an unknown/error value. This feature is not supported in the standalone *nWave* window.

Supply Net

The Supply Net signal is displayed as a composite signal under the design scope without suffix (for example, *vdd*), and with members **state**, **alias**, and **voltage** in the *Get Signals* form, as shown in the following figure:

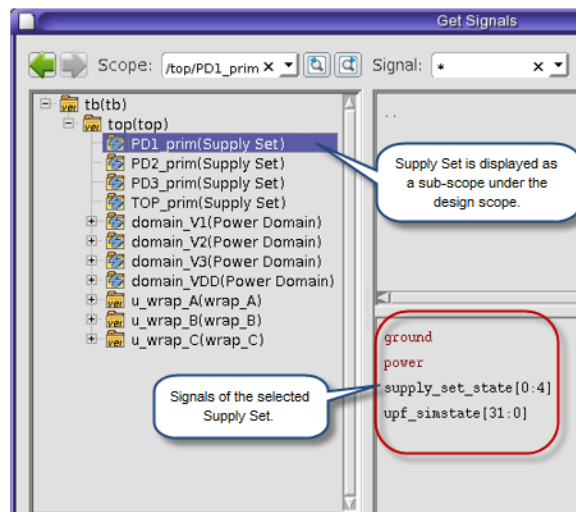
Figure 35 Supply Net Signal in the Get Signals Form



Supply Set

Supply Set is displayed as a sub-scope under the design scope in the *Get Signals* form, as shown in as shown in the following figure:

Figure 36 Supply Net Signal in the Get Signals Form

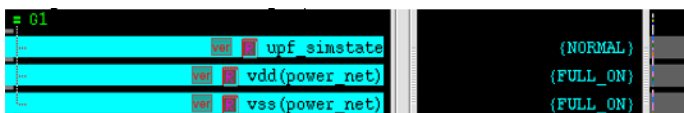


The following signals are displayed under this supply set scope:

- Supply set state: `upf_simstate`
- Supply set alias: `supply_set_state`
- Power/ground net: `power/ground`

Verdi adds the `upf_simstate` signal instead of the Supply Set signal, when you drag-and-drop a Supply Set from the Power Manager to the waveform, as shown in as shown in the following figure:

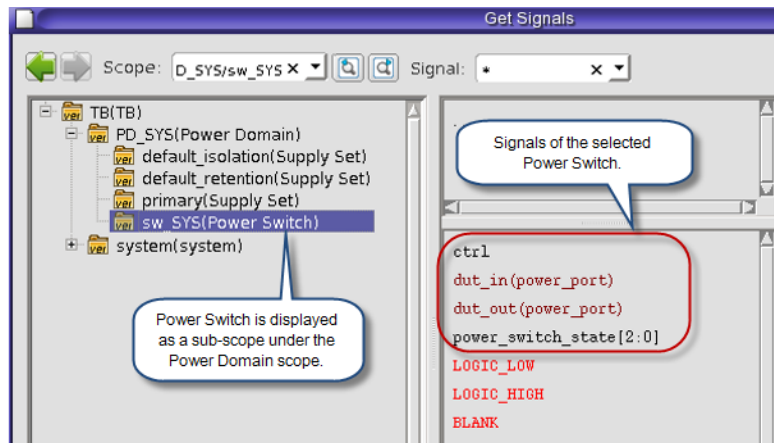
Figure 37 *upf_simstate signal in the Waveform*



Power Switch

Power Switch is displayed as a sub-scope under the Power Domain scope in the *Get Signals* form, as shown in the following figure:

Figure 38 *Power Switch in the Get Signals Form*

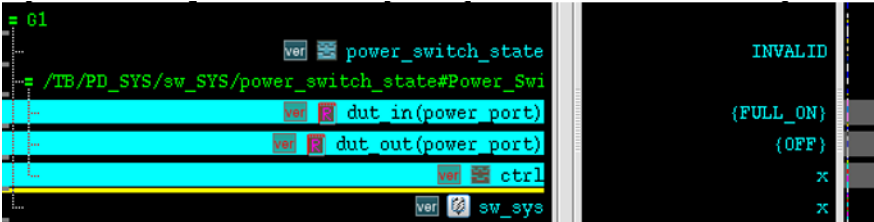


The following signals are displayed under the Power Switch scope:

- Power Switch: `power_switch_state`
- **Input, output, and control** ports

Verdi adds the `power_switch_state` signal instead of the Power Switch signal, when you drag-and-drop a Power Switch from the Power Manager to the waveform, as shown in the following figure:

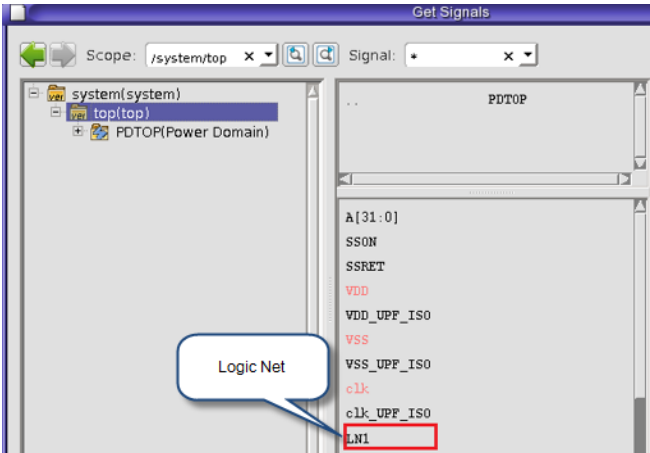
Figure 39 *power_switch_state signal in the Waveform*



Logic Net

Logic Net is displayed as a single signal without suffix in the *Get Signals* form, as shown in the following figure:

Figure 40 *Logic Net in the Get Signals Form*



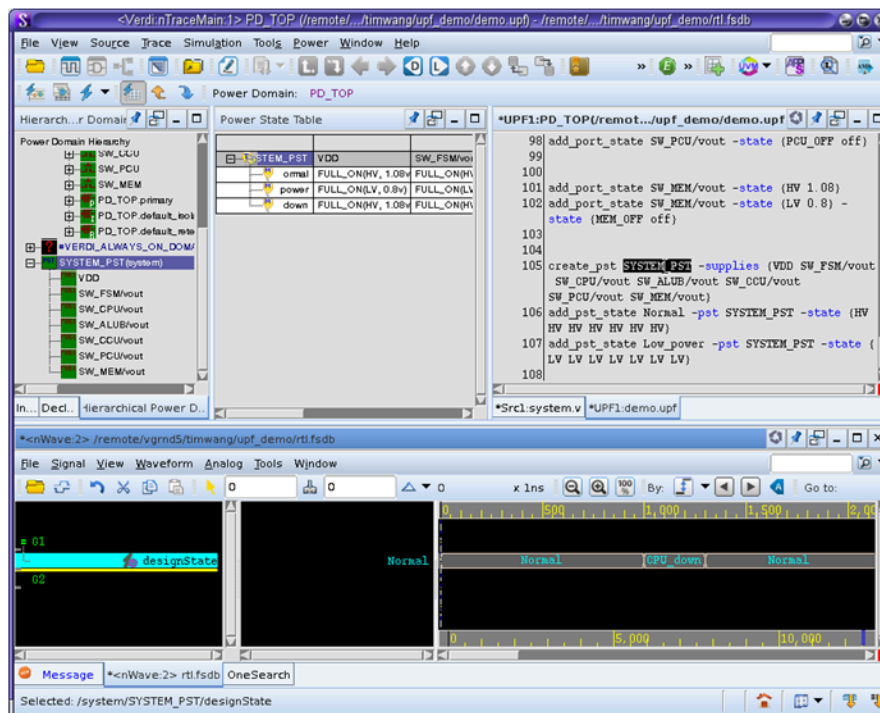
Power Mode Table

Power Mode Table is displayed as a single signal without suffix, as shown in the following figure:

This feature is not supported in the standalone *nWave* window.

Chapter 3: Using Power-Aware Debug Power-Aware Support in Other Panes

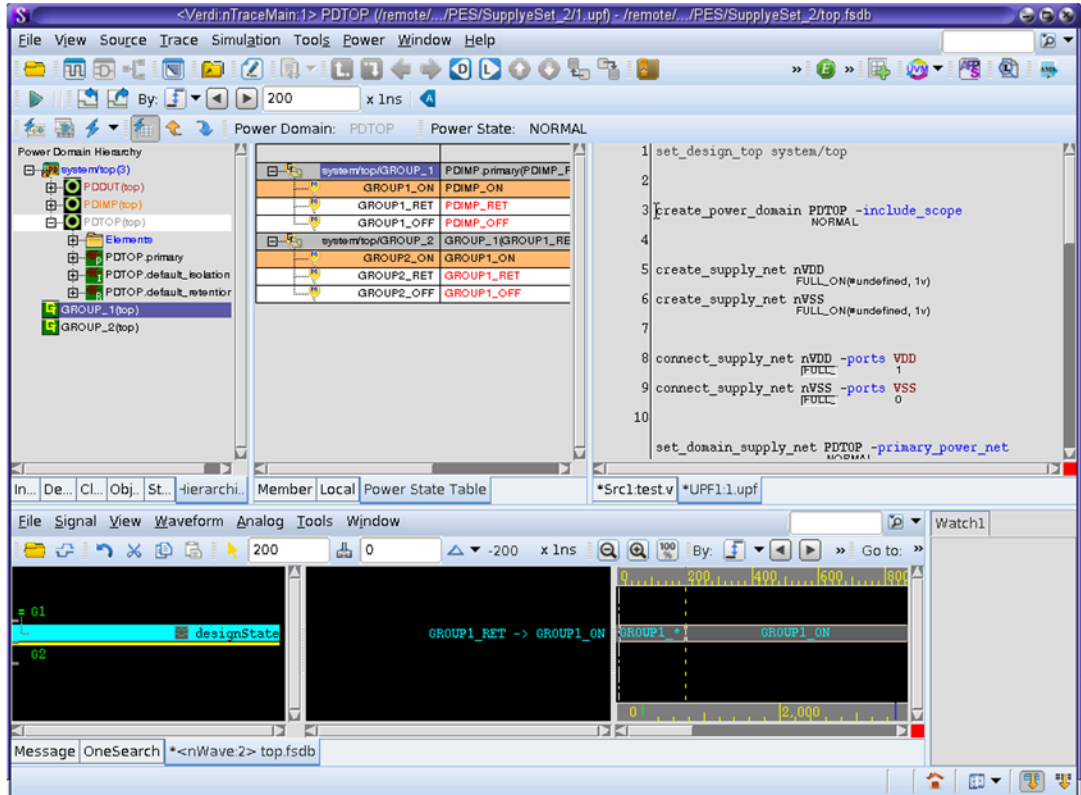
Figure 41 Power Mode Table Without Suffix



Power State Group

Power State Group is displayed as a single signal without suffix, as shown in the following figure:

Figure 42 Power State Group Table Without Suffix



This feature is not supported in the standalone *nWave* window.

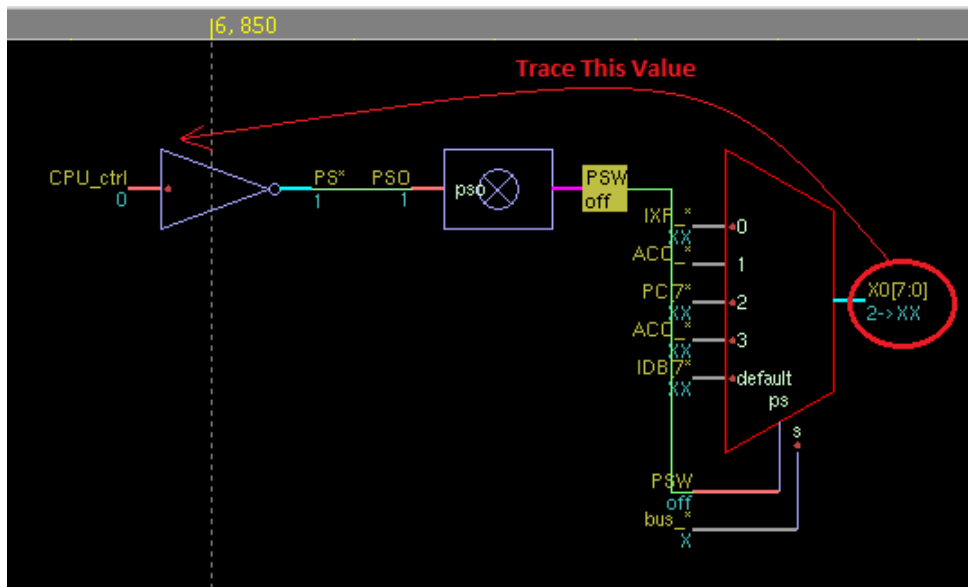
Temporal Flow View

The *Temporal Flow View (TFV)* pane supports power-aware debugging, as illustrated in [Figure 43](#).

Note:

For UPF, only MVSIM simulators are supported.

Figure 43 Trace This Value to Get Power Related Root Cause in TFV



The *TFV* window also supports power domain highlight and RET/ISO annotation for retention/isolation latches and Flip-Flops. For details, see the [Highlighting Power Domain, Power Objects, and Power Signals](#) section.

Tracing Driver/Load on Low Power Design Elements

The following sections describe the driver/load tracing on low power design elements:

- [Power Port](#)
- [Power Net](#)
- [Power Logic Net](#)
- [Power Logic Port](#)
- [Power Supply Set](#)
- [Power Domain](#)
- [Power Switch](#)
- [HDL Signals](#)

Power Port

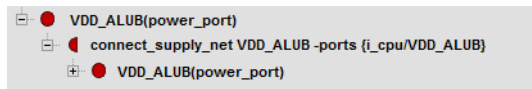
Verdi displays the power port name in the **Trace** tab of the *message* frame. The name of the power port is displayed in the following format:

```
port_name(power_port)
```

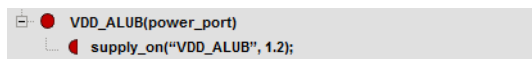
Power Port Drivers

The following are the possible drivers for a power port:

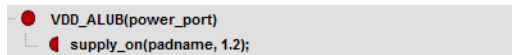
- Connected Net (specified using `connect_supply_net` command)
 - Supply net connected to HighConn of an input or inout port
 - Supply net connected to LowConn of an output or inout port



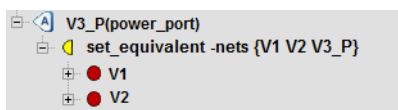
- The `supply_on`, `supply_off`, and `supply_partial_on` functions in the testbench.
 - The driver statement for the specified power port is listed if the pad name of `supply_on`, `supply_off`, or `supply_partial_on` is a string literal.



- Otherwise, the driver statement is listed as possible result of all power ports.



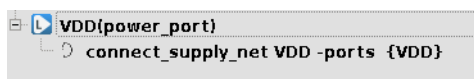
- The nets and ports of electrical or functional equivalence



Power Port Loads

The following are the possible loads for a power port:

- Connected Net (specified using `connect_supply_net` command)
 - Supply net connected to LowConn of an input or inout port
 - Supply net connected to HighConn of an output or inout port



- The nets and ports of electrical or functional equivalence



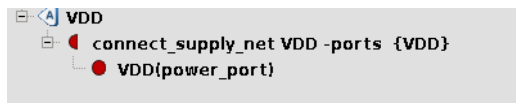
Power Net

Verdi displays the power net name in the **Trace** tab of the *message* frame.

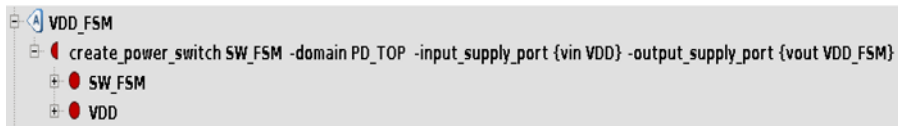
Power Net Drivers

The following are the possible drivers of a power net:

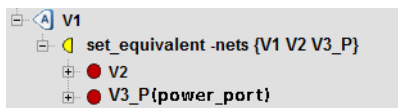
- Connected Port
 - Supply net connected to LowConn and port is input or inout
 - Supply net connected to HighConn and port is output or inout



- Power switch and its input supply port if the net is connected to output supply port of power switch.



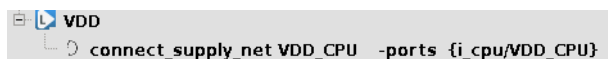
- The nets and ports of electrical or functional equivalence



Power Net Loads

The following are the possible loads for a power net:

- Connected Port
 - Supply net connected to HighConn and port is input or inout
 - Supply net connected to LowConn and port is output or inout



- Input supply port of power switch

```
vin(power_port)
  create_power_switch SW_FSM -domain PD_TOP -input_supply_port {vin VDD} -output_supply_port {vout VDD_FSM} \
```

- The nets and ports of electrical or functional equivalence

```
V1
  set_equivalent -nets {V1 V2 V3_P}
```

- Supply functions of supply set

```
V1
  create_supply_set SS1 -function {power V1}
```

```
V1
  set_domain_supply_net PD_TOP -primary_power_net V1 -primary_ground_net V2
```

Power Logic Net

Verdi displays the power logic net name in the **Trace** tab of the *message* frame.

Power Logic Net Drivers

The following are the possible drivers for a power logic net:

- Connected port
 - Input port or inout port which net is connected to its LowConn
 - Output port or inout port which net is connected to its HighConn

```
L1
  connect_logic_net L1 -ports L1_P
  L1_P
```

- Power switch if the net is connected to its ack_port

```
L1
  create_power_switch SW1 -ack_port {ack L1}
  SW1
```

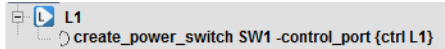
Power Logic Net Loads

The following are the possible loads for a power logic net:

- Connected Port
 - Input port or inout port which net is connected to its LowConn
 - Output port or inout port which net is connected to its HighConn

```
L1
  connect_logic_net L1 -ports L1_P
```

- Power switch if the net is connected to its control port



- Isolation strategy if the net is connected to isolation signal



- Retention strategy if the net is connected to save_signal/restore_signal



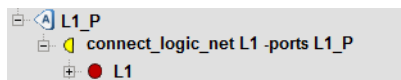
Power Logic Port

Verdi displays the power logic port name in the **Trace** tab of the *message* frame.

Power Logic Port Driver

The following is the possible driver for a power logic port:

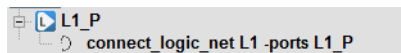
- Connected net
 - Supply net connected to HighConn and port is input or inout
 - Supply net connected to LowConn and port is output or inout



Power Logic Port Load

The following is the possible load for a power logic port:

- Connected net
 - Supply net connected to LowConn and port is input or inout
 - Supply net connected to HighConn and port is output or inout



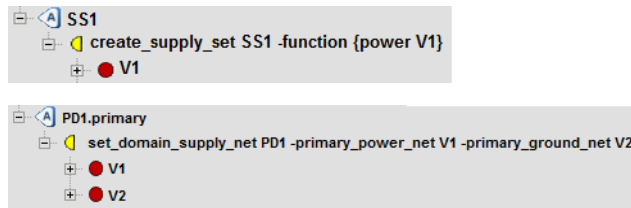
Power Supply Set

Verdi displays the power supply set name in the **Trace** tab of the *message* frame.

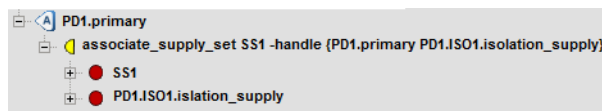
Power Supply Set Drivers

The following are the possible drivers for a power supply set:

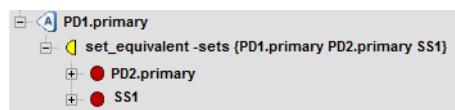
- Power supply net connected to supply functions



- Associated supply sets



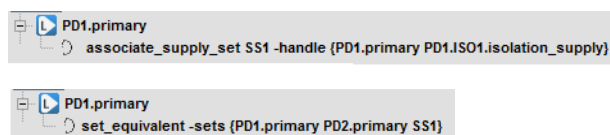
- The supply sets of electrical/functional equivalence



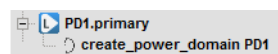
Power Supply Set Loads

The following are the possible loads for a power port:

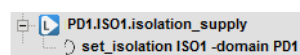
- Power supply set which is associated or equivalent



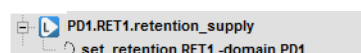
- Power domain (only primary)



- Isolation strategy



- Retention strategy



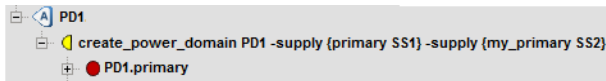
Power Domain

Verdi displays the power domain name in the **Trace** tab of the *message* frame.

Power Domain Drivers

The following is the possible driver for a power domain:

- Primary supply set



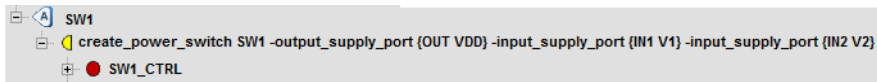
Power Switch

Verdi displays the power switch name in the **Trace** tab of the *message* frame.

Power Switch Drivers

The following is the possible driver for a power switch:

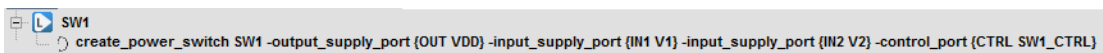
- Control net of a power switch



Power Switch Loads

The following is the possible load for a power switch:

- Net which is connected to output_supply_port or ack_port of a power switch



HDL Signals

The driver and load result for HDL signals is based on the VCS low power debugging file(lp_debug.xml). This XML file allows Verdi to display the instrumented signals as drivers for SRSN(set_related_supply_net), SPA(set_port_attributes), isolation, and retention strategies.

Drivers

The following are the possible drivers for a HDL signal:

- [Primary Power Supply](#)
- [Power Strategy](#)

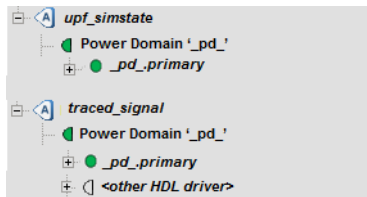
Primary Power Supply

- Power Domain

If the traced signal is `upf_simstate` (instrumented signal) under a HDL scope, Verdi displays the power domain of this scope as a driver as shown in the following figure.

If the traced signal is not driven by SRSN, SPA, or isolation cell, and the related power domain is off, Verdi displays the related power domain as driver. The HDL drivers of the traced signal are inactive at the same time as shown in the following figure.

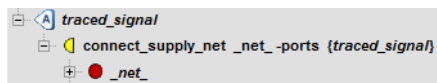
Example-1: The following figure shows the example driver tracing result when power domain ‘_pd_’ is “on” -> “off”:



- `connect_supply_net`

Traced signal is an input port

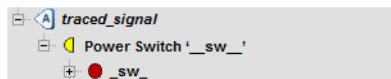
Example:



- `create_power_switch`

Traced signal is an ack_port

Example:



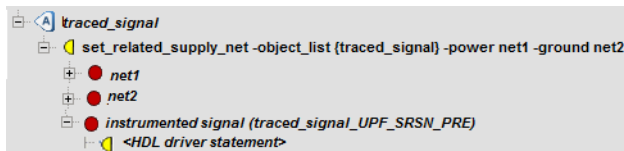
- `set_related_supply_net`

Traced signal is an object

Chapter 3: Using Power-Aware Debug

Tracing Driver/Load on Low Power Design Elements

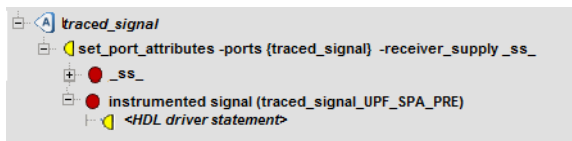
Example:



- **set_port_attributes**

Traced signal is an input port

Example:



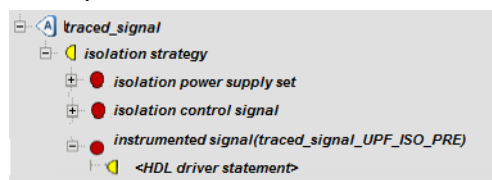
Power Strategy

- **Isolation**

Traced signal is an instrumented signal

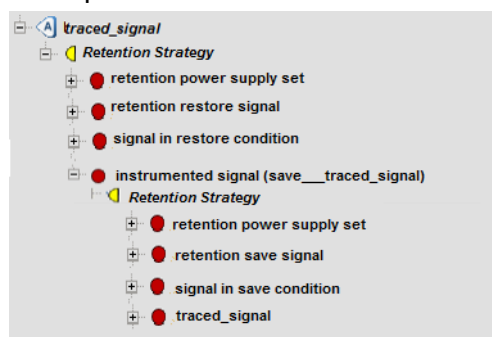
Chapter 3: Using Power-Aware Debug Tracing Driver/Load on Low Power Design Elements

Example:



- Retention

Example:

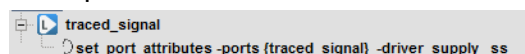


Loads

The following are the possible loads for a HDL signal:

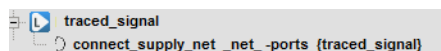
- set_port_attributes

Example:



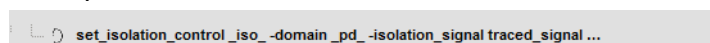
- connect_supply_net

Example:



- Isolation control signal and input signal

Example:



- Retention save/restore signal/condition or the signal to which the retention strategy is applied.

Example:

```
set_retention_control_ret_-domain_pd_-save_signal traced_signal -restore_signal traced_signal
```

- Power switch control signal

Example:

```
create_power_switch_sw_-domain_pd_-control_port traced_signal ...
```

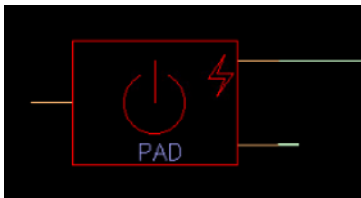
Support for Power Pad Cell, Power Switch Cell, and Punch Ports in nSchema

The following sections describe the support for debugging power pad cells, power switch instrumented cells, and punch ports in *nSchema*:

- [Debugging Power Pad Cells in nSchema](#)
- [Debugging Power Switch Instrumented Cells in nSchema](#)
- [Debugging Punch Ports in nSchema](#)

Debugging Power Pad Cells in nSchema

Verdi allows you to debug power pad cells in the Schematic window. Verdi provides the schematic symbol as shown in the following figure to represent the power pad cells in the Schematic window:

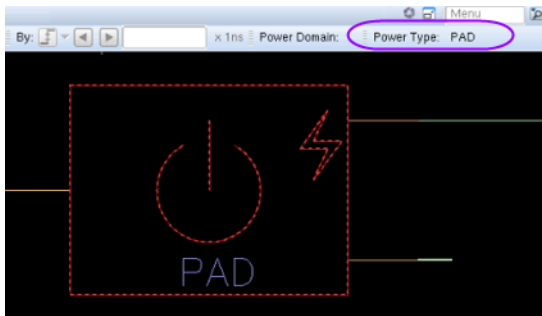


If you double-click a power pad cell schematic symbol, the first line of the parent module is highlighted in the Verilog source code.

By default, the power pad cell schematic symbol is highlighted in red color (ID_RED5). You can set the desired highlight color for the power pad cells using the **Color/Font** page of the *Preferences* dialog box (**Schematics** -> **Color/Font**).

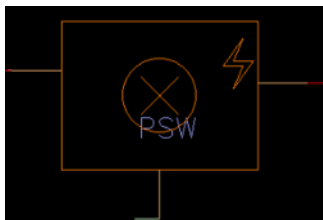
You can drag-and-drop the power pad cell schematic symbol to the waveform window and *Watch pane*.

The toolbar “Power Type” displays “PAD” for the power pad cell.



Debugging Power Switch Instrumented Cells in nSchema

Verdi allows you to debug the power switch instrumented cells in the Schematic window. Verdi provides the schematic symbol as shown in the following figure to represent the power switch instrumented cells in the Schematic window:

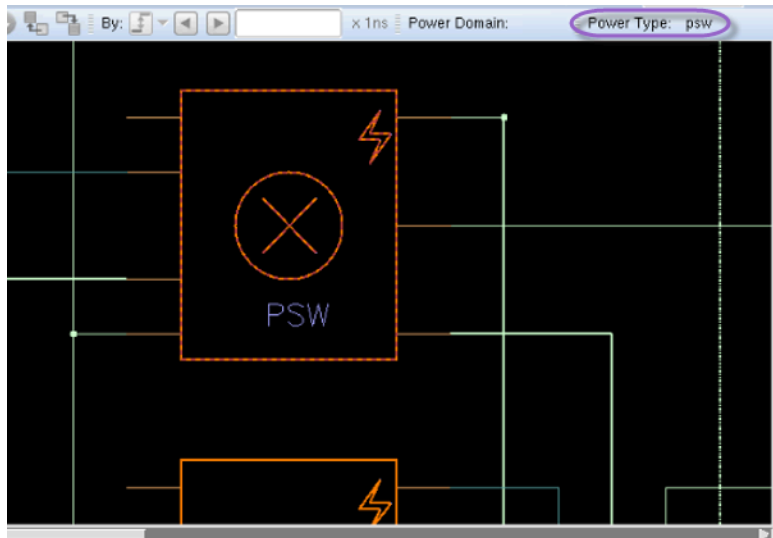


You can double-click the power switch instrumented cell schematic symbol to view its definition in the *UPF Source Code* view.

By default, the power switch instrumented cell schematic symbol is highlighted in orange color. You can set the desired highlight color for the power switch instrumented cells using the **Color/Font** page of the *Preferences* dialog box (**Schematics** -> **Color/Font**).

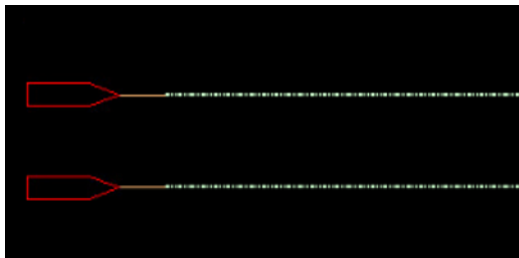
You can drag-and-drop the power switch instrumented cell schematic symbol to the waveform window and *Watch pane*.

The toolbar “Power Type” displays “psw” for the power switch instrumented cell, as shown in the following figure:



Debugging Punch Ports in nSchema

The punch ports are created in the schematic database. Verdi allows you to debug the punch ports in the Schematic window. Verdi provides the schematic symbol as shown in the following figure to represent the punch ports in the Schematic window:



By default, the punch port schematic symbol is highlighted in red color (ID_RED5) and its connected net's line style is *LINE_DASH_DOT_DOT*. You can set the desired highlight color for the punch ports using the **Color/Font** page of the *Preferences* dialog box (**Schematics** -> **Color/Font**).

Power-Aware Debug Support on Temporal Flow View (TFV) in Transition Mode

Verdi supports the following low power design elements on TFV in transition mode:

- [Supply Set of Power Domain](#)
- [Supply Net/Supply Port](#)
- [Power Switch](#)
- [Isolation](#)
- [Retention](#)
- [SRSN/SPA](#)
- [Boundary Cell](#)

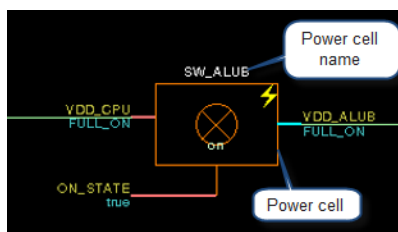
Transition trace mode is represented by the double arrow  icon on the tracing path.

Low power cells/elements are represented by the power  icon on the upper-right corner of the cell. This icon allows you to distinguish between the low power cells and normal HDL cells. This icon is supported for the following low power cells:

- Supply set cell
- Supply net/Supply port cell
- Power switch cell
- Isolation cell
- Retention cell
- SRSN/SPA cell

You can use the **Power Cell Name** toolbar option (**TFV window -> View -> Signal -> Power Cell Name**) to view the power cell name on the power cell as shown in the following figure:

Figure 44 Power Cell Name Displayed Above the Power Cell



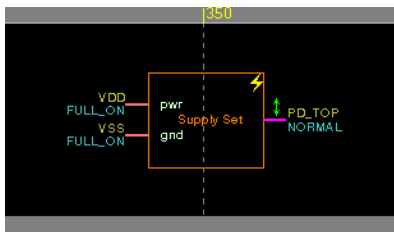
Supply Set of Power Domain

When you trace power transition on TFV, Verdi displays the supply set cell for the power domain if you specify the power domain. For example, as shown in the following sample UPF code.

```
create_power_domain PD_TOP set_domain_supply_net PD_TOP \  
-primary_power_net VDD \  
-primary_ground_net VSS
```

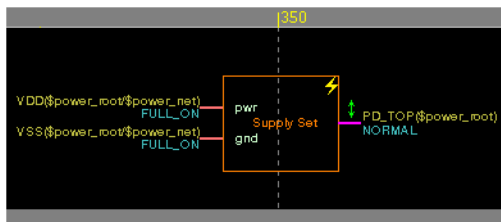
In the transition trace mode, supply set is displayed as follows:

Figure 45 Supply Set in Transition Trace Mode



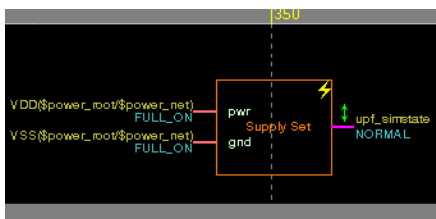
The default color of the supply set cell is light orange. The hierarchical name of the explicit supply set signal in transition mode is `$power_root`, as shown in the following figure:

Figure 46 Hierarchy Name of Explicit Supply Set in Transition Mode



For implicit supply set signal, the name of the supply set output signal is `upf_simstate`, as shown in the following figure:

Figure 47 Name of the Implicit Supply Set Signal



Supply Net/Supply Port

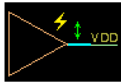
Verdi displays the supply port as a port cell and the supply net as a connection between power cells.

Consider the following sample UPF code:

```
create_supply_net VDD -domain PD_TOP
create_supply_port VDD -domain PD_TOP
connect_supply_net VDD -ports {VDD}
create_supply_net VSS -domain PD_TOP
create_supply_port VSS -domain PD_TOP
connect_supply_net VSS -ports {VSS}
```

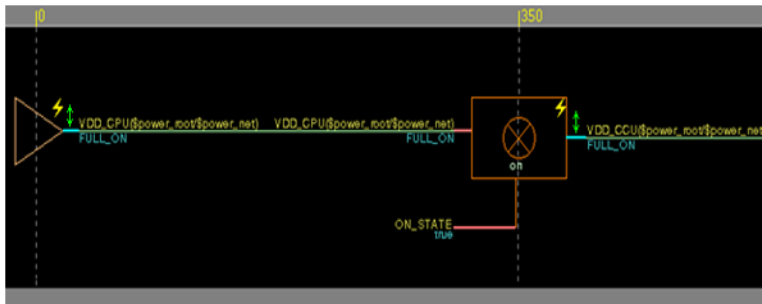
In the transition trace mode, the supply net/supply port is displayed as follows:

Figure 48 Supply Net/Supply Port in Transition Trace Mode



The hierarchical name of the supply net signal in transition mode is `$power_root/$power_net`, as shown in the following figure:

Figure 49 Hierarchy Name of Supply Net/Supply Port in the Transition Mode



Power Switch

Verdi displays only the active state ports of the power switch.

Consider the following sample UPF code:

```
create_power_domain PD_FSM \
  -elements {CHILD1 CHILD2 CHILD3 MASTER}
create_supply_net VDD_FSM -domain PD_TOP
create_power_switch SW_FSM \
  -domain PD_TOP \
  -input_supply_port {vin VDD} \
  -output_supply_port {vout VDD_FSM} \
```

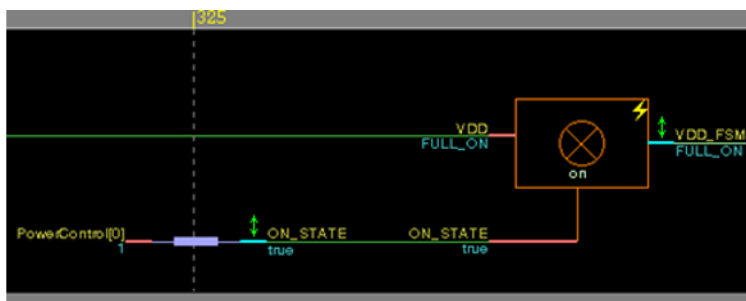
Chapter 3: Using Power-Aware Debug

Power-Aware Debug Support on Temporal Flow View (TFV) in Transition Mode

```
-control_port {en pmc/PowerControl[0]}\
-on_state {ON_STATE vin {en}} \
-off_state {OFF_STATE {!en}}\
-ack_port {my_ack_port my_ack_net}
```

In the transition trace mode, the power switch is displayed as follows:

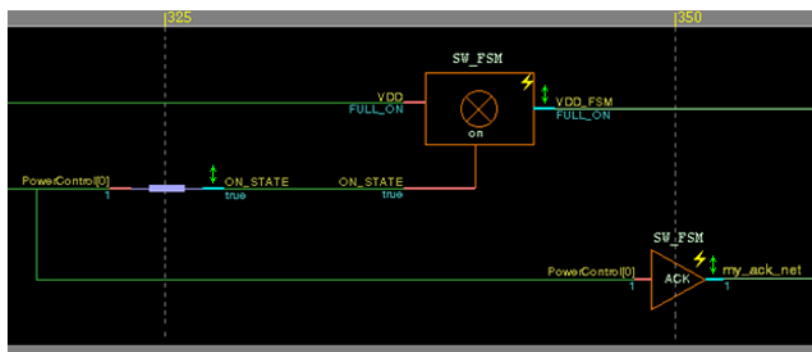
Figure 50 Power Switch in Transition Trace Mode



The default color of the power switch cell is light orange.

Verdi displays the acknowledge port (-ack_port) of the power switch as follows:

Figure 51 Ack_port of the Power Switch in Transition Trace Mode



All control nets of the power switch are the fanin cones of the acknowledge port. You can enable the **Power Cell Name** option to mark the power cell name above the acknowledge port.

Isolation

Verdi displays the isolation cells for the specified domain in the HDL trace path.

Consider the following sample UPF code:

```
set_isolation iso_ALUB \
-domain i_cpu/PD_ALUB \
-isolation_power_net VDD -isolation_ground_net VSS \
```

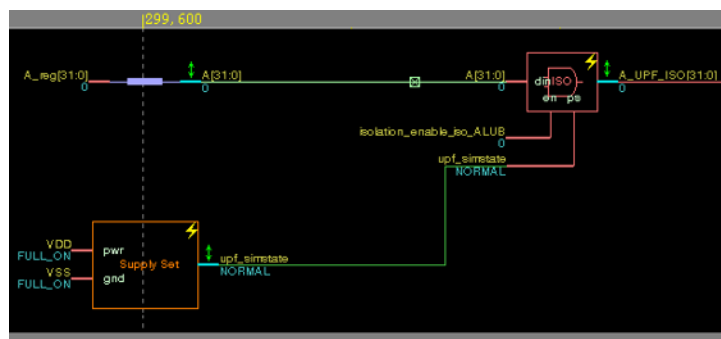
Chapter 3: Using Power-Aware Debug

Power-Aware Debug Support on Temporal Flow View (TFV) in Transition Mode

```
-clamp_value 0 \
-applies_to outputs
set_isolation_control iso_ALUB \
-domain i_cpu/PD_ALUB\
-isolation_signal pmc/ALUB_iso\
-isolation_sense high\
-location self
```

In the transition trace mode, the isolation cell is displayed as shown in the following figure. For implicit supply set, Verdi displays `upf_simstate` connected with assigned power nets.

Figure 52 Isolation Cell in Transition Trace Mode



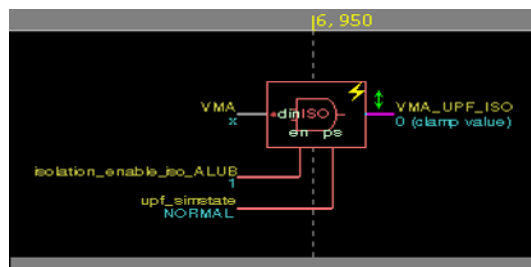
The default color of an isolation cell is pink.

Signal Naming

The naming of fanin/fanout signals follows the isolation naming rule. An instrumented signal is inserted before or after the ISO cell. The naming rule is `signal_name_UPF_ISO/` `signal_name__UPF_ISO_PRE`. The `en` port is also connected with the instrumented signal `isolation_enable_power_cell_name`.

When the `en` port is true, the fanout signal value of the ISO cell is the clamp value. Verdi also marks it as clamp value on the value label.

Figure 53 Isolation Fanout Value is the Clamp Value When the `en` Port is True



Retention

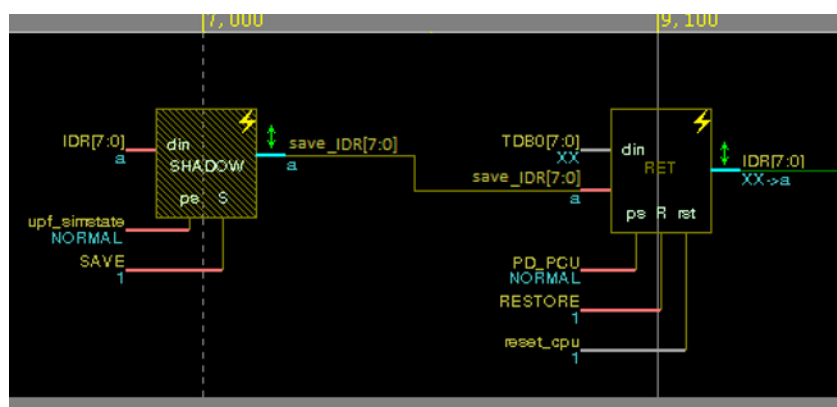
For the `set_retention` command specified in the UPF, Verdi displays the retention cells for the specified domain register.

Consider the following sample UPF code:

```
set_retention gprs_ret -domain GPRS \  
-retention_power_net VDDG -retention_ground_net VSS  
set_retention_control gprs_ret -domain GPRS \  
-save_signal {gprs_save high} \  
-restore_signal {gprs_nrestore low}
```

For the save state of retention, a shadow register cell with label `SHADOW` is inserted in the trace path to represent how the circuit holds the signal value. For the restore state, the fanout signal value is restored from the shadow register.

Figure 54 Retention Cell in Transition Trace Mode



The default color of the retention cell is yellowish-brown. The coloring of the retention cells and connection line is same. The color of the retention (restore) fanout signal does not follow the preference color setting.

The naming of the fanin/fanout signals follows the power naming rule. Following is the retention instrumented signal name:

```
save_signal_name
```

SRSN/SPA

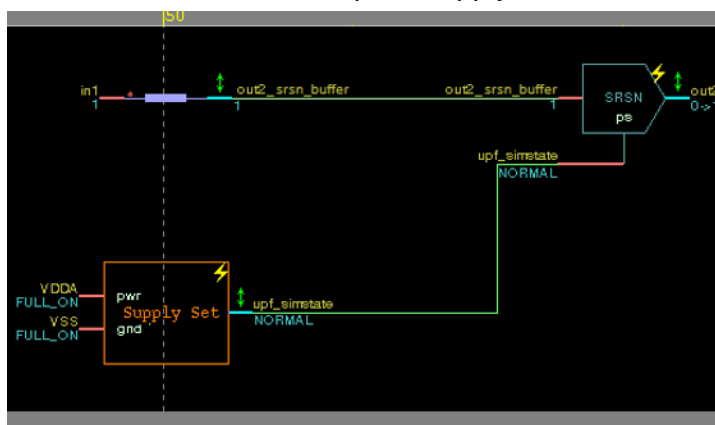
For the `set_related_supply_net` or `set_port_attribute` command specified in the UPF, Verdi displays the SRSN/SPA cells in the HDL trace path.

Consider the following sample UPF code:

```
set_related_supply_net -object_list {instA/out1 instA/in1} -power VDD
-ground VSS
set_related_supply_net -object_list {instA/out2} -power VDDA -ground VSS
```

In the transition trace mode, the SRSN/SPA cell is displayed as shown in the following figure. For implicit supply set, Verdi displays `upf_simstate` connected with assigned power nets.

Figure 55 SRSN Cell with Implicit Supply Set in Transition Trace Mode

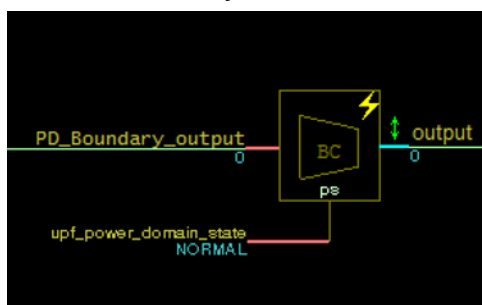


The default color of the SRSN/SPA cell is greenish-blue. An instrumented signal is inserted before the SRSN/SPA cell. The naming rule is `signal_name_srsn_buffer`.

Boundary Cell

For the XML flow, Verdi displays the boundary cell in the TFV window if it is instrumented.

Figure 56 Boundary Cell in Transition Trace Mode



Displaying Annotation Values for Power Signals in the Power Source Code Pane

Verdi displays annotation values for the following power signals in the Power Source Code pane:

- Power switch
- Supply set
- Power state table
- Power state group

Viewing Power Switch Signal Annotation

Verdi displays the `power_switch_state` signal value for a power switch, as shown in the following figures:

Figure 57 Power Switch Annotation in the Power Source Code Pane

```
30 create_power_switch SW_CPU -domain PD_TOP -
    ON_STATE NORMAL
    input_supply_port {vin VDD} -output_supply_port {vout VDD_CPU} \
    [FULL FULL_ON(1.08v)] [FULL FULL_ON(1.08v)]
31 -control_port {en
    1
    pmc/PowerControl[1]} -on_state {ON_STATE vin {en}} -off_state {
    1 [FULL 1]
    OFF_STATE {!en}}
    1
```

Figure 58 Power Switch State Value in the Signal List Pane

Signal	Value	Type
SW CPU		
power_switch_state	ON_STATE	Power State
vin	FULL_ON(1.08v)	Supply Port(Port In)
vout	FULL_ON(1.08v)	Supply Port(Port Out)
en	1	Control Port

Viewing Supply Set Signal Annotation

Verdi displays the `upf_simstate` and `supply_set_state` signal values in the following format for a supply set.

```
upf_simstate(supply_set_state)
```

The following figures show supply set annotation in the Power Source Code pane and Signal List table:

Figure 59 Supply Set Annotation in the Power Source Code Pane

```
63 create_power_state_group GP1
    RUN_GP1
64 add_power_state -group GP1 \
    RUN_GP1
65 -state RUN_GP1 {-logic_expr {TOP_prim == TOP_HV &&
    PD1_prim == PD1_HV} }[
    NORMAL(TOP_HV)
    NORMAL(PD1_HV)
```

Figure 60 UPF Simstate and Supply Set State Values in the Signal List Pane

GP1	designState	RUN_GP1	PSG State
TOP_prim		NORMAL(TOP_HV)	Supply Set
upf_simstate		NORMAL	Simstate
supply_set_state		TOP_HV	Power State
PD1_prim		NORMAL(PD1_HV)	Supply Set
upf_simstate		NORMAL	Simstate
supply_set_state		PD1_HV	Power State

Viewing Power State Table Signal Annotation

Verdi displays the `designState` signal value for a Power State Table (PST), as shown in the following figures:

Figure 61 PST Annotation in the Power Source Code Pane

```
106 create_pst SYSTEM_PST -supplies {VDD SW FSM/vout SW CPU/vout
    Normal
    [FULL_ON(1.08v) [FULL_ON(1.08v) [FULL_ON(1.08v)
    SW ALUB/vout SW CCU/vout SW PCU/vout SW MEM/vout}
    [FULL_ON(1.08v) [FULL_ON(1.08v) [FULL_ON(1.08v) [FULL_ON(1.08v)
107 add_pst_state Normal -pst SYSTEM_PST -state {HV HV HV HV HV HV HV
    Normal
}
```

Figure 62 Design State Value in the Signal List Pane

Signal	Value	Type
SYSTEM_PST		
designState	Normal	PST State
VDD	FULL_ON(1.08v)	Supply Port
SW FSM/vout	FULL_ON(1.08v)	Supply Port

Viewing Power State Group Signal Annotation

Verdi displays the `designState` signal value for a Power State Group (PSG), as shown in the following figures:

Figure 63 PSG Annotation in the Power Source Code Pane

```
63 create_power_state_group GP1
   RUN_GP1
64 add_power_state -group GP1 \
   RUN_GP1
65 -state RUN_GP1 {-logic_expr {TOP_prim == TOP_HV &&
   NORMAL(TOP_HV)
   PD1_prim == PD1_HV} }[
   NORMAL(PD1_HV)
```

Figure 64 Design State Value in the Signal List Pane

GP1			
designState	RUN_GP1	PSG State	
TOP_prim	NORMAL(TOP_HV)	Supply Set	
upf_simstate	NORMAL	Simstate	
supply_set_state	TOP_HV	Power State	
PD1_prim	NORMAL(PD1_HV)	Supply Set	
upf_simstate	NORMAL	Simstate	
supply_set_state	PD1_HV	Power State	

Viewing and Editing UPF Source Files

Verdi displays the UPF files of the current design in the *Filelist View*. It also allows you to edit UPF source when the file is opened in the *Source View*. The following sections describe this feature in detail:

- [Viewing UPF files in the FileList View](#)
- [Enabling UPF File Editing Feature in All Flows](#)
- [Adding Edit Option in RMB Menu](#)
- [Tcl Command](#)

Viewing UPF files in the FileList View

Verdi lists all UPF files in the *Filelist View*. The UPF files are listed under the UPF node, whereas HDL files are listed under the HDL node, as shown in the following figures:

Figure 65 Viewing UPF Files in the Filelist View

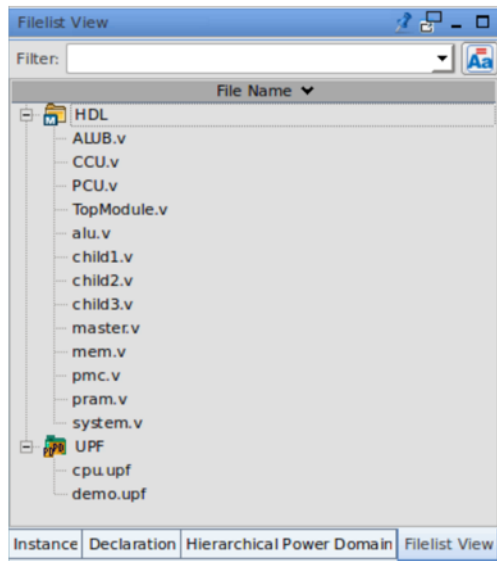
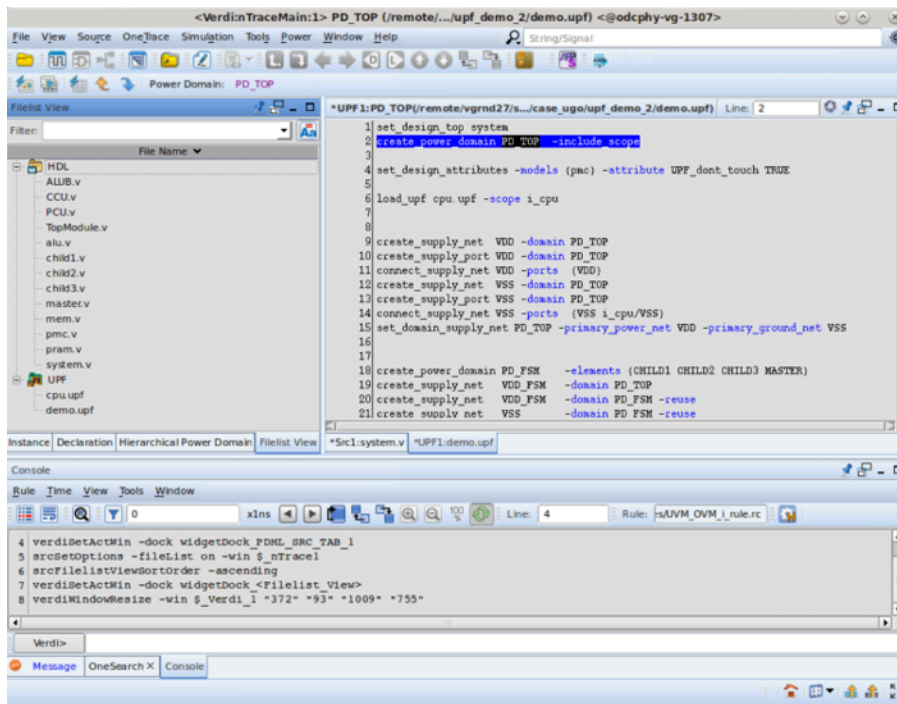


Figure 66 Viewing UPF Files



You can double-click the HDL/UPF node to collapse or expand tree node. You can select a file to browse by double-clicking on it or using the `FilelistViewEditSourceFile` Tcl command. The HDL files are displayed in **Src: filename.v** format and the UPF files are displayed in **UPF: filename.upf** format in the *Source View* as shown in the following figures:

Figure 67 Viewing UPF Files in the Source View

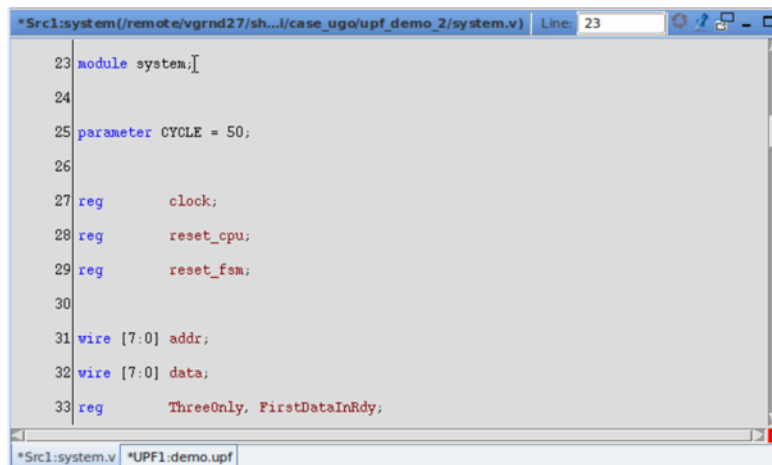
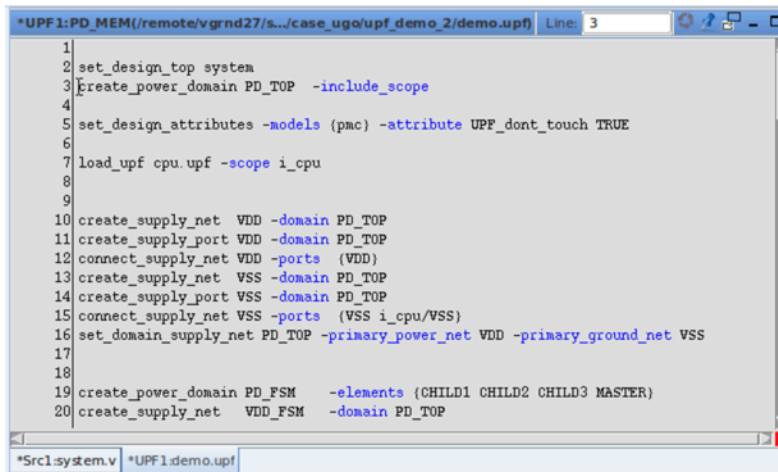
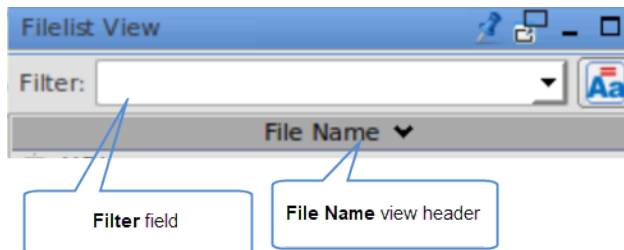


Figure 68 Viewing HDL Files in the Source View



You can sort the files by alphabetical order using the **File Name** view header. You can use the **Filter** field to filter nodes by name.

Figure 69 Sorting and Filtering files



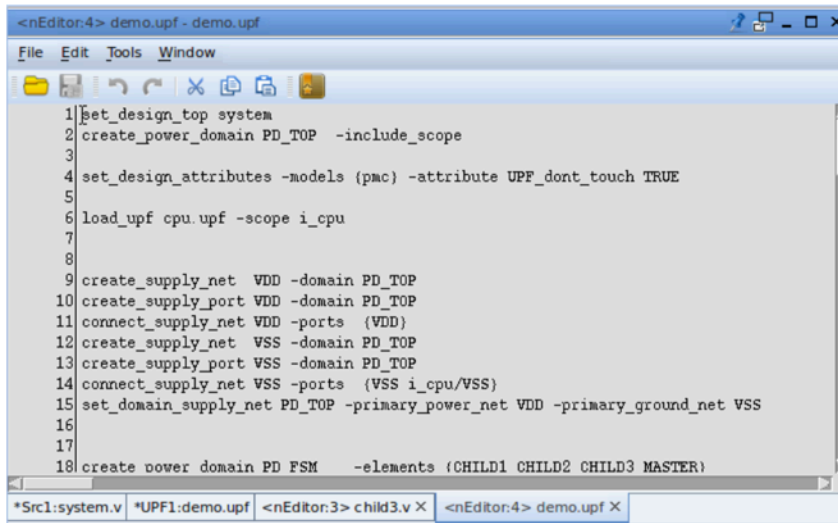
Enabling UPF File Editing Feature in All Flows

You can edit an UPF file in the **UPF Source View** using the **Edit Source File**  toolbar option or **E** shortcut.

Figure 70 Edit Source File Toolbar Icon



Figure 71 Edit UPF Source File

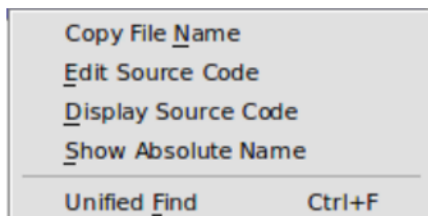


You can also enable this feature by clicking an UPF file in the *Filelist View* and using the **Edit Source Code** right-click option or using the [FilelistViewEditSourceFile](#) Tcl command to edit the file.

Adding Edit Option in RMB Menu

The **Edit Source Code** right-click option is added in the *Filelist View* to edit UPF source files.

Figure 72 Add Edit Source Code Right-Click Option



Tcl Command

You can use the following Tcl command to edit UPF files:

FilelistViewEditSourceFile

Command

```
FilelistViewEditSourceFile
```

Description

Allows you to edit source files.

Syntax

```
-file filename
```

Arguments

```
-file filename
```

Edit source file.

Example

```
FileListViewEditSourceFile -file demo.upf
```

Support for the set_repeater UPF Command

The `set_repeater` command defines a strategy for inserting repeater cells (buffers) for ports on the interface of a power domain. Repeater cells are placed within the domain, driven by input ports of the domain, and driving the output ports of the domain.

Verdi supports the `set_repeater` command as per the IEEE 1801-2015 UPF LRM. This command allows you to insert buffers inside the specified domain. This helps you to insert a buffer with a specific supply on the required side of the domain boundary pins.

Following is the syntax of the `set_repeater` command:

```
set_repeater strategy_name
-domain domain_name
[-elements element_list]
[-exclude_elements exclude_list]
[-applies_to <inputs | outputs | both>]
-repeater_supply supply_set_ref
[-name_prefix string] [-name_suffix string]
[-update]
```

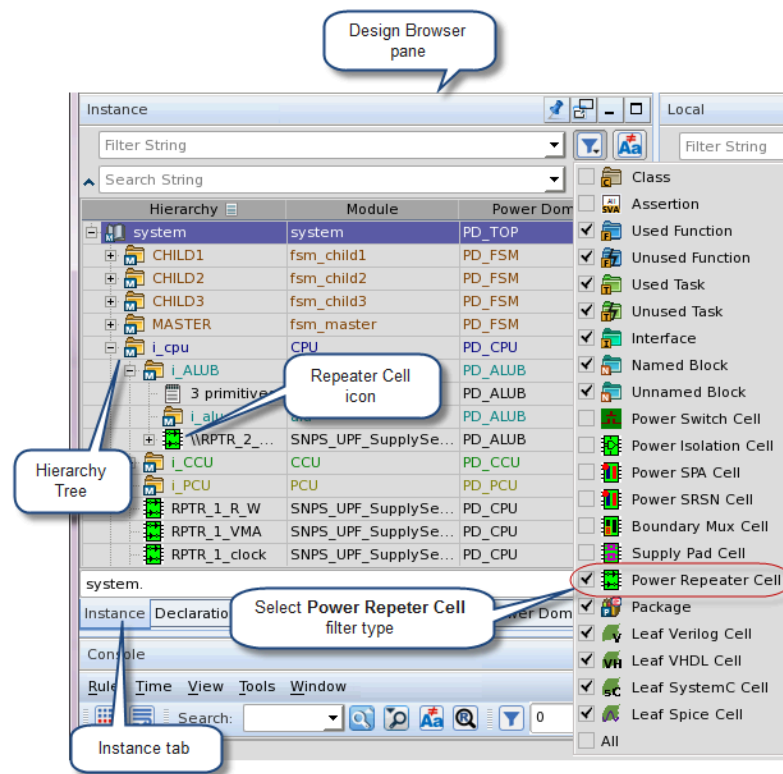
The following sections describe Verdi debug support for the `set_repeater` command:

- [Debugging Repeater Cells in nTrace](#)
- [Debugging Repeater Cells in Power Domain Hierarchy Pane](#)
- [Debugging Repeater Cells in nWave](#)
- [Debugging Repeater Cells in nSchema](#)
- [Trace Driver/Load Support for Repeater Cells](#)
- [Debugging Repeater Cells in Temporal Flow View](#)

Debugging Repeater Cells in nTrace

Verdi displays the repeater cells in the *Design Browser* pane as shown in the following figure. The  icon in the hierarchy tree represents the repeater cell. The repeater cell nodes are bit-based and are grouped under an array node.

Figure 73 Repeater Cells in Instance Hierarchy



You can filter repeater cells in the *Design Browser* pane using the **Power Repeater Cell** filter type. The repeater cells are not displayed by default in the hierarchy tree. To view the repeater cells in the hierarchy tree, select the **Power Repeater Cell** filter type from the *Filter Type* list as shown in the above figure.

You can double-click repeater cell in the hierarchy cell to view its definition in the UPF Source view.

In the *Source Code* view, the repeater cell is annotated as follows:

- If the HDL port is input to the repeater cell, Verdi displays annotation as  for the input HDL port by default. Verdi annotates the port with a string (*rptr_in*) as upper annotation when the **Display Annotation Symbol** preference option (**Power Aware** ->

Miscellaneous) is not selected. By default, this option is selected and Verdi displays annotation symbols for HDL ports.

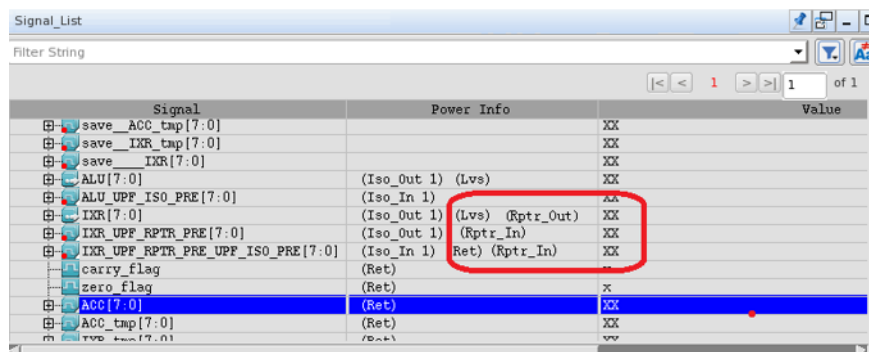
```

30 module CPU(clock, reset, VMA, R_W, data, addr);
31 input clock;
32 input reset;
33 output VMA;
34 output R_W;
35 inout [7:0] data;
36 output [7:0] addr;

```

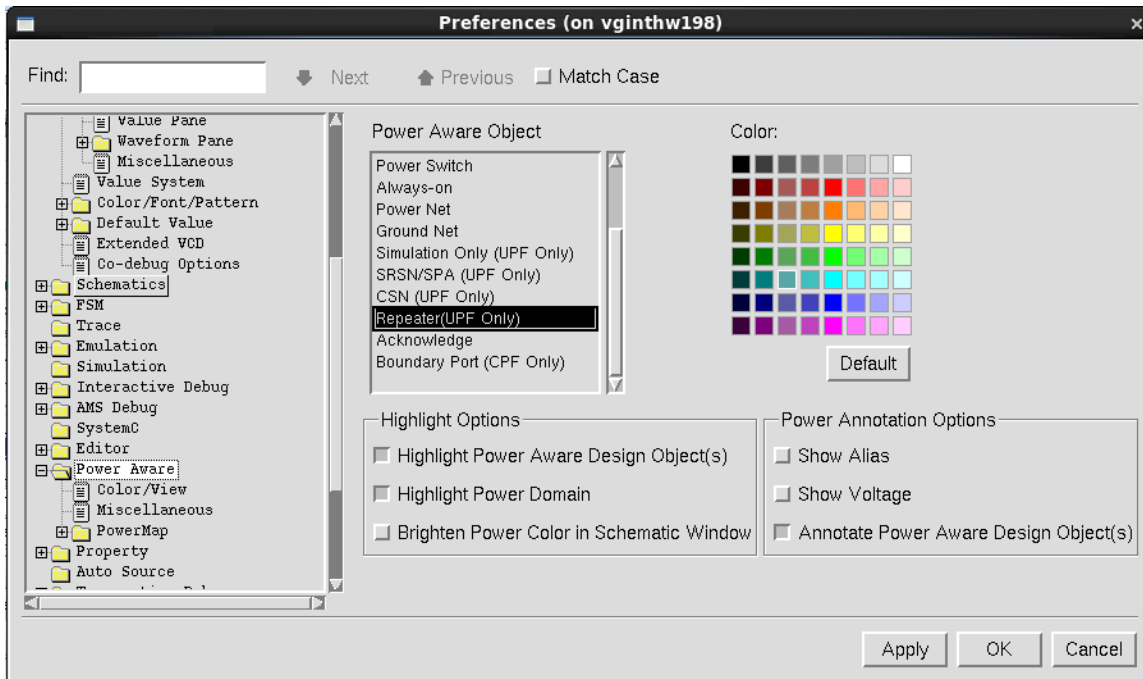
- If the HDL port is output to the repeater cell, Verdi displays annotation as  for the output HDL port by default. Verdi annotates the port with a string (rp_ptr_out) as upper annotation when the **Display Annotation Symbol** preference option is not selected.
- If the high-conn and low-conn of a HDL port are connected to the repeater cell, Verdi displays annotation as  for the inout HDL port by default. Verdi annotates the port with a string (rp_ptr_inout) as upper annotation when the **Display Annotation Symbol** preference option is not selected.

The rp_ptr_in, rp_ptr_out, and rp_ptr_inout signals are displayed as Rp_ptr_In, Rp_ptr_Out, and Rp_ptr_Inout respectively in the *Signal pane* as shown in the following figure:



Signal	Power Info	Value
save_ACC_tmp[7:0]		XX
save_IEXR_tmp[7:0]		XX
save_IEXR[7:0]		XX
ALU[7:0]	(Iso_Out 1) (Lvs)	XX
ALU_UPF_ISO_PRE[7:0]	(Iso_In 1)	XX
IEXR[7:0]	(Iso_Out 1) (Lvs) (Rp_ptr_Out)	XX
IEXR_UPF_RPTR_PRE[7:0]	(Iso_Out 1) (Rp_ptr_In)	XX
IEXR_UPF_RPTR_PRE_UPF_ISO_PRE[7:0]	(Iso_In 1) Ret) (Rp_ptr_In)	XX
carry_flag	(Ret)	X
zero_flag	(Ret)	X
ACC[7:0]	(Ret)	XX
ACC_tmp[7:0]	(Ret)	XX
tmp[7:0]	(Ret)	XX

Verdi allows you to set the highlight color for the repeater signal name in the *Source Code* view. To enable this feature, select **Repeater** from the *Power Aware Object* list in the *Preferences* form as shown in the following figure. Select the desired color from the color palette and then click the **OK** button.



Debugging Repeater Signals in the Source Code View

Verdi displays tooltip information for the repeater signals in the *Source Code* view as follows:

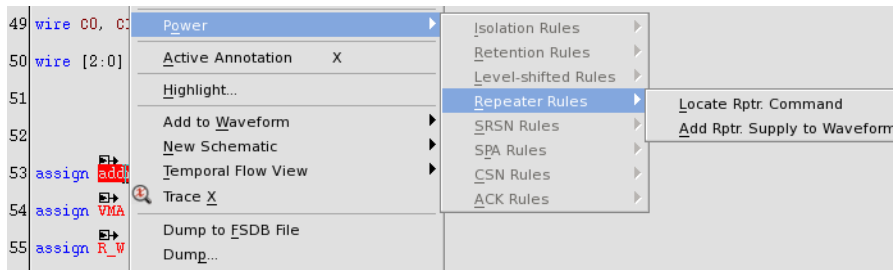
- If the objective signal is connected to the output of the repeater cell, Verdi displays tooltip information for the repeater cell in the following format:

```
addr = IDB;
Repeater Strategy(Driver) : i_cpu/RPTR_CPU
Repeater Supply : i_cpu/PD_CPU.primary
```

- If the objective signal is connected to the input of the repeater cell, Verdi displays tooltip information for the repeater cell in the following format:

```
clock(clock)
Repeater Strategy(Load) : i_cpu/RPTR_CPU
reset(reset)
Repeater Supply : i_cpu/PD_CPU.primary
```

In the *Source Code* view, you can right-click a repeater signal and select **power -> Repeater Rules** option, as shown in the following figure, to debug the repeater signal.



The **Locate Rptr. Command** sub-option allows you to locate the definition of the selected repeater signal in the *Power Source Code* pane. For example, as shown in the following figure:

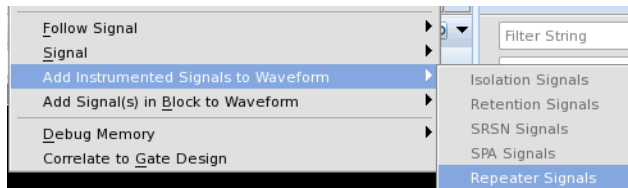
```

9 create_supply_port VSS -domain PD_CPU
10 create_supply_net VSS -domain PD_CPU
11 connect_supply_net VSS -ports {VSS}
12 set_domain_supply_net PD_CPU -primary_power_net VDD_CPU -primary_ground_net VSS
13 set_repeater RPTR_CPU -domain PD_CPU -applies_to both -repeater_supply PD_CPU.primary
14

```

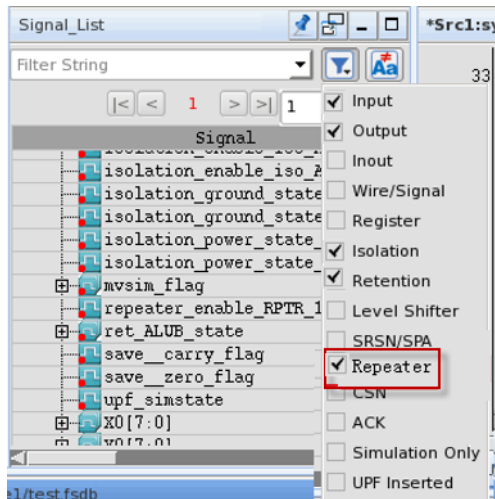
The **Add Rptr. Supply to Waveform** sub-option allows you to add the repeater supply of the selected repeater signal to the waveform.

To add the instrumented signals of a repeater signal to the waveform, right-click a repeater signal in the *nTrace Source Code* view or *Signal pane* and select **Add Instrumented Signals to Waveform -> Repeater Signals** option, as shown in the following figure:



Debugging Repeater Signals in the Signal Pane

You can filter the repeater signals in the *Signal pane* using the **Repeater** filter type.



This filter type is selected by default and repeater signals are displayed in the *Signal pane*. Verdi hides the repeater signals in the *Signal pane* when this filter type is unselected.

New Tcl Commands and Options

- A new option `-rptrCell` is added to the existing Tcl command `srcHierTreeFilterType`.

Description

The `-rptrCell` option allows you specify show/hide the repeater cells in the hierarchy tree.

Syntax

```
srcHierTreeFilterType -win winId -rptrCell [on | off]
```

on: hides the repeater cells in the hierarchy tree

off: shows the repeater cells in the hierarchy tree

Example

```
srcHierTreeFilterType -win $_nTracel -rptrCell off
```

- A new option `RepeaterSignal` is added to the existing Tcl command `paSetDisplayAttr`.

Description

The `RepeaterSignal` option allows you to specify the highlight color for the repeater signal name in the *Source Code* view.

Syntax

```
paSetDisplayAttr RepeaterSignal -color color_id
```

Example

```
paSetDisplayAttr RepeaterSignal -color ID_CYAN6
```

- A new option `-repeater` is added to the existing Tcl command `srcSignalViewFilterByType`.

Description

The `-repeater` option allows you to specify show/hide the repeater signals in the *Signal pane*.

Syntax

```
srcSignalViewFilterByType -repeater[on | off]
```

`on`: shows the repeater signal in the *Signal pane*

`off`: hides the repeater signal in the *Signal pane*

Example

```
srcSignalViewFilterByType -repeater off
```

- The new Tcl command `srcSignalViewAddRPTRInstruToWave` allows you to add instrumented signals of a repeater signal to the waveform.

Description

Specify add instrumental signals to waveform for the repeater signal.

Syntax

```
srcSignalViewAddRPTRInstruToWave -win win_id
```

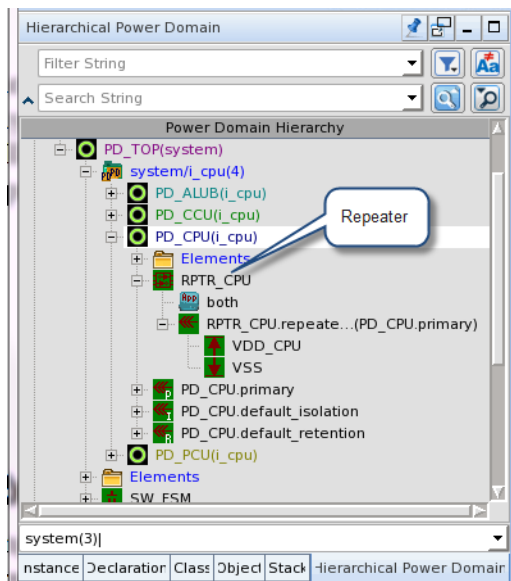
Example

```
srcSignalViewAddRPTRInstruToWave -win $_nTrace1
```

Debugging Repeater Cells in Power Domain Hierarchy Pane

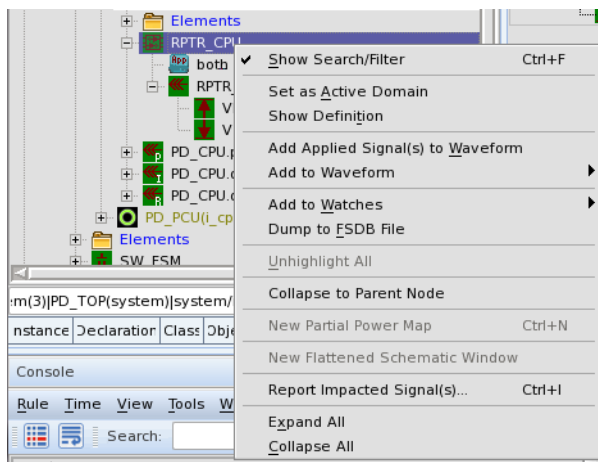
For the power design including the `set_repeater` UPF command, Verdi now displays the repeater strategy in the *Power Domain Hierarchy* pane as shown in the following figure.

The  icon in the power domain hierarchy tree represents the repeater.



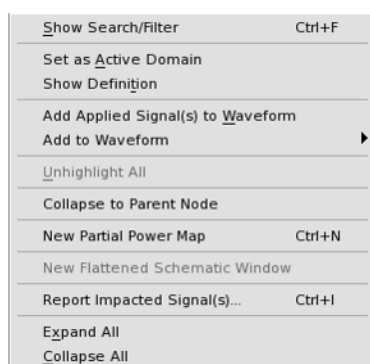
You can double-click on a repeater node to view its definition in the *Power Source Code* pane.

The following figure shows the right-click menu options of the repeater node in Interactive Mode:

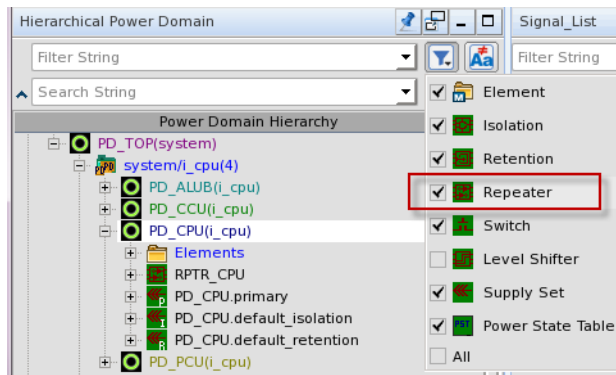


The following figure shows the right-click menu options of the repeater node in Post-Processing Mode:

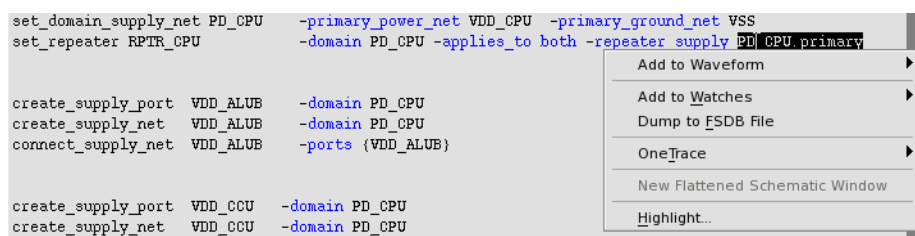
Chapter 3: Using Power-Aware Debug Support for the set_repeater UPF Command



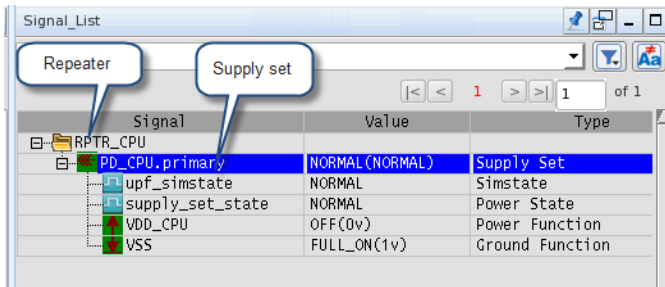
You can filter the repeaters in the power domain hierarchy tree using the **Repeater** filter type as shown in the following figure. This filter type is selected by default. The repeater nodes are hidden in the power domain hierarchy tree when the **Repeater** filter type is unselected.



In the *Power Source Code* pane, you can drag-and-drop the repeater supply name from `-repeater_supply` and element name from `-element` to the waveform and *Watch pane*. You can also right-click the supply name or element name to add them to waveform or *Watch pane*. The following figure shows the right-click menu options for the supply name and element name.



The *Signal* pane displays the supply set from `-repeater_supply` as shown in the following figure:



New Tcl Commands and Options

- A new option `-showRepeater` is added to the existing Tcl command `pdmlSetViewOption`.

Description

The `-showRepeater` option allows you to show/hide repeater node in the power domain hierarchy tree.

Syntax

```
pdmlSetViewOption -showRepeater [on | off]
```

Example

```
pdmlSetViewOption -showRepeater off
```

- A new option `-rptr` is added to the existing Tcl command `srcSignalViewDropPower`.

Description

The `-rptr` option allows you to drop repeater into signal list.

Syntax

```
srcSignalViewDropPower -rptr "path"
```

Example

```
srcSignalViewDropPower -rptr "tb/dut/inst_iso_in_reset"
```

- A new option `-Rptr` is added to the existing Tcl command `pdmlFindCommand`.

Syntax

```
pdmlFindCommand -Rptr "path"
```

Example

```
pdmlFindCommand "tb/dut/inst_rpnr" -Rpnr
```

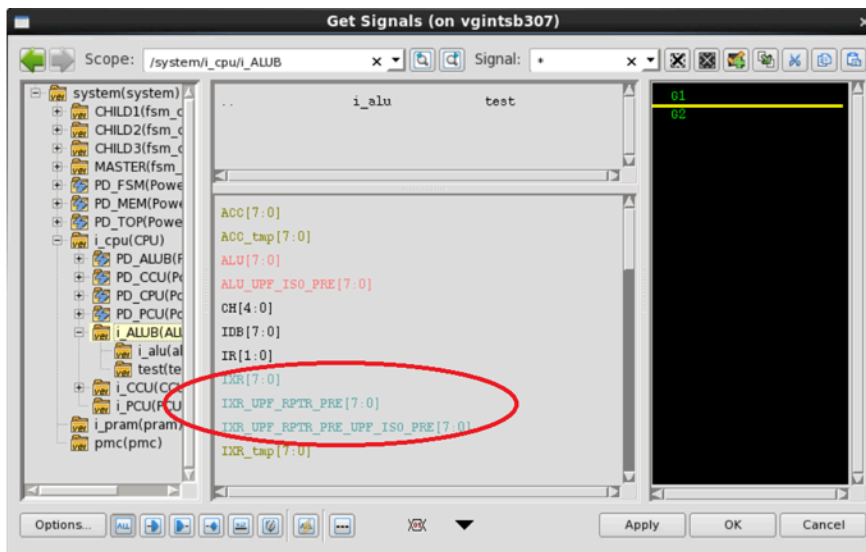
Debugging Repeater Cells in nWave

Verdi allows you to debug repeater signals in *nWave* window. The following sections describe this enhancement:

- [Setting Highlight Color for Repeater Signals](#)
- [Displaying In/Out Direction for Repeater Signals in Waveform](#)
- [Applying Repeater Rules](#)
- [Power Masking for Repeater Signals](#)
- [Filtering of Repeater Signals in Get Signals Dialog Box](#)
- [New Tcl Commands and Options](#)

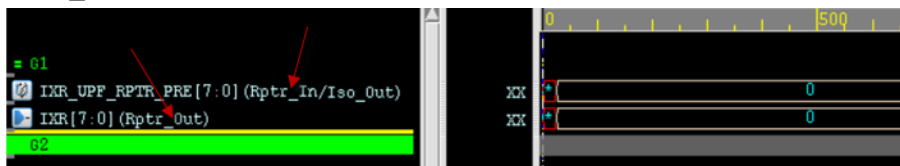
Setting Highlight Color for Repeater Signals

Verdi highlights the repeater signals in the *Get Signals* dialog box. By default, the repeater signals are highlighted in light-green color (ID_CYAN3) as shown in the following figure. You can set the desired highlight color for the repeater signals using the **Color/View** page of the *Preferences* dialog box (**Power Aware -> Color/View -> Power Aware Object -> Repeater (UPF Only)**).



Displaying In/Out Direction for Repeater Signals in Waveform

Verdi now displays the in/out direction for the repeater signals as `Rptr_In`, `Rptr_Out`, or `Rptr_Inout` in the *nWave* window.



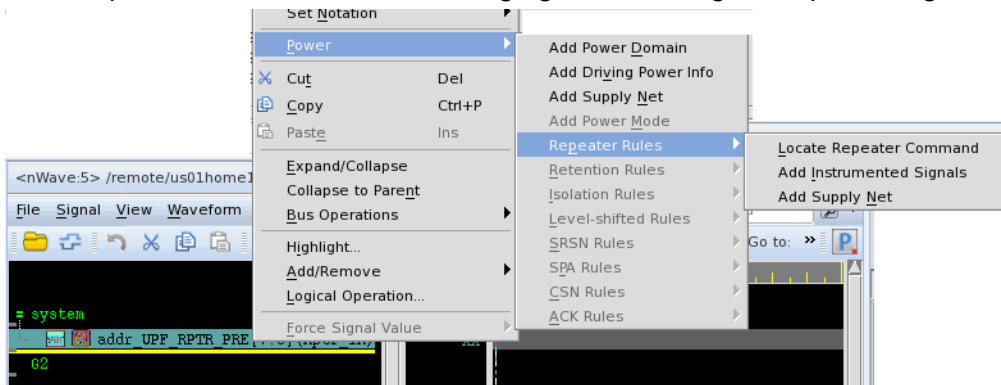
If multiple repeater rules are present in a signal, then Verdi displays the in/out direction for the repeater signals in the following format:

(Multi-Rptr, Rptr_x)

Where, x can be `Rptr_In`, `Rptr_Out`, or `Rptr_Inout`.

Applying Repeater Rules

In the waveform view, you can right-click a repeater signal and select **power -> Repeater Rules** option, as shown in the following figure, to debug the repeater signal.



The **Locate Repeater Command** sub-option allows you to locate the definition of the selected repeater signal in the *Power Source Code* pane. For example, as shown in the following figure:

```

9 create_supply_port VSS -domain PD_CPU
10 create_supply_net VSS -domain PD_CPU
11 connect_supply_net VSS -ports {VSS}
12 set_domain_supply_net PD_CPU -primary_power_net VDD_CPU -primary_ground_net VSS
13 set_repeater RPTR_CPU -domain PD_CPU -applies_to both -repeater_supply PD_CPU.primary
14

```

The **Add Instrumented Signals** sub-option allows you to add the instrumented signals of the selected repeater signal to the waveform view.

The **Add Supply Net** sub-option allows you to add the supply nets of the selected repeater signal to the waveform view.

Power Masking for Repeater Signals

Verdi supports masking or shading of the repeater signals when the repeater supply set state is turned off. This allows you to distinguish whether the value changes on a repeater signal is because of the repeater supplies being turned off.

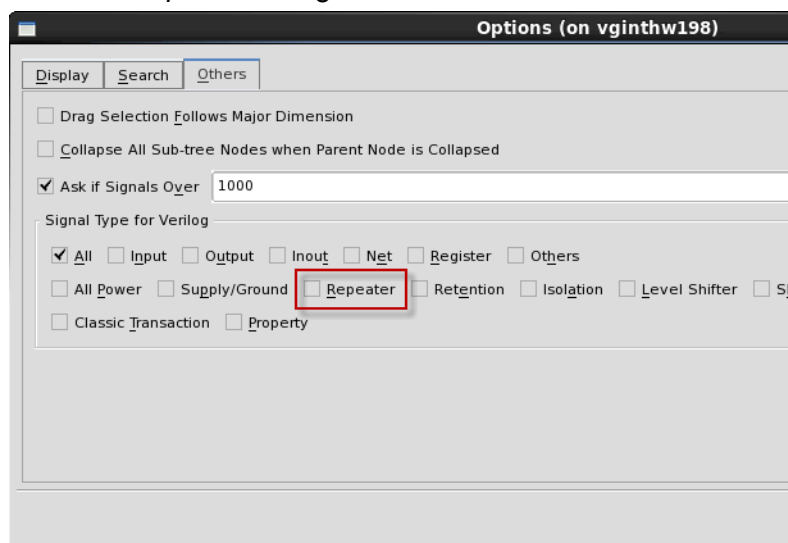
If a signal is repeater output, Verdi applies power off masking criteria with Repeater as the first priority.

Following is the format of the tooltip on waveform for repeater power off mask:

```
Supply: supply set state signal name
```

Filtering of Repeater Signals in Get Signals Dialog Box

You can filter the repeater signals in the *Get Signals* dialog box using the **Repeater** check box in the *Options* dialog.



New Tcl Commands and Options

- The new Tcl command `wvFindRptrCmd` allows you to locate the repeater command in the *Power Source Code* pane.

Description

Locate the Repeater command that applies to the Repeater signal in the *Power Source Code* pane.

Syntax

```
wvFindRptrCmd [-win windowId] -signal signal_full_name
```

Where,

-win - Specify the window ID of the invoking *nWave* window

-signal - Specify the signal to find the corresponding repeater command

Return Value

1 if succeeds, 0 if fails.

Example

```
wvFindRptrCmd -win $_nWave2 -signal "/system/a"
```

- The new Tcl command `wvAddRptrInstru` allows you to add the repeater instrumented signals.

Description

Add the repeater instrumented signals to *nWave*.

Syntax

```
wvAddRptrInstru [-win windowId] -signal signal_full_name
```

Where,

-win - Specify the window ID of the invoking *nWave* window

-signal - Specify the signal to add its instrumented signals

Return Value

1 if succeeds, 0 if fails.

Example

```
wvAddRptrInstru -win $_nWave2 -signal "/system/a"
```

- The new option `-rptr` is added to the existing `wvAddSupplyNet` Tcl command to add repeater supply net.

Description

The `-rptr` option allows you to add supply net for one signal in *nWave*.

Syntax

```
wvAddSupplyNet [-win windowId] -signal signal_full_name [-all | -rptr  
| -iso | -ret | -srsn | -spa]
```

Where,

- win - Specify the window ID of the invoking *nWave* window
- signal - Specify the signal to add supply net
- rptr- Add supply net for repeater rule

Return Value

1 if succeeds, 0 if fails.

Example

```
wvAddSupplyNet -win $_nWave2 -signal "/system/a" -rptr
```

- A new option value `Rptr_Power_Off_Layer` is added to the existing Tcl command `wvSetDisplayAttr`.

Description

The option value `Rptr_Power_Off_Layer` allows you to change repeater power off masking pattern.

- A new option value `-repeater` is added to the existing Tcl command `wvGetSignalSetOptions`.

Description

The option value `-repeater` allows you to filter repeater signals in the *Get Signals* dialog signal list.

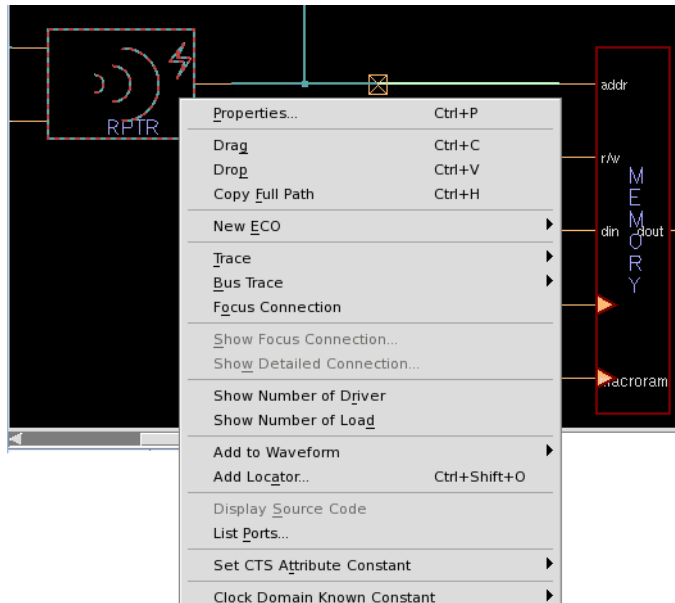
Debugging Repeater Cells in nSchema

Verdi allows you to debug repeater cells in the Schematic window. Verdi provides the schematic symbol as shown in the following figure to represent repeater cells in the Schematic window.



You can double-click the repeater schematic symbol to view its definition in the *Power Source Code* pane.

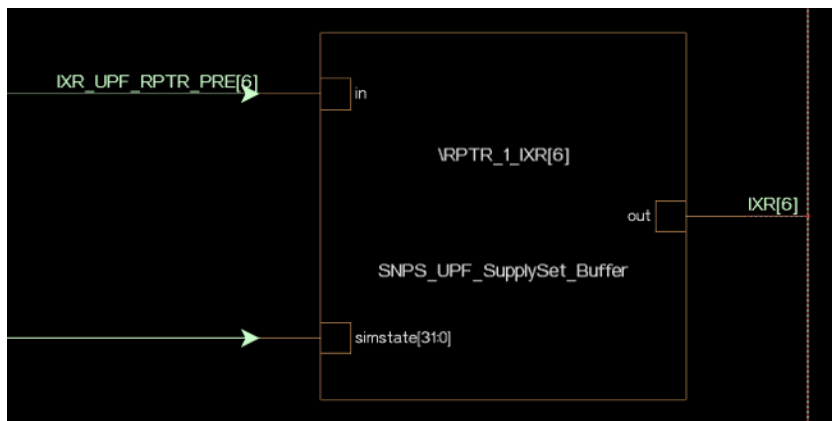
You can drag-and-drop the repeater schematic symbol to the waveform window and *Watch pane*. The following figure shows the right-click menu options for the repeater schematic symbol:



Trace Driver/Load Support for Repeater Cells

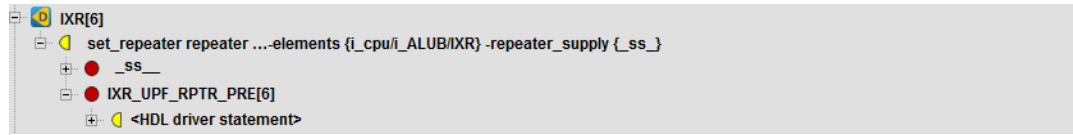
Verdi allows you to trace driver/loads for the `set_repeater` related instrumented cell/signals in the *nTrace* window.

Consider the following figure where the signal `IXR[6]` connects to the output port of the instrumented cell.



Trace Driver

For the instrumented cell for repeater shown in the above figure, if you trace the signal `IXR[6]`, then Verdi displays the tracing result as shown in the following figure:

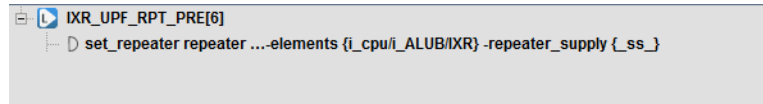


Verdi displays the `set_repeater` command as driver. The driver displays the related power supply (`_ss_`) and instrumented signal (`IXR_UPF_RPTR_PRE[6]`) as contributors. You can further expand the instrumented signal to view the drivers for the output port. You can hover the mouse pointer on a contributor signal to view its information in a tooltip.

Verdi performs static trace if the fsdb file is not provided.

Trace Load

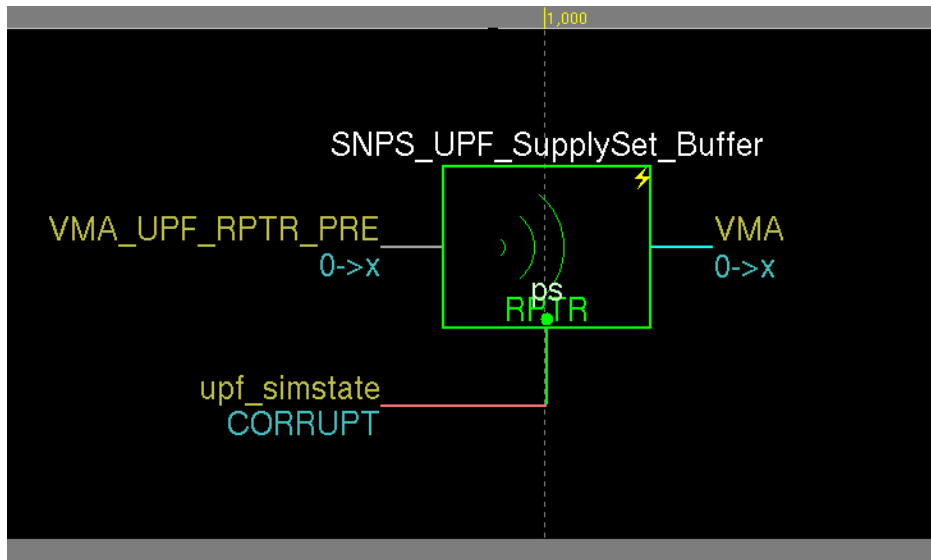
If you trace the load of the signal `IXR_UPF_RPTR_PRE[6]`, Verdi displays the tracing result as shown in the following figure:



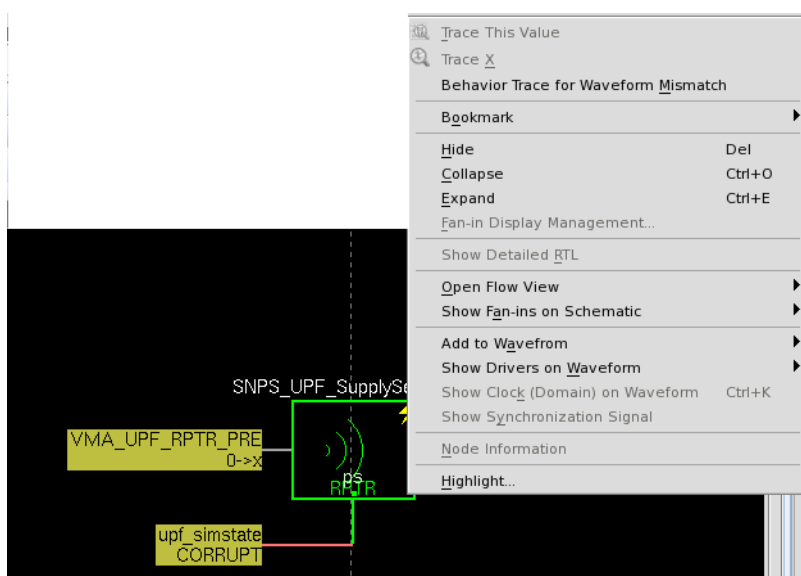
Verdi does not display any contributors for traced load.

Debugging Repeater Cells in Temporal Flow View

Verdi allows you to debug repeater cells in the *Temporal Flow View* (TFV). Verdi provides the following symbol to represent repeater cells in TFV.



By default, the repeater cells are highlighted in light-grey color (ID_CYAN8). You can set the desired highlight color for the repeater cells using the **Color/View** page of the *Preferences* dialog box (**Power Aware** -> **Color/View** -> **Power Aware Object** -> **Repeater (UPF Only)**). The naming of the instrument signals follows the VCS naming rule. The following figure shows the right-click menu options for the repeater symbol in TFV:



Support Pass Through connect_supply_net and connect_logic_net commands as Pass-Through Trace Result

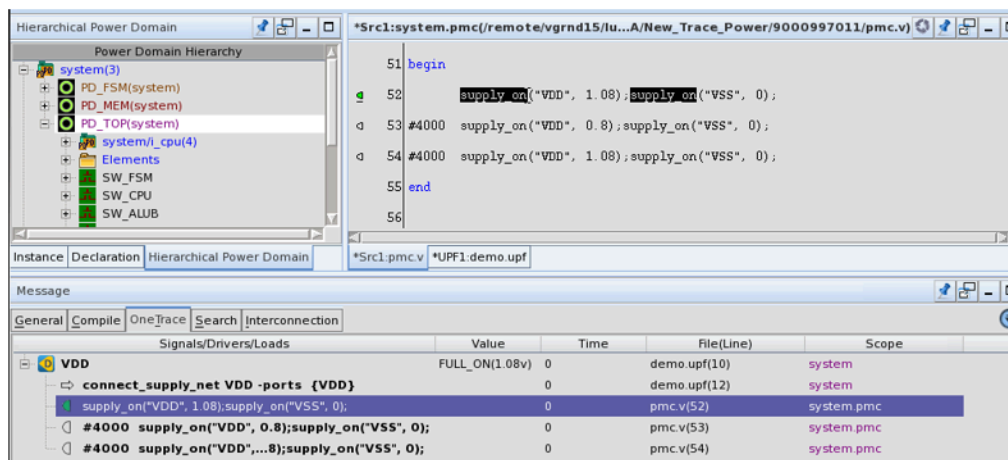
During tracing, Verdi supports pass through `connect_supply_net` and `connect_logic_net` commands as pass-through trace result. The trace result does not stop at every `connect_supply_net` and `connect_logic_net` command during tracing.

This feature is controlled using the **Pass Through Connect Supply/Logic Net Commands** preference option (**Trace -> Pass Through Connect Supply/Logic Net Commands**) in the *Preferences* form. This option is *off* by default. When you enable this option by clicking it, the trace result passes through the `connect_supply_net` and `connect_logic_net` commands during tracing. The setting is saved in the `novas.rc` file.

This feature is not supported when the `connect_supply_net` and `connect_logic_net` commands contain RTL signals.

For example, in the following figure, the trace driver passes through the `connect_supply_net` commands as pass through trace result and displays the root cause as a supply function.

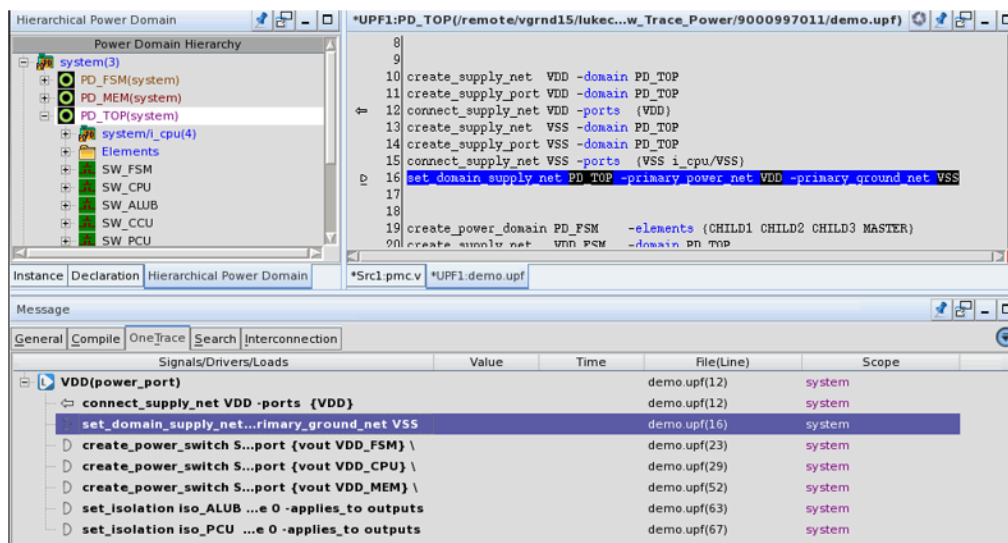
Figure 74 Driver Result Displays CSN Command as Pass Through Trace Result



The pass through trace result is hidden by default, you can use the right-click menu options in the *OneTrace* pane to view the pass through trace result.

In the following figure, the trace load passes through the `connect_supply_net` commands as pass through trace result.

Figure 75 Load Result Displays CSN Command as Pass Through Trace Result



The pass through trace result is hidden by default, you can use the right-click menu options in the *OneTrace* pane to view the pass through trace result.

Power Aware Trace Driver/Load Support for SPICE Signals

For SPICE+UPF designs, Verdi power aware debug supports tracing drivers/loads for SPICE signals. Verdi displays the SPICE signal name in the **Trace** tab, for example, as shown in the following figure:

Figure 76 Tracing SPICE Signals With Power Analysis

Signals/Drivers/Loads	Value	Time	File(Line)	Scope
vdd	2.99V	819.53339	/remote/us0.../top_spi(7)	TestBench.I1.I1.I1.I1.I1
connect_supply_net VDDH -ports {VDDH I1/I1/I1/I1/vdd I1/I1/I1/I1/vdd}	819.53339	819.53339	top_int_sup_upf(26)	TestBench.I1
VDDH	FULL_O...9998v	819.53339	top_int_sup_upf(14)	TestBench.I1
connect_supply_net VDDH -ports {VDDH I1/I1/I1/I1/vdd I1/I1/I1/I1/vdd}	819.53	819.53	top_int_sup_upf(26)	TestBench.I1
VDDH(power_port)	FULL_O...9998v	819.53	top_int_sup_upf(9)	TestBench.I1
connect_supply_net VDDH -ports {I1/VDDH }	819.53	819.53	testBench.upf(23)	TestBench
VDDH	FULL_O...9998v	819.53	testBench.upf(7)	TestBench
connect_supply_net VDDH -ports {I0/vdd_dig_high }	819.53	819.53	testBench.upf(12)	TestBench
vdd_dig_high	2.99V	819.53	/remote/us0.../top_spi(48)	TestBench.I0
vdd	2.99V	819.53	/remote/us0.../top_spi(7)	TestBench.I1.I1.I1.I1.I1
vdd	2.99V	819.53	/remote/us0.../top_spi(7)	TestBench.I1.I1.I1.I1.I2
mp1 z a vdd vdd p w=2.00u l=0.35u	2.99V	819.53339	/remote/us0.../top_spi(9)	TestBench.I1.I1.I1.I1.I1

In the above figure, the SPICE signal `vdd` connects to the power supply `VDDH`, and `VDDH` is driven by the SPICE signal `vdd_dig_high`.

Limitations

Following are the limitations of this feature:

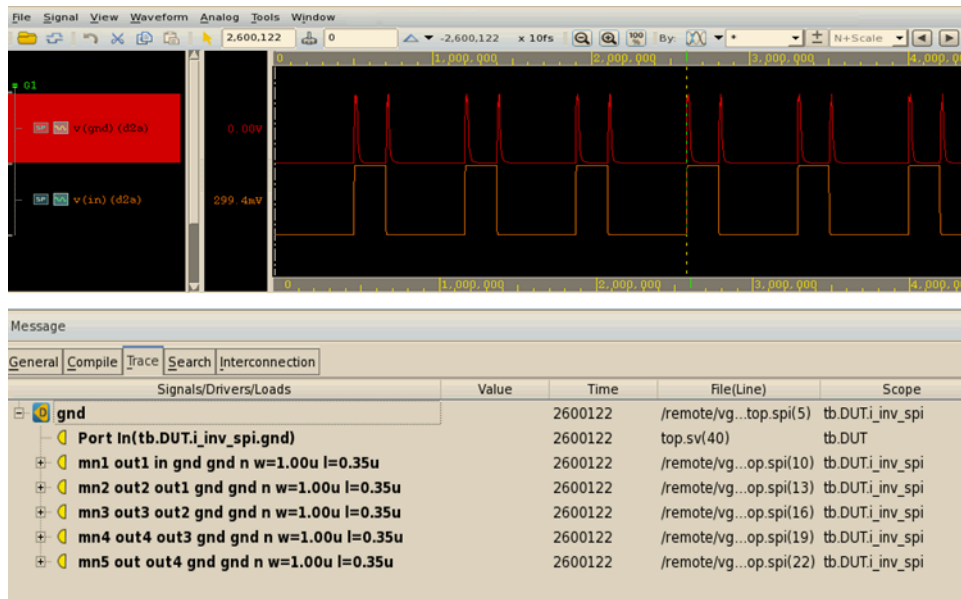
- In a SPICE domain, the contributor analysis support is limited. For example, the ports of primitive devices are inout ports, therefore they all are listed as contributors. The contributors are listed only by judging port directions, as shown in the following figure:

Figure 77 Contributors Analysis is Limited for Primitive Gates

Signals/Drivers/Loads	Value	Time
out	519000	519000
mn5 out out4 gnd gnd n w=1.00u l=0.35u	519000	519000
gnd	519000	519000
out	519000	519000
out4	519000	519000
mp5 out out4 vdd vdd p w=2.00u l=0.35u	519000	519000

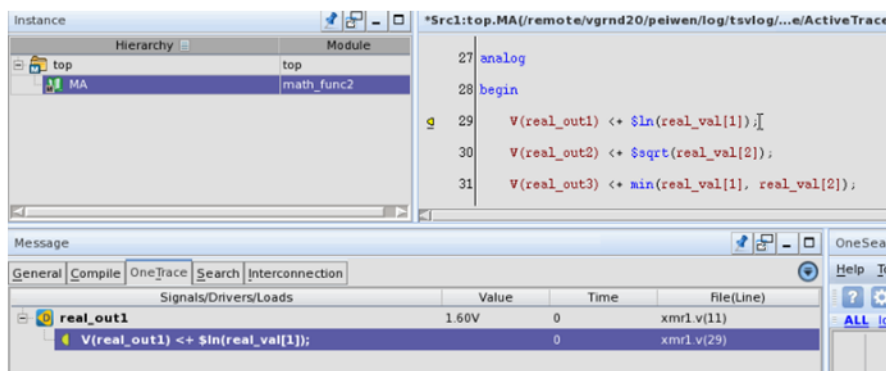
- Active driver analysis is not supported for analog signals with continuous waveform. In this case, the status of drivers is “possible drivers”.

Figure 78 Contributors Analysis is Limited for Primitive Gates



- Active driver analysis is not supported for the drivers under analog block. In this case, the status of drivers is “possible drivers”. For example, in the following figure, Trace Driver for top.MA.real_out1 lists the possible driver under analog block.

Figure 79 Tracing Drivers Under Analog Block

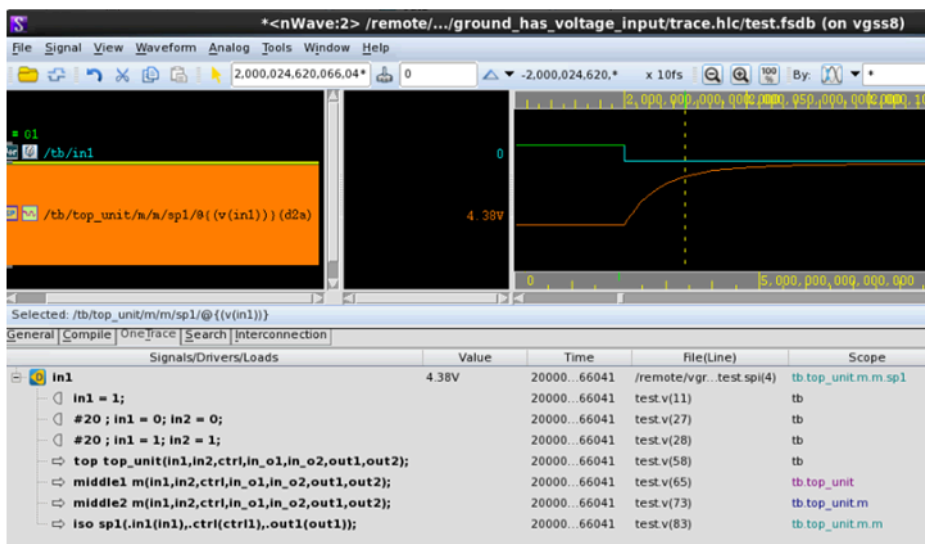


- Active driver analysis for analog signal with continuous waveform (driven by digital signals) is not supported. In this case, the status of driver is “inactive”.

In the following figure, the active trace is triggered at 20.00024620066 s. Verdi cannot figure that it is driven by the value change of digital signal at 20 s since it is continuous

value. Therefore, Verdi cannot get the correct time of the value change. The status can be “inactive”.

Figure 80 Active Driver Analysis for Analog Signal With Continuous Waveform (Driven by Digital Signals)

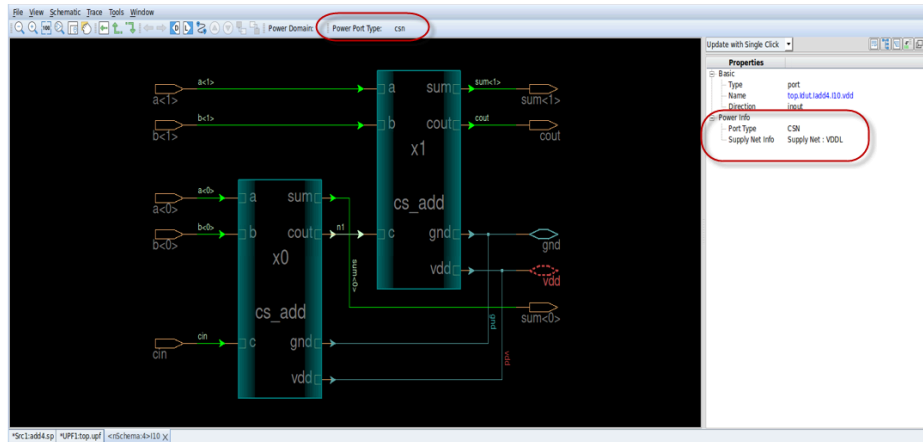


Reporting the SPICE Supply Port Connection to UPF Supply Net

Verdi displays the properties of the UPF supply net connected to the SPICE supply port only using the `connect_supply_net` UPF command. When you right-click a SPICE port (specified by the `connect_supply_net` command) in the schematic and select Properties,

Verdi displays the supply port type (CSN) and supply net information in the Properties dialog box, as shown in the following figure:

Figure 81 Reporting the SPICE Supply Port Connection to UPF Supply Net



4

Power-Aware Debug Functions

The following sections discuss about power-aware debug functions:

- [Highlighting Power Domain, Power Objects, and Power Signals](#)
- [On-Demand Dumping of Low Power Signals into FSDB File in Interactive Debug Mode](#)
- [Reporting Impacted Signals](#)
- [Listing HDL Signals](#)
- [Listing Power Related X](#)
- [Power Sequence Checking](#)
- [Displaying Isolation Cell Information](#)
- [Debugging VCS Low Power Instrumented Objects in Verdi](#)
- [Adding Instrumented Signals to nWave for Different Policies](#)
- [Power Aware Tracing for Boundary MUX Cells](#)
- [Directly Importing Liberty Files and Displaying Power Cell Information](#)
- [Debug Support for nwell and pwell Supply Functions](#)
- [Displaying Power State Group in Power State Table Frame](#)
- [Viewing Supply Net Relationship](#)
- [Viewing the State Names and Supply Values of a Supply Net](#)
- [Dumping Low Power Debug DB XML File During Compilation In UFE Flow](#)

Highlighting Power Domain, Power Objects, and Power Signals

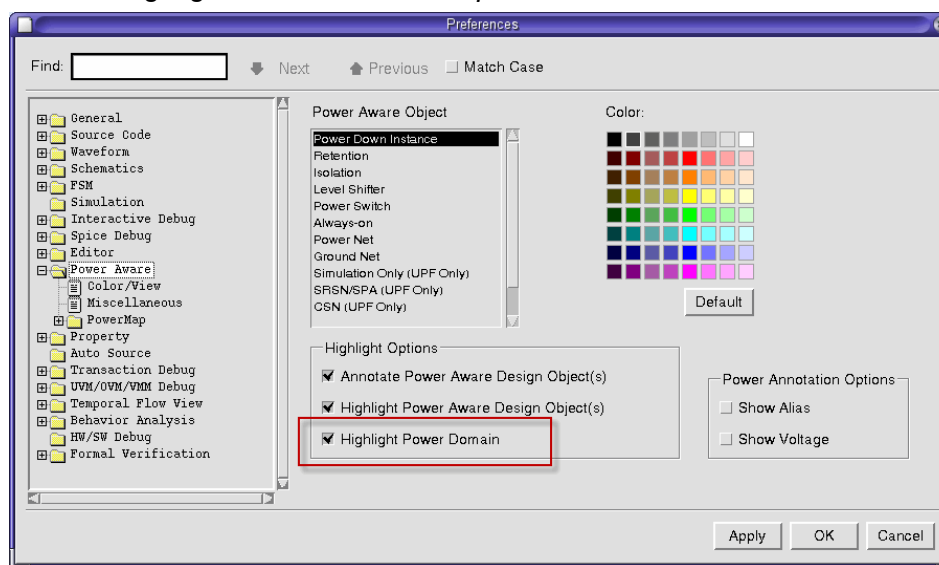
The following sections explain how to highlight power domain and power objects:

- [Highlighting Power Domain](#)
- [Highlighting Power-Aware Objects](#)
- [Highlighting Power Signals](#)

Highlighting Power Domain

To highlight different power domains in the Verdi platform, the **Highlight Power Domain** option (default value is *on*) on the *Power Aware* tab of the *Preferences* form (invoked with **Tools -> Preferences**) must be turned *on*. Refer to the preference setting displayed in [Figure 82](#).

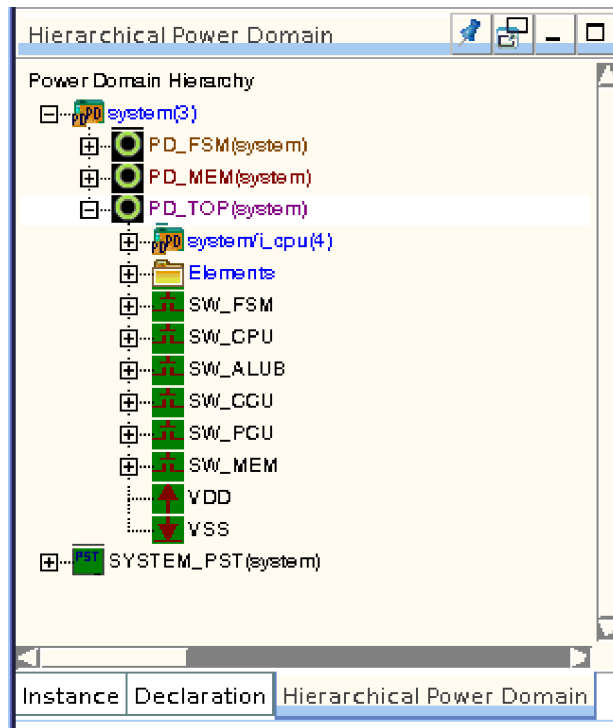
Figure 82 Highlight Power Domains Option



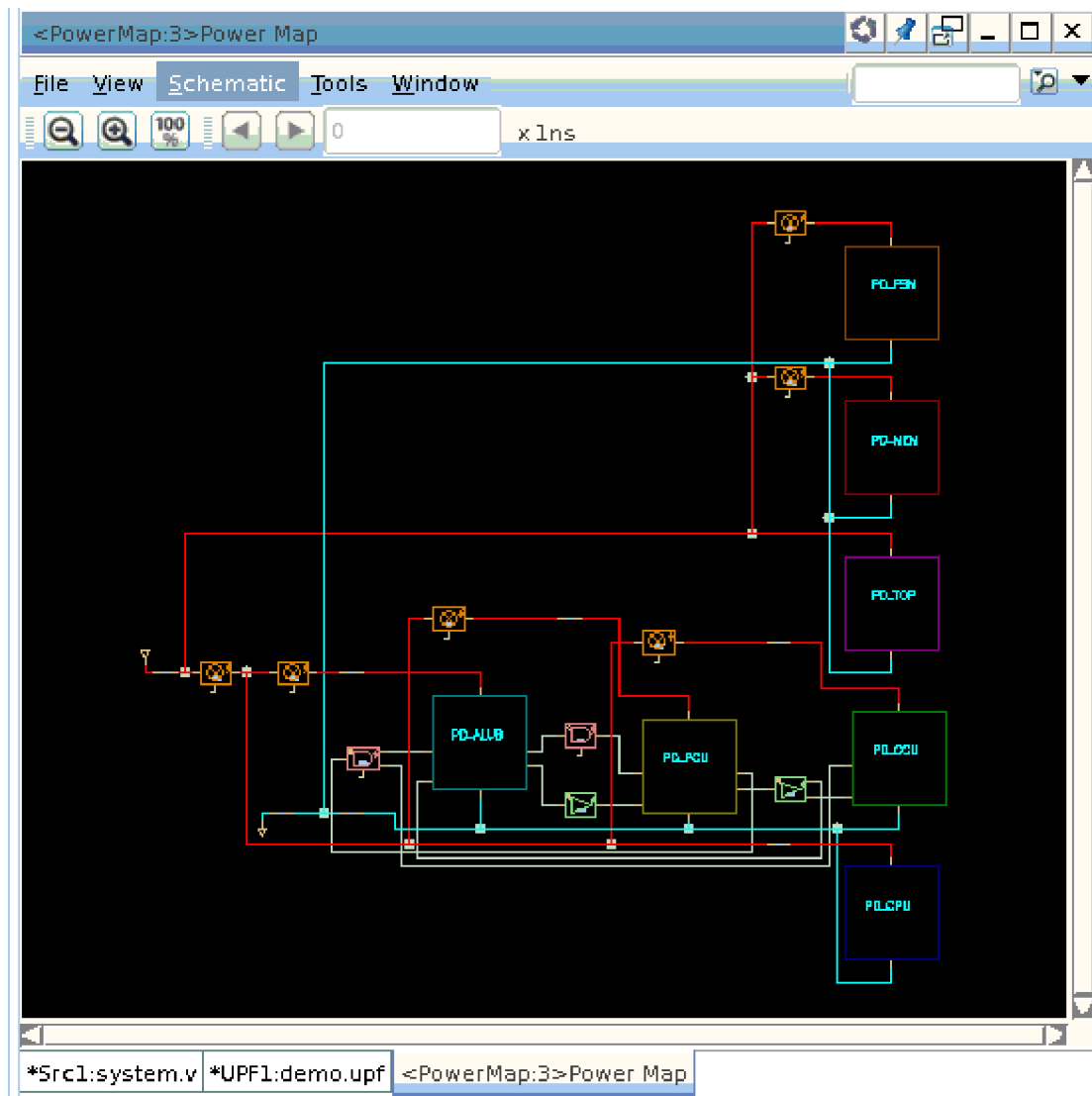
The domain colors can be redefined by the **Power -> Highlight Power Domain** option in the *main* window.

The following figures show the panes that support power domain highlight:

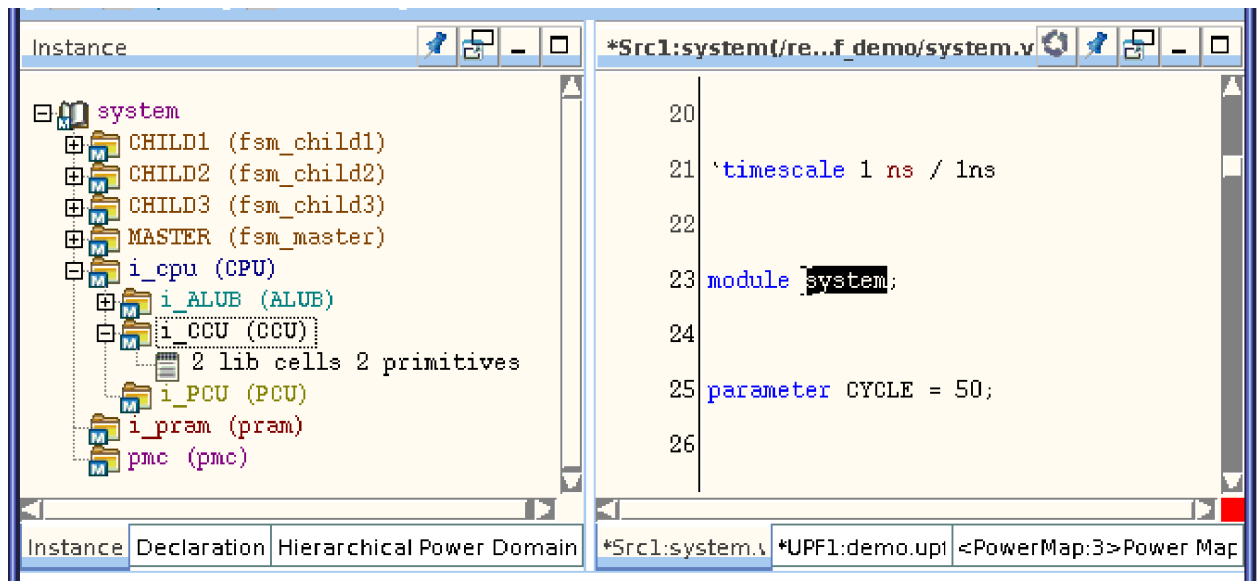
- *Power Domain List Pane*



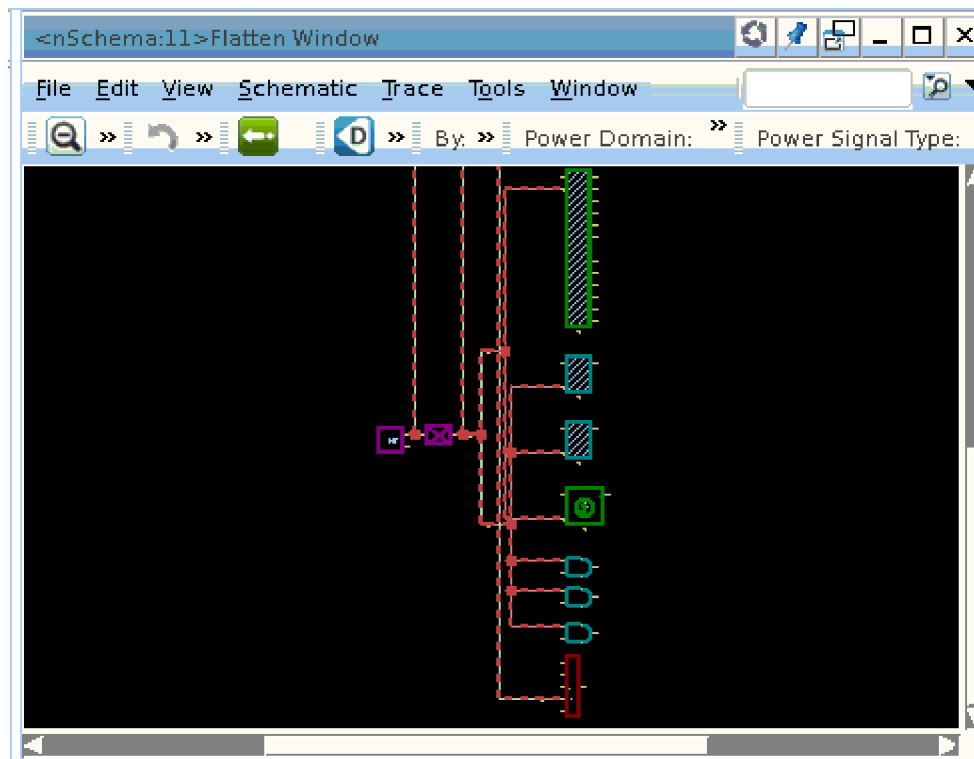
- *Power Map pane*



- *Design Browser pane*

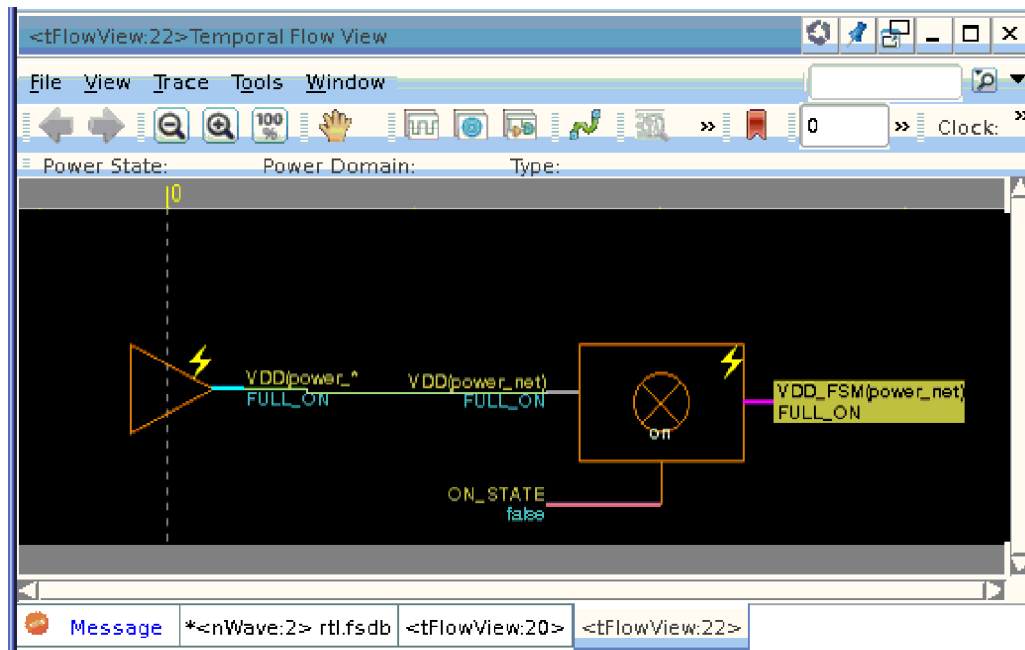


- *nSchema* window



- Temporal Flow View (TFV) pane

The *TFV* pane supports power domains highlight and RET/ISO annotation for retention/isolation latches and flip-flops. The path shown in *TFV* has instances highlighted with the color of the respective power domains and the RET/ISO annotation for retention/isolation latches/flip-flops, if there are any in the path. The power domain name and status is displayed on the toolbar.

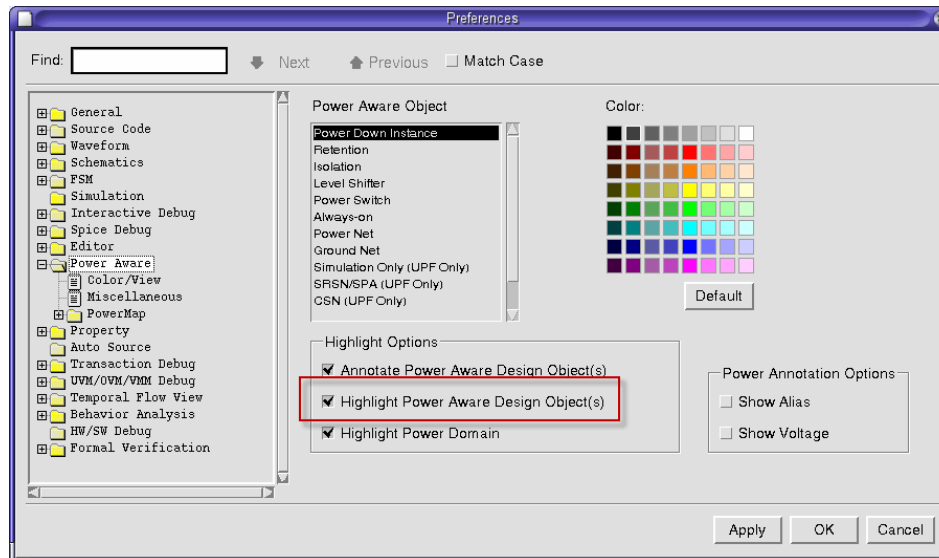


Highlighting Power-Aware Objects

To highlight *Power-Aware Objects*, the **Highlight Power Aware Design Object(s)** option (default value is *on*) must be turned *on*. Refer to the preference setting displayed in [Figure 83](#).

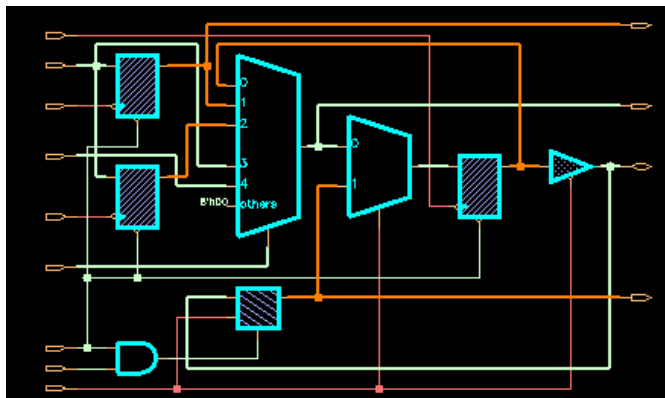
The *Power-Aware Objects* include **Power Down Instance**, **Retention Object**, **Isolation Object**, **Level Shifted Object**, **Power Switch Object**, **Always-on Object**, **Boundary Port Object**, **Power Net**, and **Ground Net**

Figure 83 Settings for Highlighting Power-Aware Design Objects



The following figures show the windows that support power objects highlight:

- *nSchema* Window

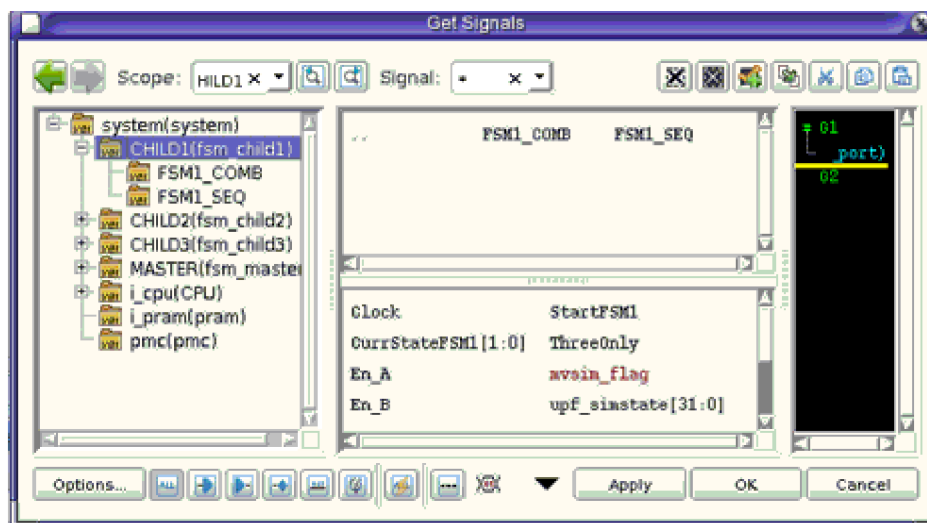


- *nWave* Window

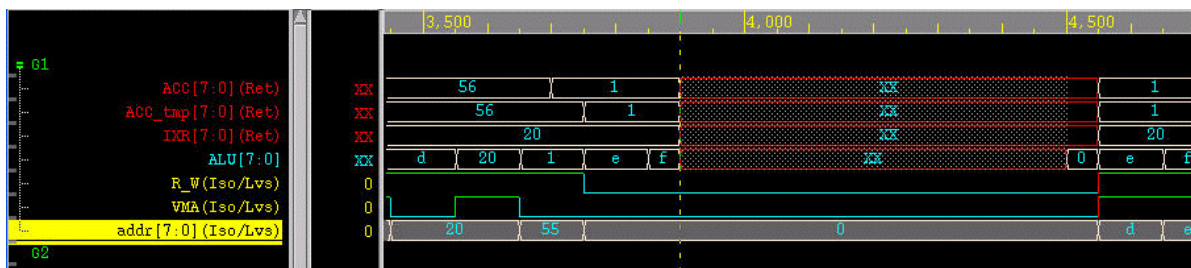
In *nWave*, when the **Get Signals** option is used, the signal color follows the setting in this option. The following figure shows an example of color highlighting for power in the *Get Signals* form. The yellow color represents retention signals.

Chapter 4: Power-Aware Debug Functions

Highlighting Power Domain, Power Objects, and Power Signals



After adding these signals to *nWave*, the signal retains the color setting in the *waveform* view. The *nWave* window also adds a suffix to the signal in the *signal* pane.



On the **Power Aware** page of the *Preferences* form, turning **on** the **Annotate Power Aware Design Object(s)** option adds annotation for Retention, Isolation, and, Level-shifter signals above the signals in the *source code* pane. [Figure 84](#) illustrates an

example of the annotation (where *ret* stands for retention) and color setting for the retention signals.

Figure 84 Color and Annotation for Retention Signals in the Design Source Code pane

```
31 module CCU(TDB, clock, reset, ret CH, ret alu_mode, ret bus_mode
32         ret C6, ret C5, ret C1, ret C0, IR);
33 input [7:0] TDB;
34 input clock;
35 input reset;
36 output [4:0] ret CH;
37 output [2:0] ret alu_mode;
```

On the **Power Aware** page of the *Preferences* form, when both **Extract Isolation Same Nets** and **Highlight/Annotate Power Aware Design Object(s)** options are turned *on*, the equivalent nets that are driven by isolation signals are also highlighted/annotated as isolation signals. Therefore, it is easy to comprehend that these equivalent nets also become isolation signals when the isolation condition occurs.

Highlighting Power Signals

Verdi supports highlighting/coloring of the following power signals:

- Power domain
- Supply net, Supply set, Supply port
- Power switch
- Logic port, Logic net
- Power State Table

PST signals must be generated using the following environment variable:

VERDI_POWER_EVALUATE_VALUE_FOR_PST_AND_PSG

- Power state group

Power state group signals must be generated using the following environment variable:

VERDI_POWER_EVALUATE_VALUE_FOR_PST_AND_PSG

For example, `system.top.GROUP_1.designState`

Chapter 4: Power-Aware Debug Functions

Highlighting Power Domain, Power Objects, and Power Signals

You must use the *Highlight* form to highlight Power signals. To open the *Highlight* form, right-click a power signal and select **Highlight** option. The **Highlight** right-click option is supported in the following views:

Source Code pane, *Signal List* pane, *Power Map* pane, *nWave*, and *Temporal Flow View* (TFV) pane.

You can also select a signal in the above views and perform **Tools -> Highlight** menu command to open the *Highlight* form.

Figure 85 Highlight Option in the Signal List Pane

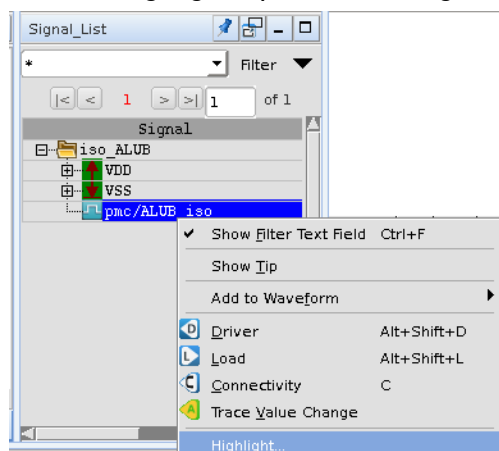
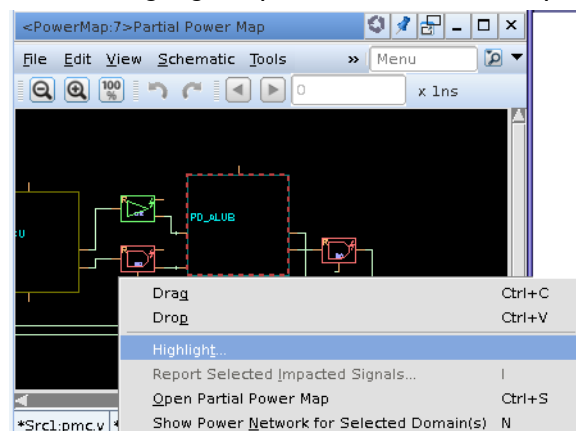
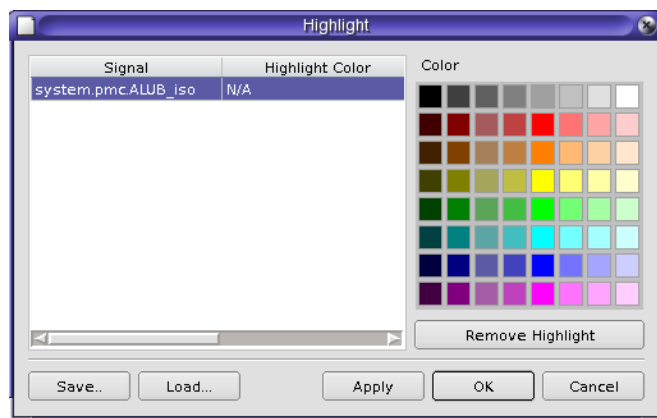


Figure 86 Highlight Option in the Power Map Pane



The *Highlight* right-click option or menu command adds the selected signals in the *Highlight* form with highlight color “N/A”, as shown in the following figure:

Figure 87 *Highlight Form*



You can select a color from the color palette in the *Highlight* form and click **OK** button to apply the selected color to the signal.

To remove the highlight color for a signal, select the signal in the *Highlight* form and click the **Remove Highlight** button.

Figure 88 *Highlighting Signals in Signal Pane, Source Pane, and nWave*

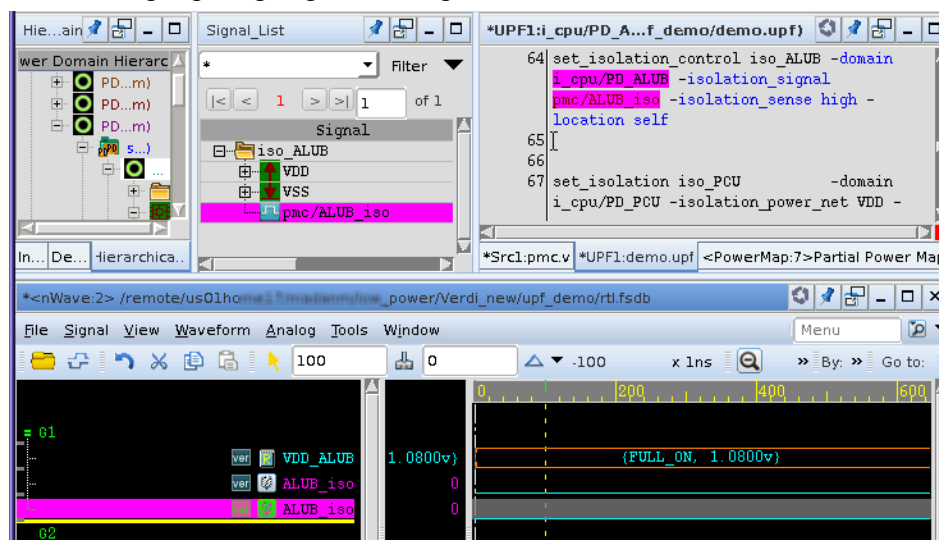
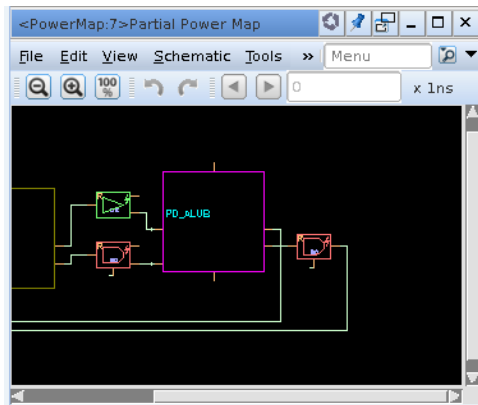


Figure 89 *Highlighting Signals in Power Map*



On-Demand Dumping of Low Power Signals into FSDB File in Interactive Debug Mode

Verdi supports on-demand dumping of the selected low power signals/objects into the FSDB file for debug using the **Dump to FSDB File** right-click option in the Power Domain Hierarchy pane, Power Source Code pane, Signal pane, and Watch pane. This feature does not require the `+fsdb+power` option.

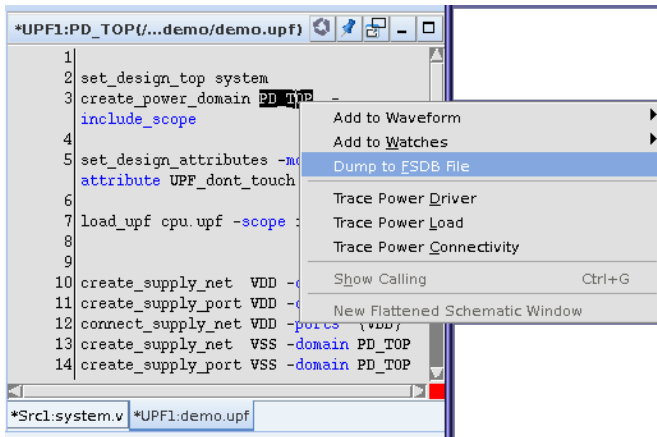
The following sections describe this feature in detail:

- [Power Source Code Pane](#)
- [Power Domain Hierarchy Pane](#)
- [Signal Pane and Watch Pane](#)
- [Dumping PST and PSG Information](#)

Power Source Code Pane

To dump the selected object into the FSDB file, right-click a low power object name in the *Power Source Code* pane and select **Dump to FSDB File** option, as shown in the following figure:

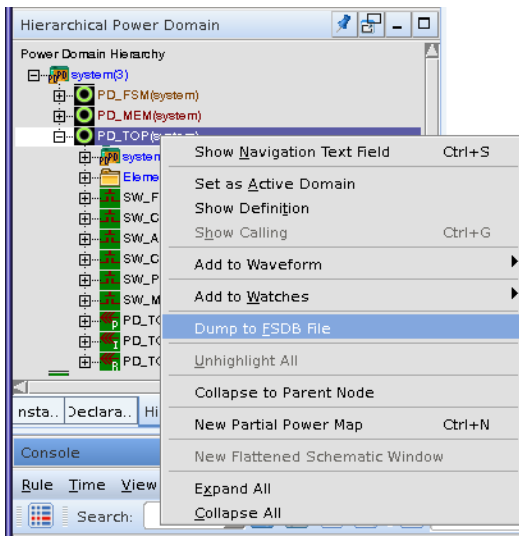
Figure 90 Adding Low Power Objects into the FSDB File



Power Domain Hierarchy Pane

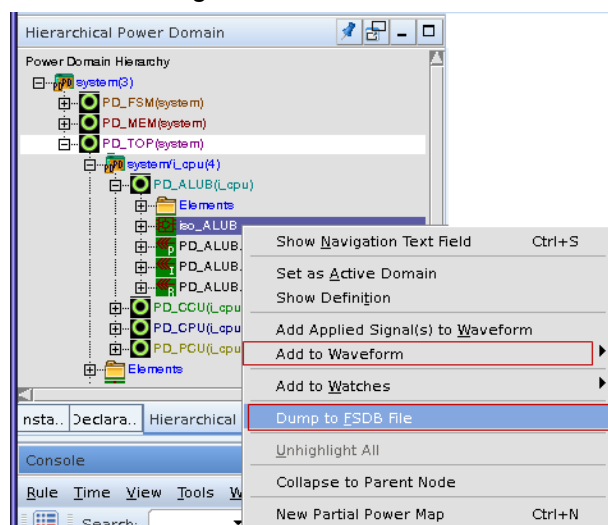
To dump the selected object node into the FSDB file, right-click a low power object node in the *Power Domain Hierarchy* pane and select **Dump to FSDB File** option, as shown in the following figure:

Figure 91 Adding Low Power Object Node into the FSDB File



You can now dump the isolation and retention nodes to the FSDB file and also add them to the waveform using the **Dump to FSDB File** and **Add to Waveform** right-click options, as shown in the following figure:

Figure 92 Adding Isolation/Retention Node into the FSDB File



The **Dump to FSDB File** option dumps only the members of the selected isolation/retention strategy, not the applied HDL signals. You can use the **Add Applied Signal(s) to Waveform** right-click option to add applied HDL signals to waveform.

Signal Pane and Watch Pane

To dump the selected object node into the FSDB file, right-click a low power object node in the *Signal* pane and select **Dump to FSDB File** option, as shown in the following figures:

Figure 93 Adding Power Object Node in Signal Pane into the FSDB File

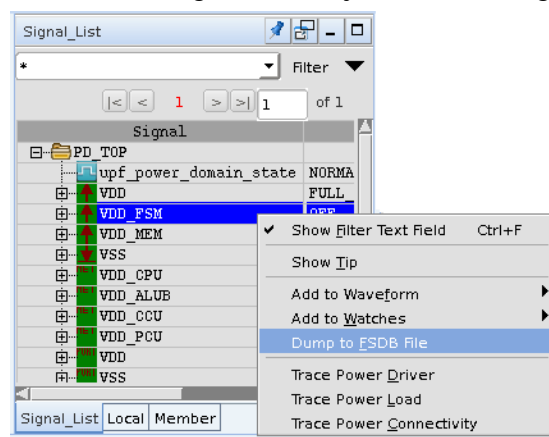
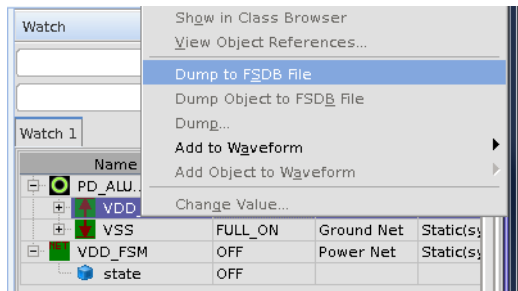


Figure 94 Adding Power Object Node in Watch Pane into the FSDB File



The **Dump to FSDB File** option can be applied to multi selected items.

The attribute signals (state/alias/voltage) are dumped as supply net/supply port.

Dumping PST and PSG Information

The **Dump to FSDB File** option supports dumping of the Power State Table (PST) and Power State Group (PSG) information into the FSDB file. To enable dumping of the PST and PSG information, you must redump FSDB (get PST and PSG value from FSDB generated by VCS).

For backwards compatibility, set the `VERDI_POWER_EVALUATE_VALUE_FOR_PST_AND_PSG` environment variable to `TRUE`. Following is the syntax of this variable:

```
setenv VERDI_POWER_EVALUATE_VALUE_FOR_PST_AND_PSG = TRUE|FALSE
```

Limitations

- Interactive console does not support drag-and-drop name change for `$fsdbDumpVars`.
You cannot directly use drag-and-drop result of power aware signals as parameter of `$fsdbDumpVars` as drag-and-drop does not support full scope name for power aware signals, and also `$fsdbDumpVars` cannot use full scope name as parameter to dump power.
- TFV needs to know accurate value at a specific time, so it is recommended to dump the power scope in the interactive mode.
- Dumper does not support dumping of attribute signals (for example, `tb.VDD.voltage`).

Reporting Impacted Signals

Typically, designers of the UPF power intent files and HDL designers are not the same. To help HDL designers quickly grasp which signals are impacted by the power intent, the **Power -> Report Impacted Signals by Scope** menu option is provided.

When the **Power -> Report Impacted Signals by Scope** menu option is executed, the *Scope Lists* form opens. Select the required scopes in the form and click the **OK** button. The *Report Power Impacted Signals* form opens up. The form lists all signals that are impacted by the power intent (imported UPF files).

The *Rule* column lists the power rule related to each signal; the *From/To Instance* columns list the driving scope and the loaded scope of each signal; and the *From/To Domain* lists the power domain of the driving and loaded scope. For an example, see [Figure 95](#).

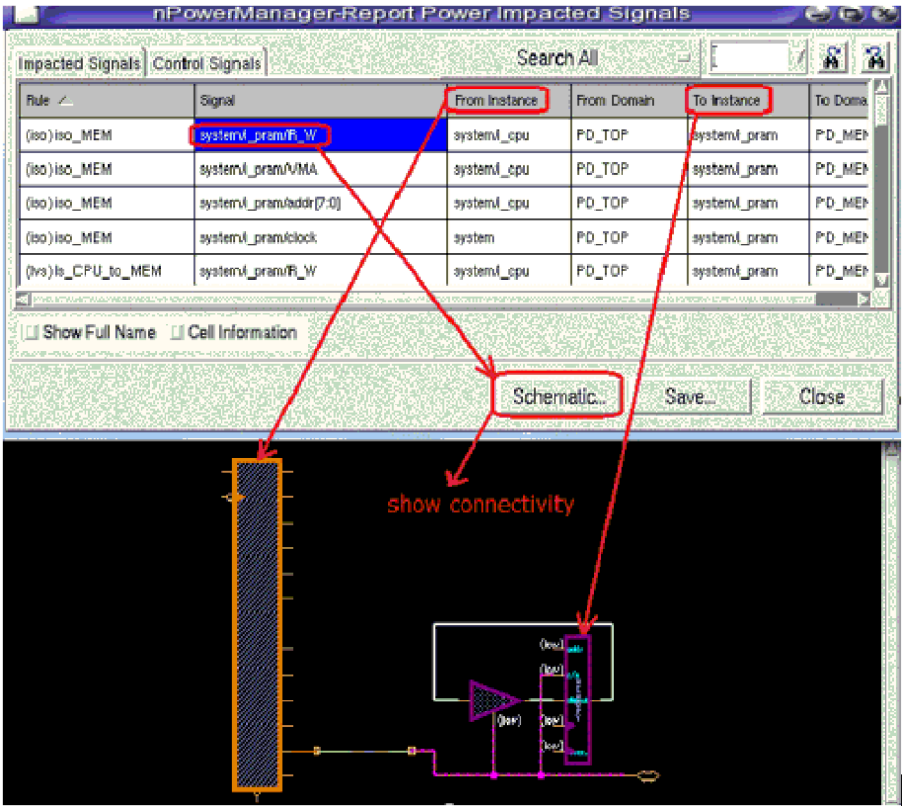
You can sort each column of the *Report Power Impacted Signals* form by clicking the heading of the column. For example, turn *on* the **Show Full Name** option to display signals with full hierarchical names, and click the heading of the *Signal* column to sort. After sorting, the list is sorted by hierarchy that makes it easy to browse by scope.

Figure 95 Report Power Impacted Signals Form

Rule	Signal	From Instance	From Domain	To
{non_cover} iso	systemEn_A	systemCHILD1	PD_FSM	sys
{non_cover} iso	systemEn_AB	systemCHILD2	PD_FSM	sys
{non_cover} iso	systemEn_AC	systemCHILD2	PD_FSM	sys
{non_cover} iso	systemEn_AD	systemCHILD2	PD_FSM	sys
{non_cover} iso	systemEn_B	systemCHILD1	PD_FSM	sys
{non_cover} iso	systemEn_BC	systemCHILD2	PD_FSM	sys
{non_cover} iso	systemEn_BD	systemCHILD2	PD_FSM	sys
{non_cover} iso	systemEn_C	systemCHILD1	PD_FSM	sys
{non_cover} iso	systemEn_CD	systemCHILD2	PD_FSM	sys
{non_cover} iso	systemEn_D	systemCHILD1	PD_FSM	sys
{non_cover} iso	systemFirstDataOutDrv	systemMASTER	PD_FSM	sys

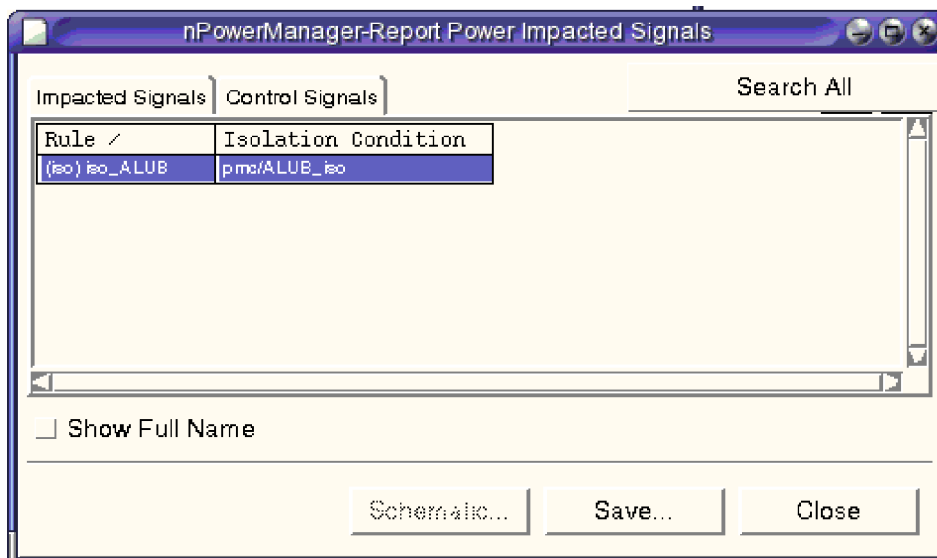
Select a signal from the form and click the *Schematic* button to open an *nSchema* window showing the connectivity of the selected signal.

Figure 96 Show Connectivity of the Selected Signal



The *Control Signals* tab of the *Report Power Impacted Signals* form shows the control signals of the retention and the isolation rules, as illustrated in [Figure 97](#).

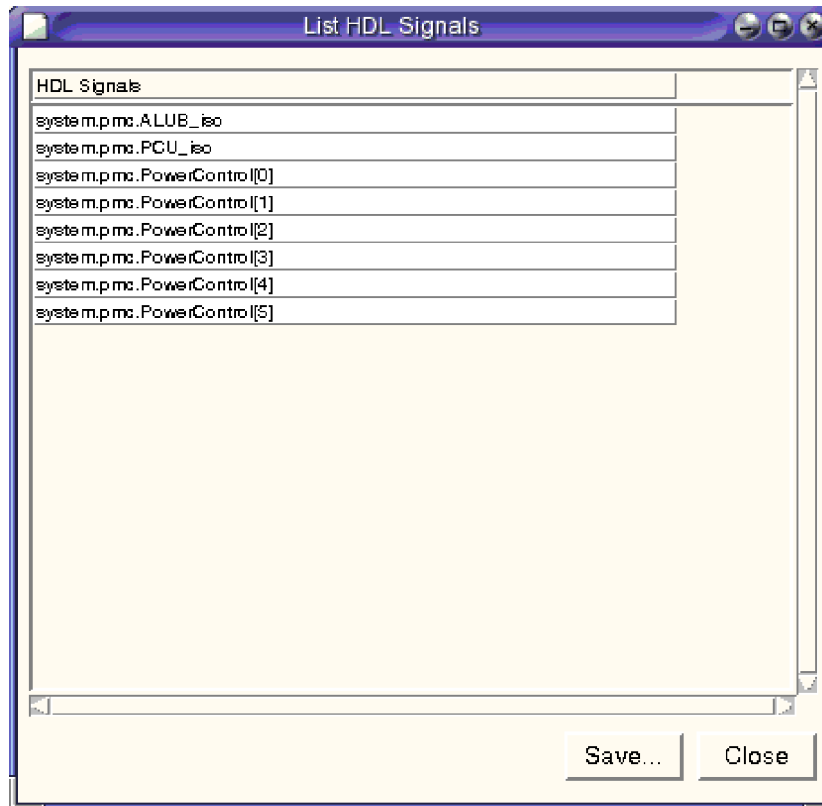
Figure 97 Control Signals of the Isolation and Retention Rules



Listing HDL Signals

You can use the **List HDL Signals** option (select **Power -> List HDL Signals**) to report all used HDL signals contained in the loaded UPF files in the *List HDL Signals* form. You can save the list to a plain text file from this form. The file can be used as an input file for the `$fsdbDumpvarsByfile` dumping option for dumping the UPF related signals.

Figure 98 Listing HDL Signals

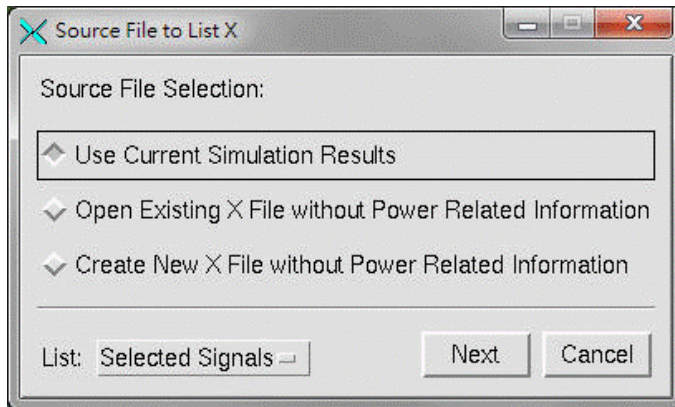


Listing Power Related X

When debugging *unknowns*, to help you to rule out signal *unknowns* (X value) that are affected by its own power domain being switched *off* (such *unknowns* are correct) and focus on *unknowns* that may be real issues, use the **List X (Tools -> List X)** option in the *main* window or *nWave*. This option highlights the *unknowns* that may be real issues (not affected by its own power domain being switched *off*) in yellow.

To use the option, select several signals in the *nWave* signal pane, and then invoke the **Tools -> List X** option from *nWave* or *main* window. The *Source File to List X* form is opened as illustrated in [Figure 99](#).

Figure 99 Source File to List X Form



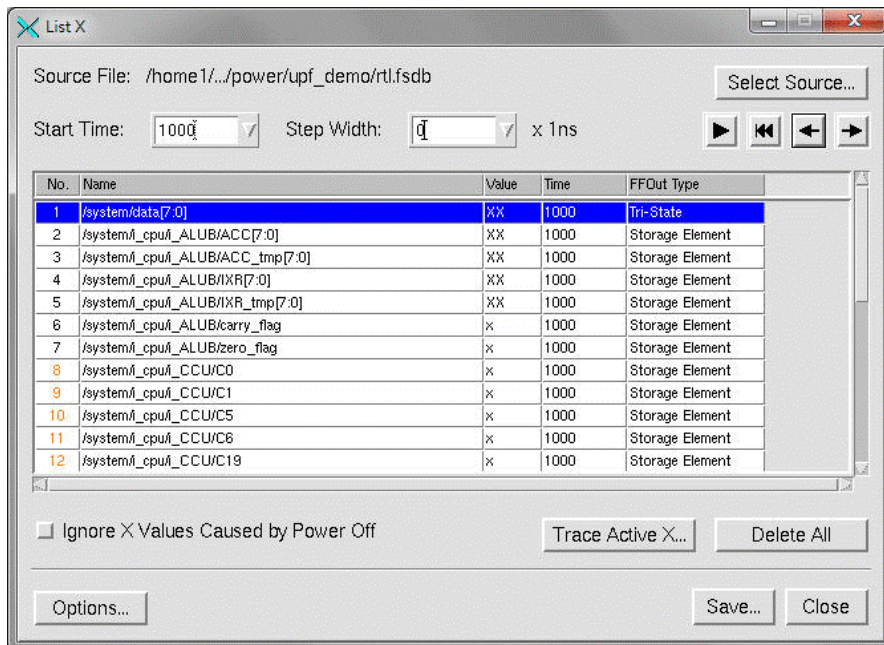
Select the **Use Current Simulation Results** option in the **Source File Selection** field. The other two options do not contain power information.

In the **List:** field

- Select the **Selected Signals** option to list *unknowns* for selected signals in *nWave*.
- Select the **All Signals** option to list *unknowns* for all signals in the design.

Click the **Next** button in the *Source File to List X* form, to view the list of all unknowns in the *List X* form, as illustrated in [Figure 100](#).

Figure 100 List X Form



Click the **Go to next time table** or the **Go to previous time table** buttons to change to next/previous time stamp. If the row number is marked in orange, it means the signal is new compared to the previous time stamp. To mask out *unknowns* that are caused by the power domain *off*, check the **Ignore X Values Caused by Power Off** check box.

To continue to trace these issues, select any row and click the **Trace Active X** button. This performs the **Trace Active X** option using the current time of the selected signal.

Power Sequence Checking

The *Power Sequence Checking* function can be used to check whether the results in the FSDB file matches your original intention in the UPF file. The Verdi platform has defined typical power rules for power-aware designs, and checks whether there are any violations by analyzing the simulation results and the power intent file.

To use this function, invoke the **Power -> Check Power Sequence** menu option from the main window that opens the *Check Power Sequence* form. There exist typical power rules in the following three categories:

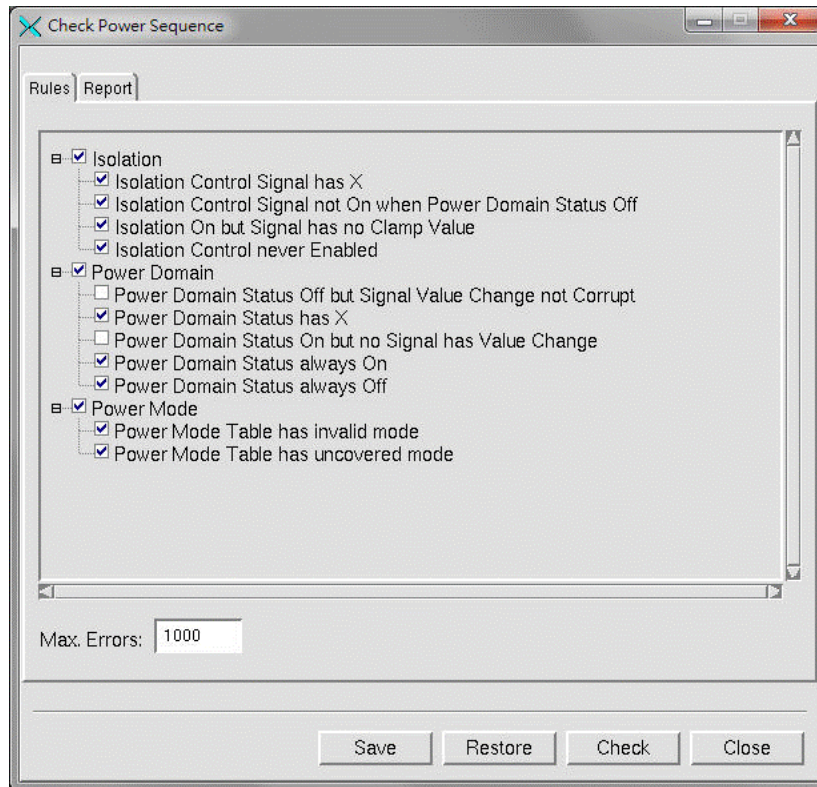
- Isolation
- Power Domain
- Power Mode

Toggle the check box in front of each rule as *on* to check the check box. Specify the maximum error number in the **Max. Errors** field, and click the **Check** button to check the power sequence, as illustrated in [Figure 101](#).

Note:

Two rules are turned *off* by default as they require more time for the checking that may impact the performance. Toggle *on* the check box, if you intend to check these two rules.

Figure 101 Check Power Sequence Form

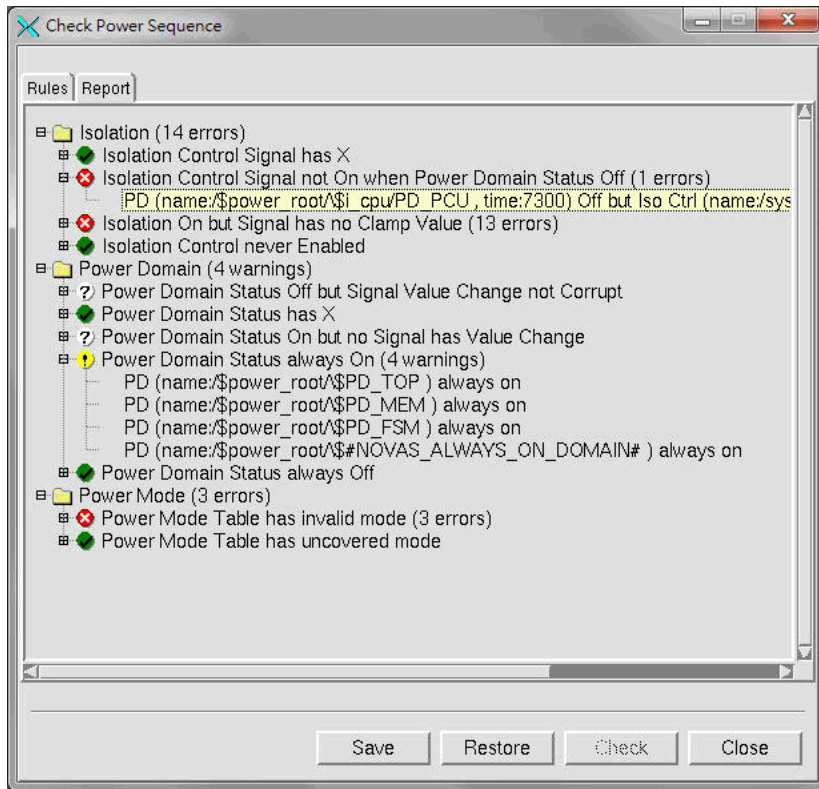


You can click the **Check** button to view the checking results in the **Report** tab of the *Check Power Sequence* form, as shown in [Figure 102](#). Violations are listed under each rule. Click the *plus* icon to expand all violations, or click the *minus* icon to collapse these.

There are some icons displayed in front of each rule that represent severity and status of each rule. The following is the list of these icons along with their meanings:

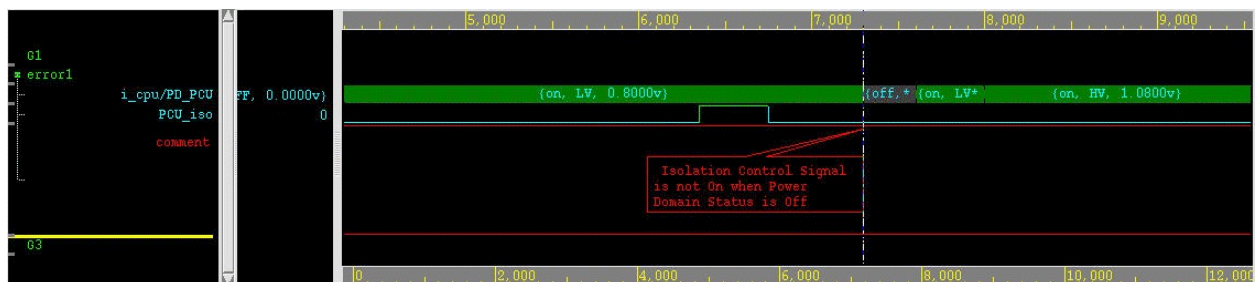
- The  icon signifies *No Error*
- The  icon signifies *Error*
- The  icon signifies *Not Checked*
- The  icon signifies *Warning*

Figure 102 Power Sequence Checking Report



The violations can also be displayed in the *nWave* window for a detailed investigation. Click the *violation* to add the related power domain and control signal to the *nWave* window. A comment is also added to indicate where the violation happens, as illustrated in Figure 103.

Figure 103 Display Violations in the *nWave* Window



Displaying Isolation Cell Information

When the **Display Isolation Annotation Symbol** option in the **Power Aware -> Miscellaneous** page of the *Preferences* form is turned *on*, isolation annotation is displayed as a symbol in the *Source Code* pane.

When the **Display Isolation Annotation Symbol** option is turned *off*, isolation annotation is displayed as a string in the *Source Code* pane (that is, the annotation string for *iso_in* signal is shown as *iso_in*; *iso_out* signal as *iso_out*; *iso_inout* signal as *iso_inout*). The default is *on*.

The setting is saved in the *novas.rc* resource file and the key is `ShowIsoSymbol`.

Note:

This option is supported only in the XML flow.

Figure 104 Annotation Display when Display Isolation Annotation Symbol is Off

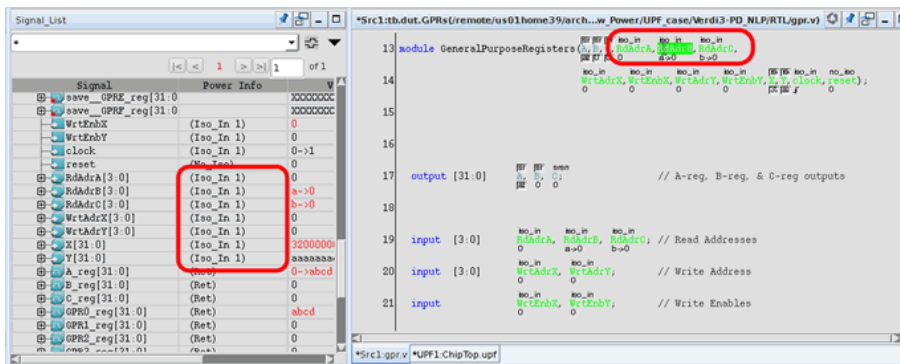
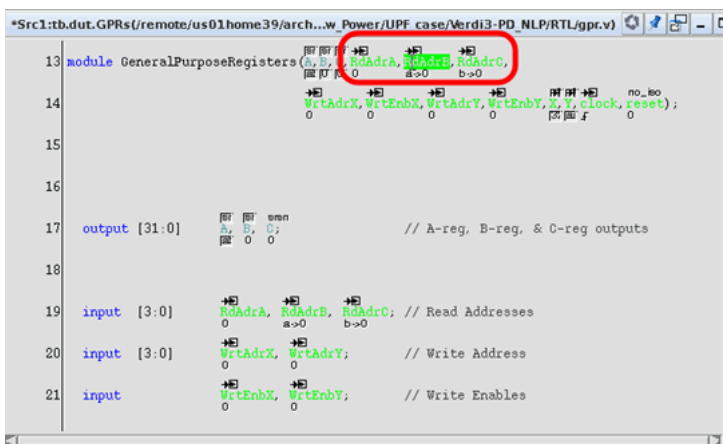


Figure 105 Annotation Display when Display Isolation Annotation Symbol is On



Annotation in the *nTrace* source code pane is summarized as follows:

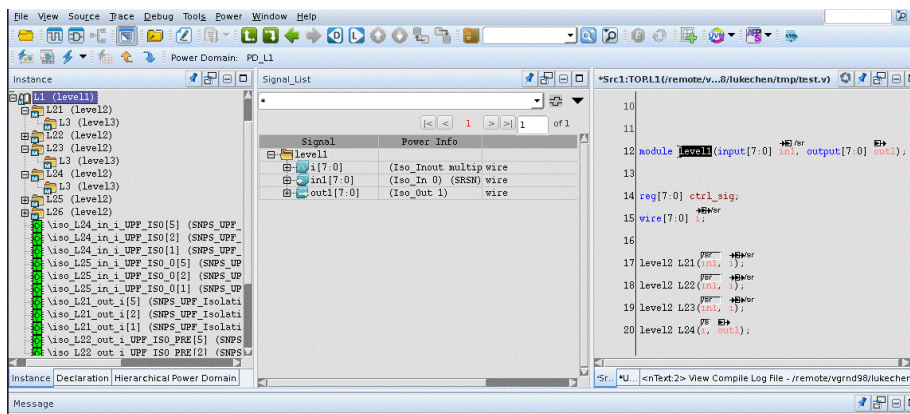
- If an HDL signal connects to an isolation cell directly, and it is used as the output of the connected isolation cell, the signal is marked as `iso_out` signal, and is annotated with the  symbol.
- If an HDL signal connects to an isolation cell directly, and it is used as the input of the connected isolation cell, the signal is marked as `iso_in` signal, and is annotated with the  symbol.
- If an HDL signal connects to an isolation cell directly, and it is used as both input and output of the connected isolation cell, the signal is marked as `iso_inout` signal, and is annotated with the  symbol.

Annotation in the **Power Info** column of the *nTrace* **Signal List** pane is shown as follows:

- `iso_in` for `iso_in` signals
- `iso_out` for `iso_out` signals
- `iso_inout` for `iso_inout` signals

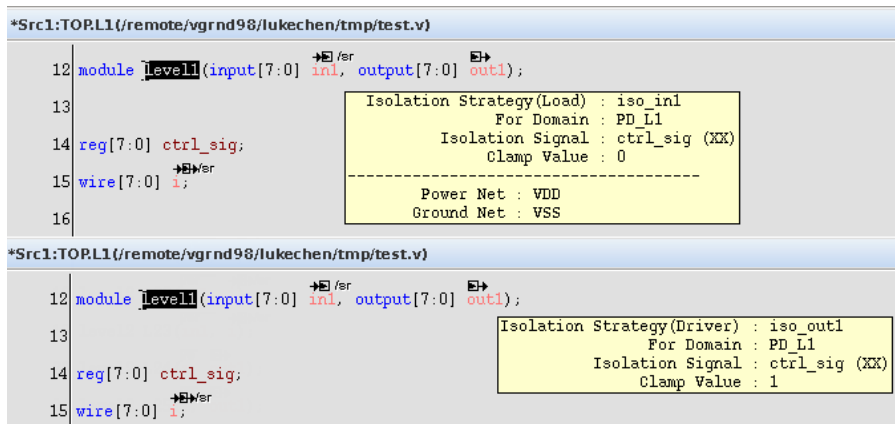
The following figure illustrates power annotation for signals in the *nTrace* **Signal List** pane:

Figure 106 Displaying Isolation Cell Information in *nTrace* **Signal List** Pane



In the *nTrace* source code pane, for `iso_in`, `iso_out`, and `iso_inout` signals, tips are shown with the driver/load isolation rules as illustrated in the following figure.

Figure 107 Tip Display



Power Annotation for isolation signals is also displayed in the *nWave* window (in the form of postfix) as follows:

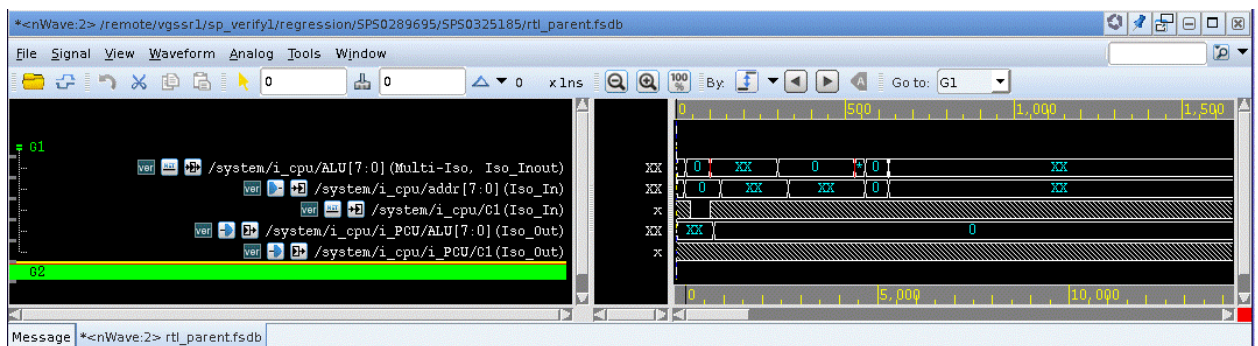
- Iso_In for iso_in signals
- Iso_Out for iso_out signals
- Iso_Inout for iso_inout signals

Note:

If there are multiple isolation rules applied to a signal, the "Multi-Iso" postfix along with the direction of the isolation cell connection is added to the signal in the *nWave* window.

The following figure illustrates power annotation for signals in the *nWave* window:

Figure 108 Displaying Isolation Cell Information in the *nWave* Window

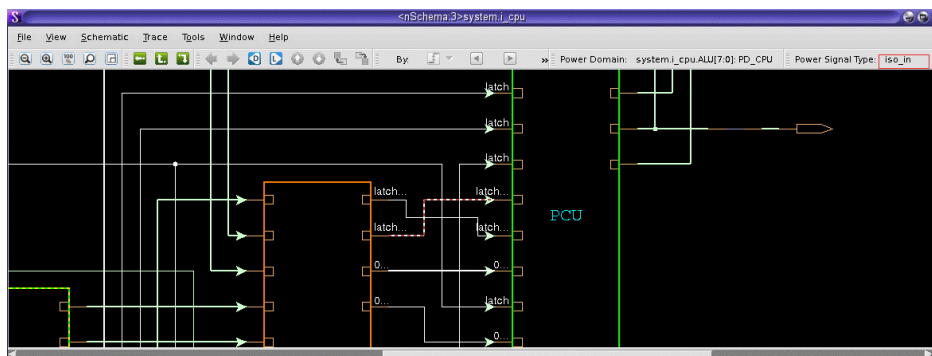


Power Annotation for isolation signals is also displayed in the *nSchema* window as follows:

- `iso_in` for `iso_in` signals
- `iso_out` for `iso_out` signals
- `iso_inout` for `iso_inout` signals

The following figure illustrates power annotation for `iso_in` signals in the *nSchema* window:

Figure 109 Displaying Isolation Cell Information in *nSchema* Window



Debugging VCS Low Power Instrumented Objects in Verdi

Verdi allows you to debug low power objects instrumented by VCS NLP. You can use the low power debug DB, an XML format file, dumped by VCS NLP to handle all instrumentation scenarios. The low power debug DB contains the following information:

- Instrumented objects information (For nTrace and Power Domain List debugging)
- Connection relationship information (For Trace Driver/Trace Load, nSchema, and Power Map debugging)

This feature helps you to get consistent results between VCS NLP and the Verdi platform. You can use VCS to generate the low power instrumented objects and reduce the duplicate effort for the Verdi parser to create the low power instrumented objects.

Use Model

Dumping Low Power Debug DB XML File During Simulation

The following is the syntax to dump DB XML file during simulation:

```
% vcs <design file> -upf <upf file> -kdb -debug_access+all <compile options>
```

Example:

```
% vcs -f *.f -upf *.upf -kdb -power=cov_pst_transition -debug_access+all
-l vcs.log
```

Note:

You can also dump a compressed XML file.

Loading the Dump XML Database File in Verdi

If you have generated Verdi kdb database using vlogan/vhdlan, VCS NLP generates the XML file. You can use the `verdi -ssf novas.fsdb` or `simv -verdi` option to automatically load the XML file.

Following is the syntax to load the dump XML database file if the kdb is generated from vericom/vhdlcom:

```
verdi -lp_xml[=file_name.xml]
```

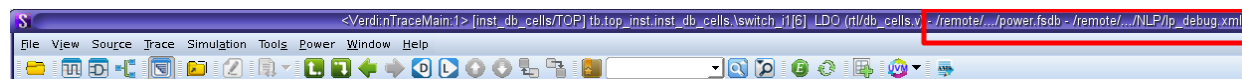
Example:

```
verdi -f *.f -sverilog -lp_xml=power.xml &
```

Viewing Dump XML Database File in Verdi

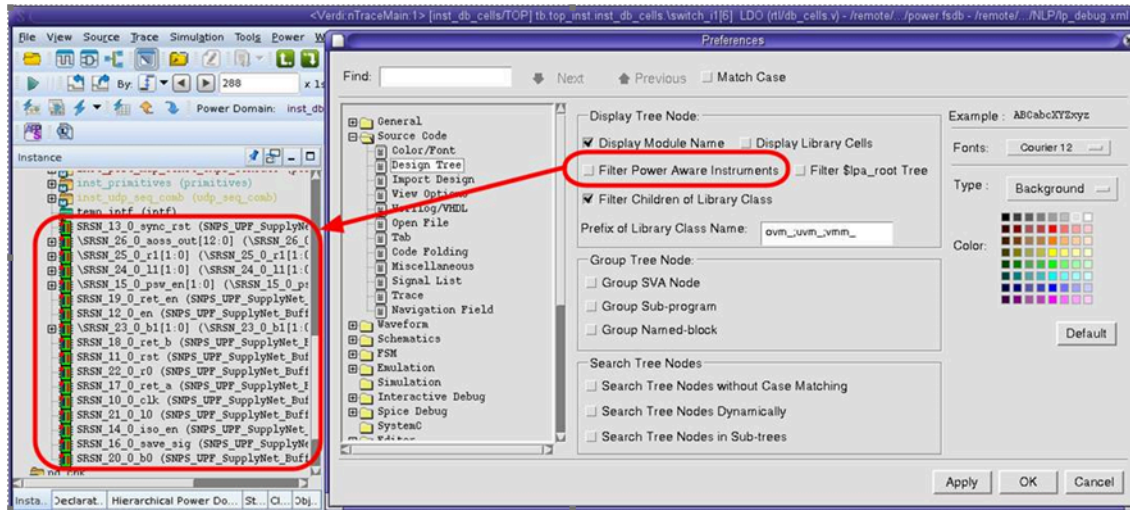
Verdi shows the name of the loaded XML file in the top banner, as shown in the following figure:

Figure 110 Viewing the name of the xml File in Top Banner



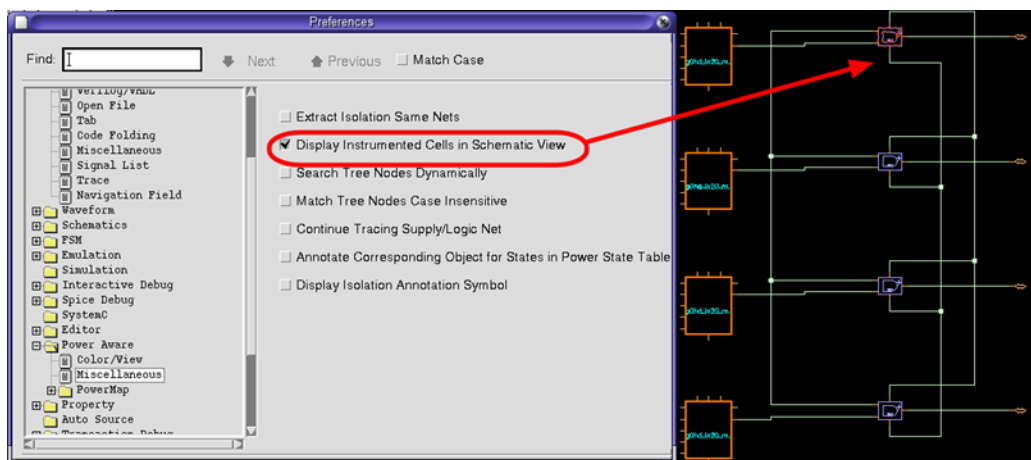
To view the instrumented power objects in the Design Hierarchy tree, clear the **Filter Power Aware Instruments** option in the **Source Code -> Design Tree** page of the **Preferences** form (invoked with **Tools -> Preferences**), as shown in the following figure:

Figure 111 Display Instrumented Power Objects in the Design Hierarchy Tree



To view the instrumented objects and connection relationship information in Verdi, the **Display Instrumented Cells in Schematic View** option in the **Power Aware -> Miscellaneous** page of the **Preferences** form (invoked with **Tools -> Preferences**) must be turned on. Refer to the preference setting displayed in Figure 112.

Figure 112 Display Instrumented Cells in Schematic View

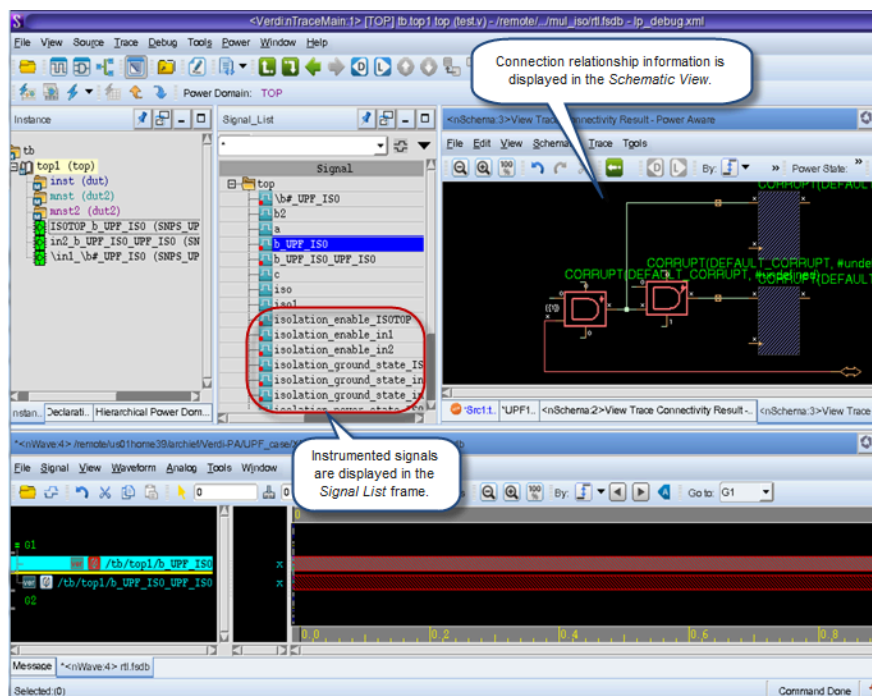


Chapter 4: Power-Aware Debug Functions

Debugging VCS Low Power Instrumented Objects in Verdi

When **Display Instrumented Cells in Schematic View** option is turned *on*, the instrumented signals are shown in the *Signal List* pane and the connection relationship information is shown in the *Schematic View*, as shown in the following figure:

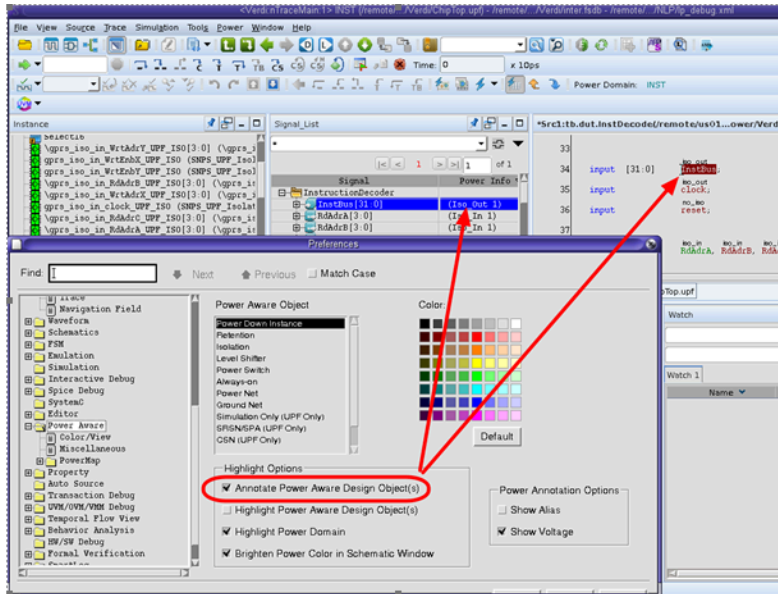
Figure 113 Viewing Instrumented Cells and Connection Relationship



Verdi shows the input or output of the isolation signals in the *Signal List* pane and *Source Code* pane, if you turn on the **Annotate Power Aware Design Object(s)** option in the

Power Aware page of the *Preferences* form (invoked with **Tools -> Preferences**), as shown in the following figure:

Figure 114 Display Input or Output of the Isolation Signals



Adding Instrumented Signals to nWave for Different Policies

The **Power Debug** mode in *Verdi Automated Debug Platform* provides *Signal List* pane right-click option: **Add Instrumented Signals to Waveform** to add instrumented signals for different policies or strategies in the *nWave* window. The option menu includes **Isolation Signals**, **Retention Signals**, **SRSN Signals**, and **SPA Signals** sub-options.

The following sections discuss about how to view the instrumented signals for the respective strategies/policies:

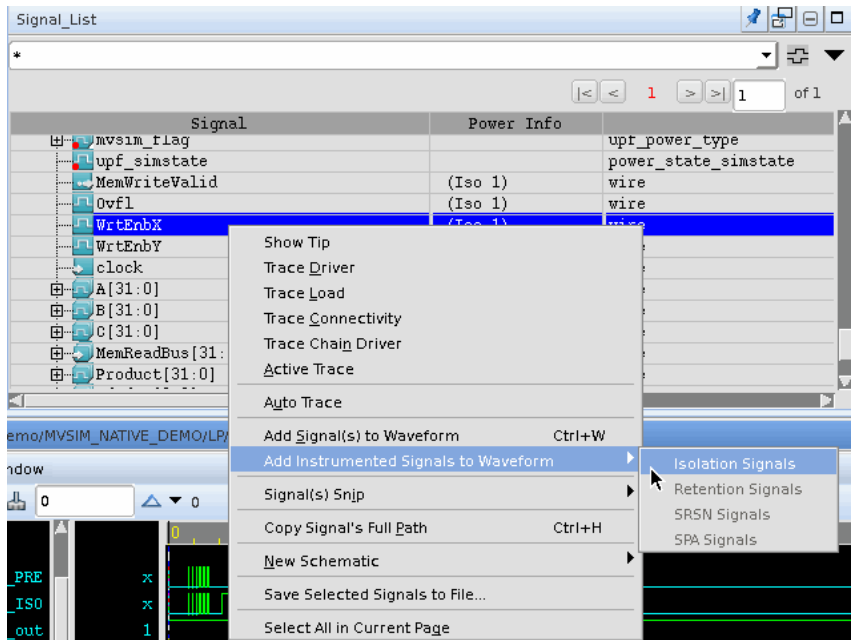
- [Isolation Signals](#)
- [Retention Signals](#)
- [SRSN Signals](#)
- [SPA Signals](#)

Isolation Signals

The **Add Instrumented Signals to Waveform -> Isolation Signals** option adds the instrumented isolation signals to the *nWave* window. As illustrated in the [Figure 115](#), right-click on a signal (for example, *WrtEnbX*) in the *Signal List* pane and click **Add**

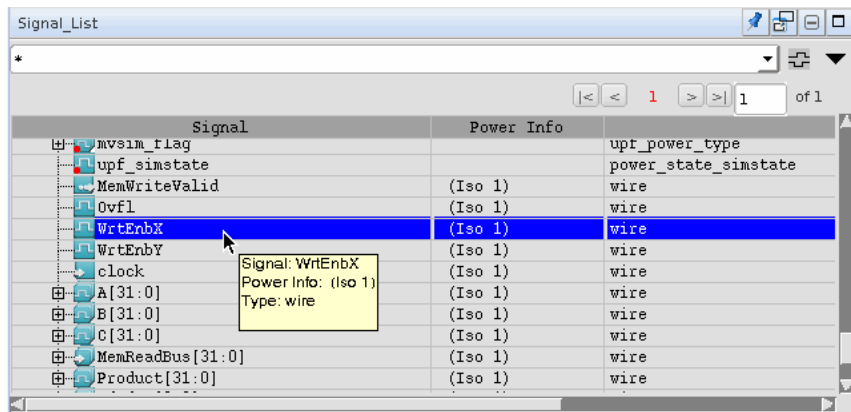
Instrumented Signals to Waveform -> Isolation Signals option to add the instrumented isolation signals to the *nWave* window.

Figure 115 Add Instrumented Isolation Signals to nWave



To enable tooltip for the signals appearing in the *Signal List* pane, right-click a signal and select **Show Tip** option. The tooltip displays signal name, type and power information, as illustrated in Figure 116

Figure 116 Tooltip for Signals in Signal List Pane

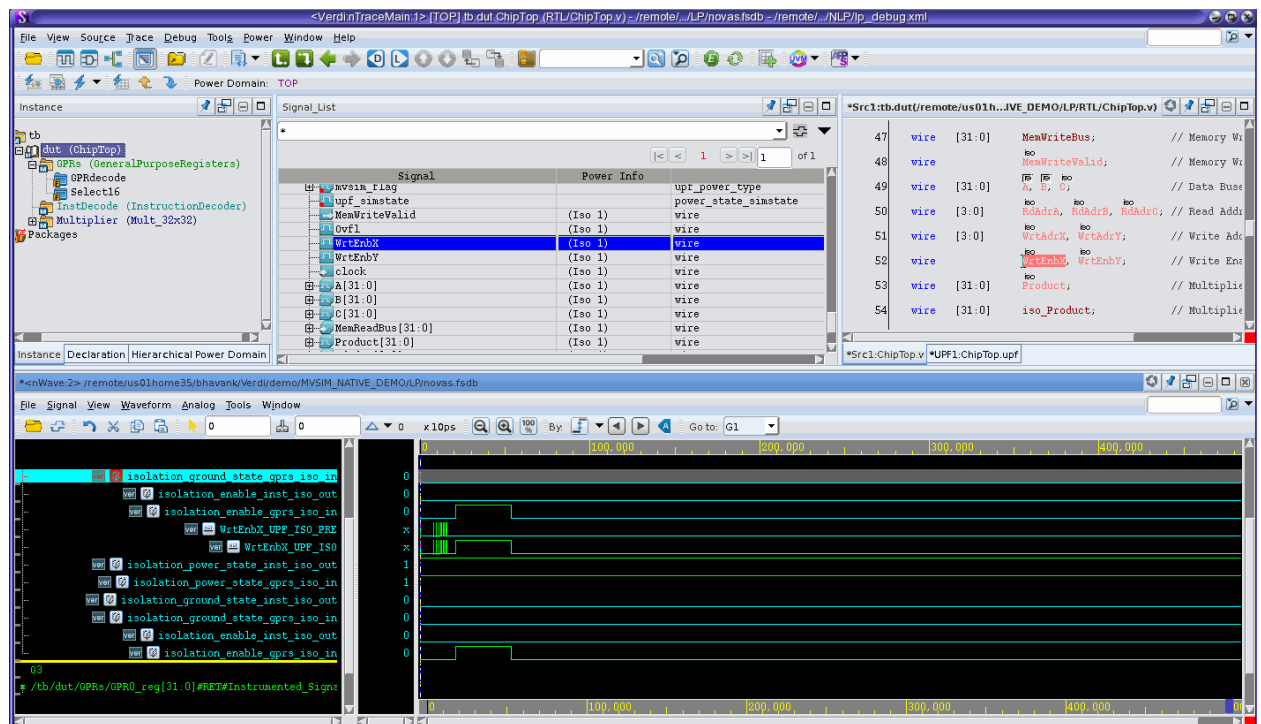


The isolation instrumented signals get added to the *nWave* window with the following naming convention:

- *_UPF_ISO
- *_UPF_ISO_PRE
- isolation_*

This is illustrated in [Figure 117](#).

Figure 117 Isolation Signals Added to nWave Window



Also, if you right click an isolation signal in the *nWave* signal pane and select **Power -> Isolation Rules -> Add Instrumented Signals**, the instrumented isolation signals get added to the *nWave* window.

Retention Signals

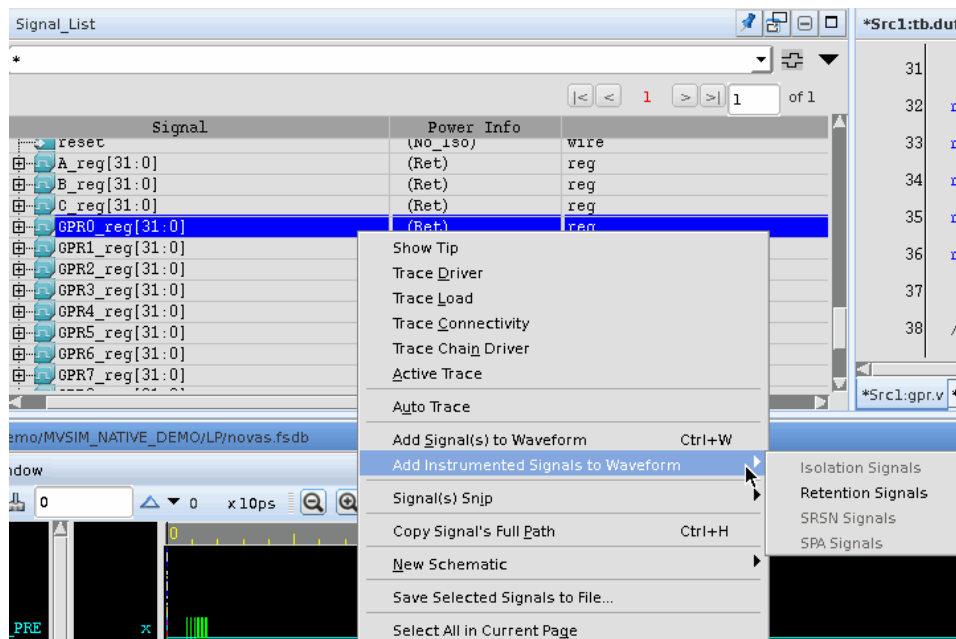
The **Add Instrumented Signals to Waveform -> Retention Signals** option allows you to add the instrumented retention signals to the *nWave* window.

As illustrated in the [Figure 118](#), right-click a signal (for example, *GPR0_reg[31:0]*) in the *Signal List* pane and click **Add Instrumented Signals to Waveform -> Retention Signals** option to add the instrumented retention signals to the *nWave* window.

Chapter 4: Power-Aware Debug Functions

Adding Instrumented Signals to nWave for Different Policies

Figure 118 Add Instrumented Retention Signals to nWave

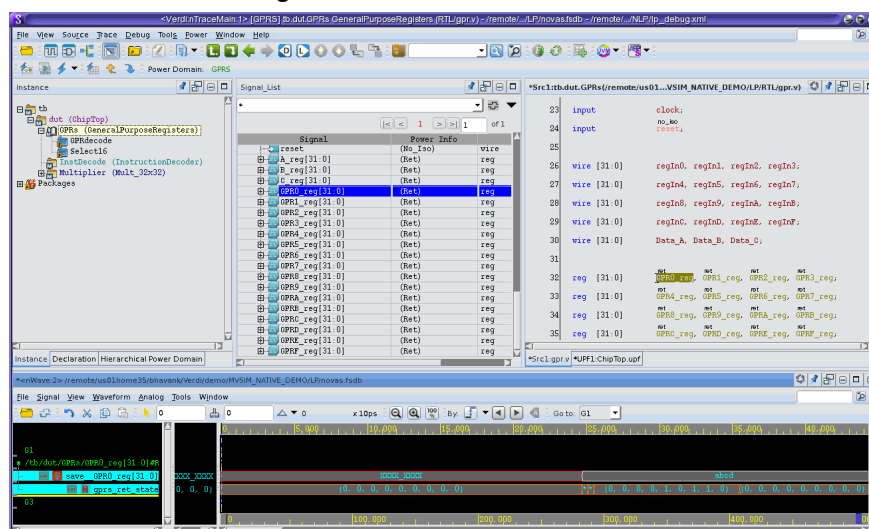


The instrumented retention signals are added to the *nWave* window with the following naming convention:

- `save_*`
- `gprs_ret_state`

This is illustrated in [Figure 119](#).

Figure 119 Retention Signals Added to nWave Window



Also, if you right click a retention signal in the *nWave* signal pane and select **Power -> Retention Rules -> Add Instrumented Signals**, the instrumented retention signals are added to the *nWave* window.

SRSN Signals

The **Add Instrumented Signals to Waveform -> SRSN Signals** option allows you to add the instrumented SRSN (*set_related_supply_net*) signals to the *nWave* window.

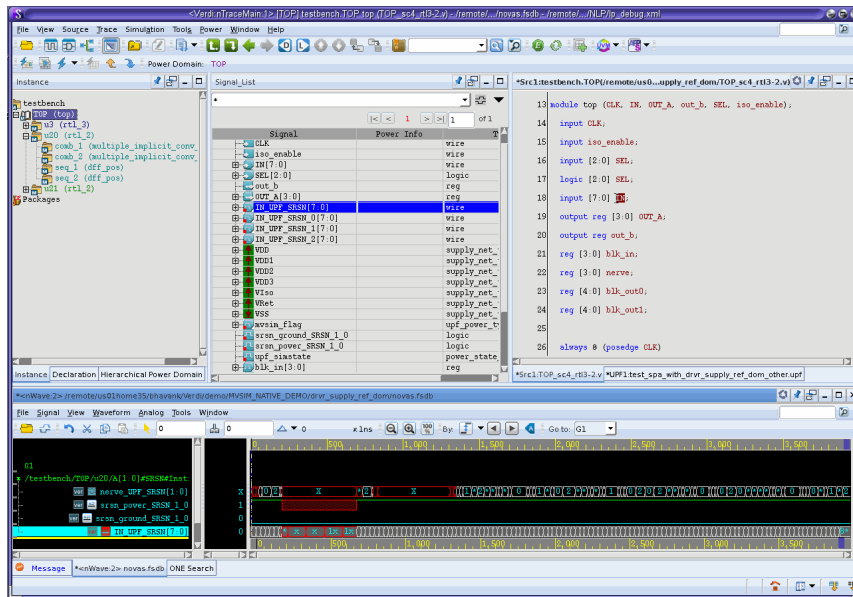
Right-click a signal in the *Signal List* pane and click **Add Instrumented Signals to Waveform -> SRSN Signals** option to add the instrumented SRSN signals to the *nWave* window.

The SRSN instrumented signals get added to the *nWave* window with the following naming convention:

- *_UPF_SRSN
- srsn_power_SRSN
- srsn_ground_SRSN

This is illustrated in [Figure 120](#).

Figure 120 SRSN Signals Added to nWave Window



Also, if you right click a SRSN signal in the *nWave* signal pane and select **Power -> SRSN Rules -> Add Instrumented Signals**, the instrumented SRSN signals are added to the *nWave* window.

SPA Signals

The **Add Instrumented Signals to Waveform -> SPA Signals** option allows you to add the instrumented SPA (`set_port_attributes`) signals to the *nWave* window.

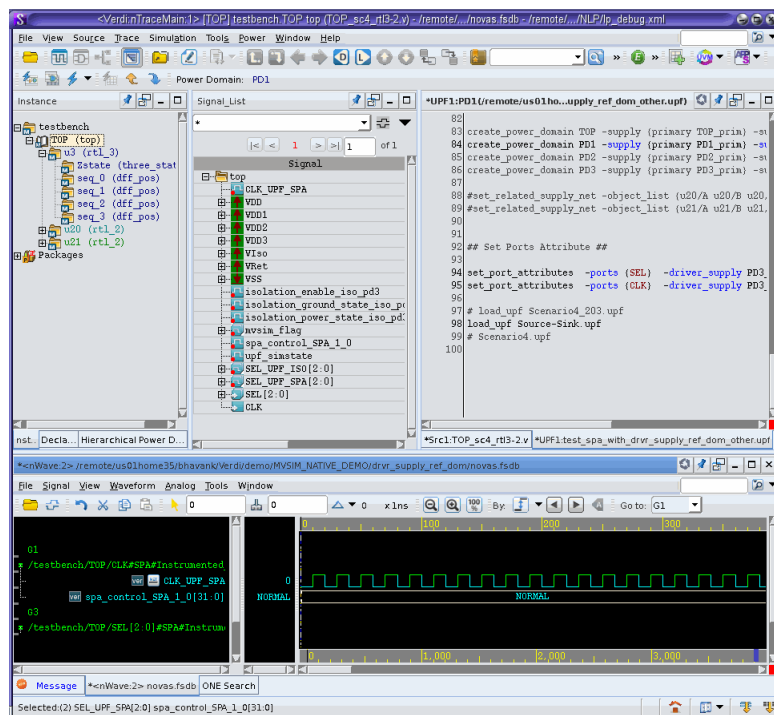
Right-click a signal in the *Signal List* pane and click **Add Instrumented Signals to Waveform -> SPA Signals** option to add the instrumented SPA signals to the *nWave* window.

The SPA instrumented signals get added to the *nWave* window with the following naming convention:

- `*_UPF_SPA`
- `spa_control_SPA`

This is illustrated in Figure 121.

Figure 121 SPA Signals Added to nWave Window



Also, if you right click a SPA signal in the *nWave* signal pane and select **Power -> SPA Rules -> Add Instrumented Signals**, the instrumented SPA signals are added to the *nWave* window.

Power Aware Tracing for Boundary MUX Cells

Verdi supports tracing boundary MUX cells of power domain in *nTrace*, *nWave*, *nSchema*, and *Temporal Flow View*. Verdi also allows you to,

- Perform power corruption calculation for power corrupted by a random cell value.
- Trace the root cause for power corruption using automatic power tracing.

This section consists of the following sub-sections:

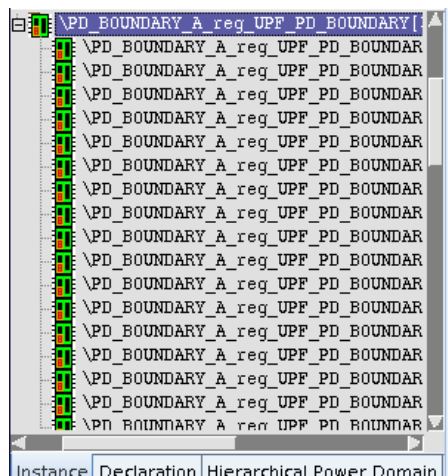
- [Support for Boundary MUX Cells in nTrace](#)
- [Support for Boundary MUX Cells in nWave](#)
- [Support for Boundary MUX Cells in nSchema](#)
- [Support for Boundary MUX Cells in Temporal Flow View](#)
- [Power Corruption Model in Temporal Flow View](#)

Support for Boundary MUX Cells in nTrace

nTrace displays boundary cells in the *Hierarchy Tree* tab and provide corresponding debug functionalities. The Boundary MUX tree nodes are bit-based and are grouped under an array node. The naming rule for Boundary MUX nodes is as follows:

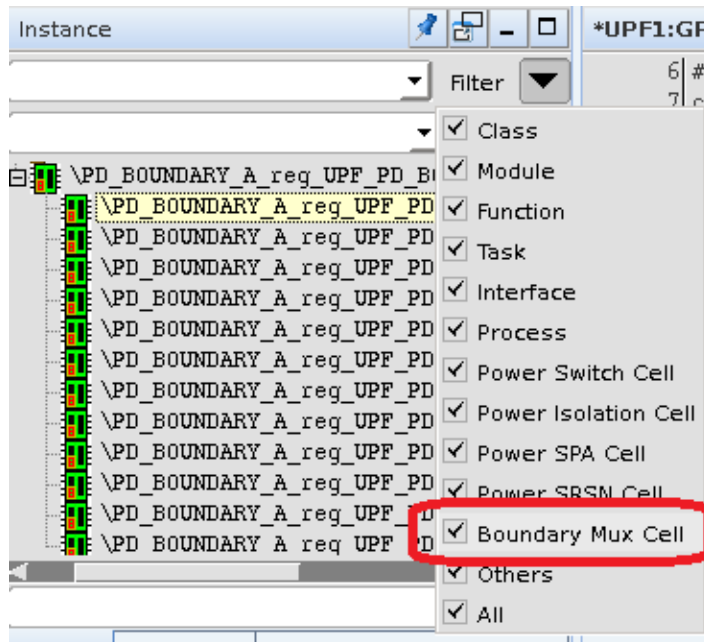
```
PD_BOUNDARY_out[bit_index] (SNPS_UPF_Boundary_Mux)
```

Figure 122 Boundary MUX Nodes



Sorting and filtering applies to Boundary MUX tree nodes also. You can filter the boundary MUX cells as shown in the following figure:

Figure 123 Filtering Boundary MUX Cells



Additionally, the boundary MUX is now integrated into *nTrace* driver/load results. If power domain is modeled as a boundary MUX, then you can see "power domain" as driving statements, related supply set, and instrumented signals as contributors. This helps you to understand the existence of boundary MUX and you can continue to trace through the instrumented signals.

Power Domain Not Modeled as Boundary MUX

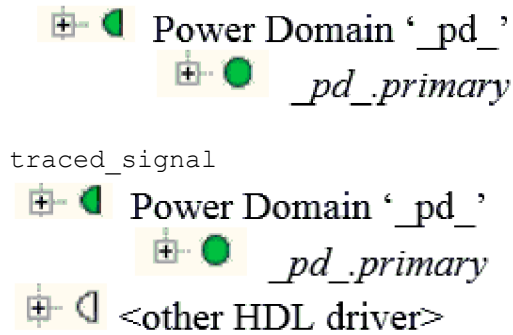
This section illustrates the different trace driver/load behaviors when power domain is not modeled as boundary MUX (for example, VCS NLP).

- If the traced signal is `upf_simstate` (instrumented signal) under HDL scope, Verdi shows the power domain of HDL scope as driver.
- If the traced signal is not driven by SRSN, SPA, or ISO cell and related power domain is *off*, Verdi shows the related power domain as driver. HDL drivers of traced signal should be inactive at the same time.

Example 1

If power domain `_pd_` is on -> off

```
upf_simstate
```



Example 2

If power domain _pd_ is off -> on

upf_simstate



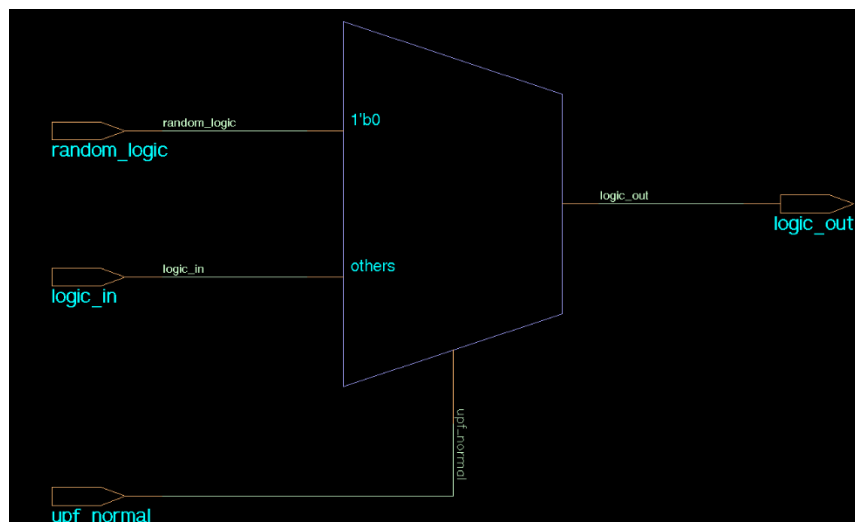
traced_signal



Power Domain Modeled as Boundary MUX

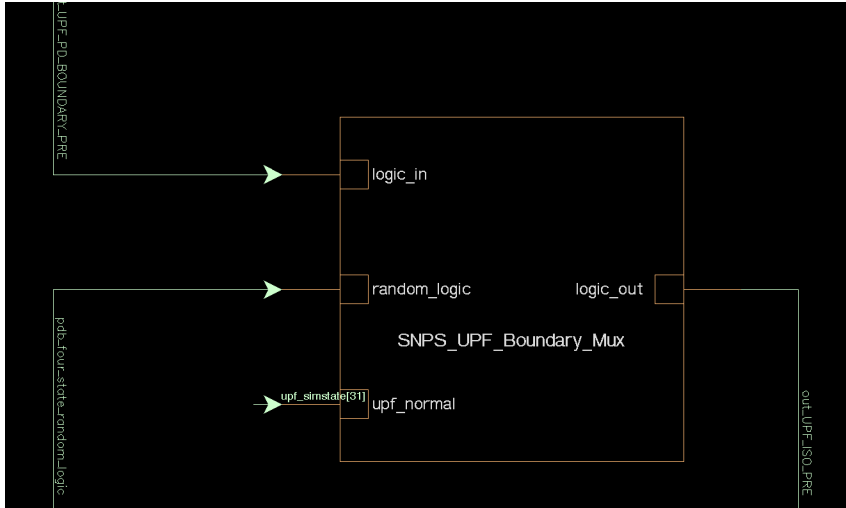
This section illustrates the different trace driver/load behaviors when power domain is modeled as boundary MUX.

Figure 124 Power Domain Modeled as Boundary MUX



The following figure illustrates the instrumented cell and instrumented signal = `traced_signal_UPF_PD_BOUNDARY_PRE`.

Figure 125 Instrumented Cell and Instrumented Signal



Example

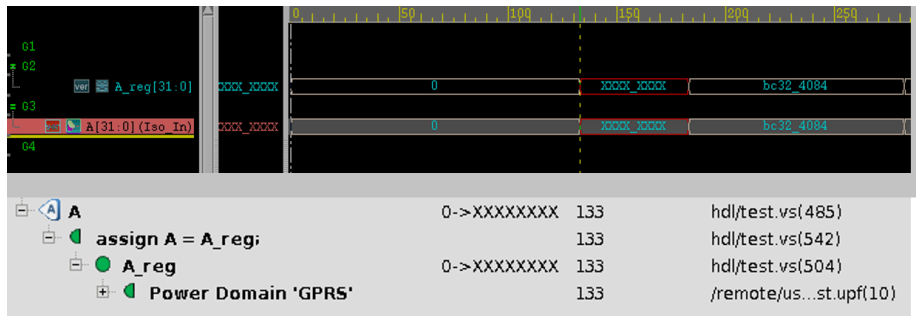
```

traced_signal
├─ Power Domain 'pd'
│   ├── _pd_primary
│   └─ instrumented signal(traced_signal_UPF_PD_BOUNDARY_PRE)
│       └─ <HDL driver statement>
  
```

The above example shows tracing results if the traced signal is the output pin of boundary MUX. For the signals that are not directly connected to boundary MUX, they are still impacted by the state change of power domain.

As shown in the following case, A is connected to `A_reg`, which is the output pin of boundary MUX. Hence, if you trace signal A, 0->X at time 133, then you find that the root-cause is the power domain (GPRS) shut off. On the other hand, if power domain is not modeled as boundary MUX and you still trace signal A, 0->X, then you see the drivers *Power Domain "GPRS"* directly.

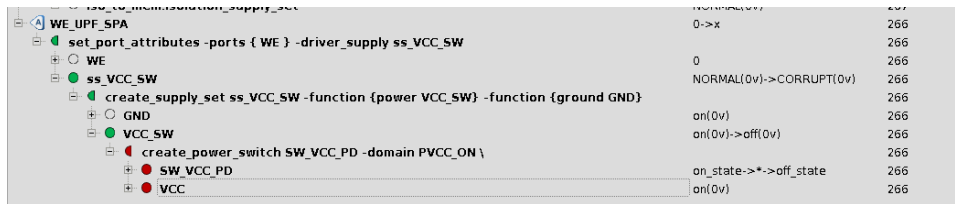
Figure 126 Power Domain "GPRS"



Limitations and Error Handling

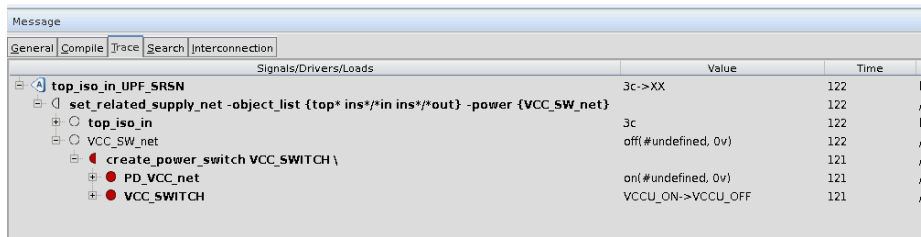
Current trace Driver/Load result is based on zero cycle delay. For example, the figure below shows that the cross-boundary (power -> HDL) part does not have any propagation delay (all results are at time 266). Hence, the correct active drivers/contributors status (the colors) is shown as follows:

Figure 127 Trace Driver/Load Results on the Output of SPA Cell



The following figure shows a counter example, where you can view the value change of VCC_SW_net occurring at time 121. However, the value change time of top_iso_in_UPF_SRSN is at time 122. In this type of cycle-delay situations, the correctness of active drivers/contributors status is not guaranteed. However, the static drivers/contributors results (which is based on netlist) is still correct regardless of the cycle-delay issues.

Figure 128 Trace Driver/Load Results on the Output of SRSN Cell



Known Issues

- Cycle delay issues affect the active drivers/contributors status of ISO, SRSN, and Boundary MUX cells.
- Incorrect netlist issues affect the active drivers/contributors status of Boundary MUX cells.
- Any missing dumped signals or incorrect dumped signals affect the active drivers/contributors status.

Support for Boundary MUX Cells in nWave

nWave supports power shading for Boundary MUX power designs. The shading is based on power zero-delay, that is, once the power condition is enabled, the corresponding shading is drawn.

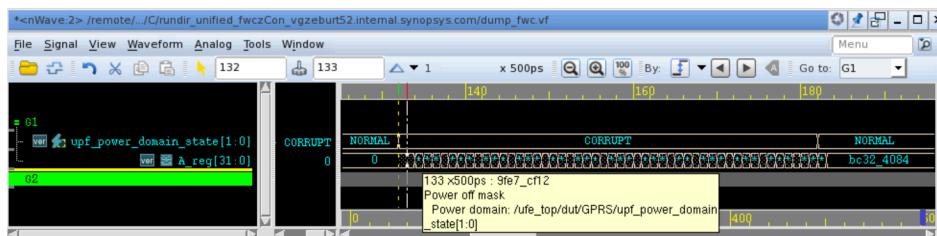
nWave supports the following kinds of power shadings:

- **Power Off:** The power-off range for HDL signals is masked and the *off* range is checked from `upf_power_domain_state` instead of `upf_simstate`.
- **Isolation:** The isolation range for HDL signals is masked according to the applied isolation condition.
- **Retention:** The retention range for HDL signals is masked according to the applied retention condition.
- **Driving Power Off:** The driving power-off ranges for HDL signals is masked and the *off* range is checked from `upf_power_domain_state` instead of `upf_simstate`.
- **Toggle:** The range for HDL signals is masked where the scope is already in a power-off corruption state and the input signals contain value changes. The *off* range is checked from `upf_power_domain_state`, instead of `upf_simstate`.

To check what condition each shading is based on, turn *on* the **Waveform -> View Options -> Waveform Pane -> Enable Tip** preference option (invoked using **Tools -> Preferences**). The tip provides additional information.

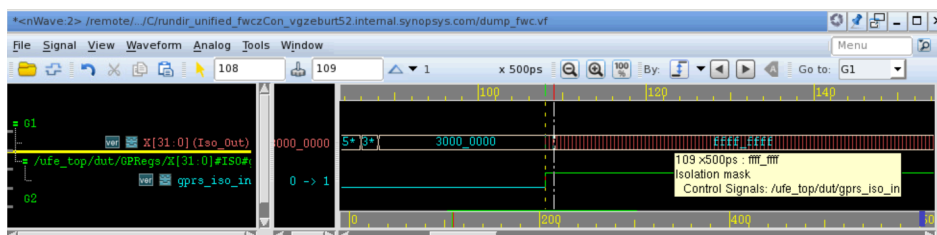
For example, the following figure shows *Power Off* shading where `upf_power_domain_state` is CORRUPT from time 132, hence, waveform draws power off shading immediately, but the HDL signal value is changed from time 133, hence, there is one cycle delay.

Figure 129 Power Off Shading



The following figure shows Isolation shading, where isolation condition is enabled from time 108, hence, waveform draws isolation shading from that time, but the HDL signal value is changed from time 109, hence, there is one cycle delay.

Figure 130 Isolation Shading



Support for Boundary MUX Cells in nSchema

nSchema also supports the symbol boundary instrument cell as *Boundary Cell (BC)* type. The usage of BC type is similar to ISO/SPA/SRSN instrument cell types. The instrument color is same as the power domain color, as illustrated in the figure below.

In the detailed RTL mode in *nSchema* (enabled using the **View -> Detailed RTL menu option**), all boundary cells are created. However, in the non-detailed RTL mode, same boundary cells with different index values are merged to 1 boundary cell.

Figure 131 Boundary Instrument Cell Symbol

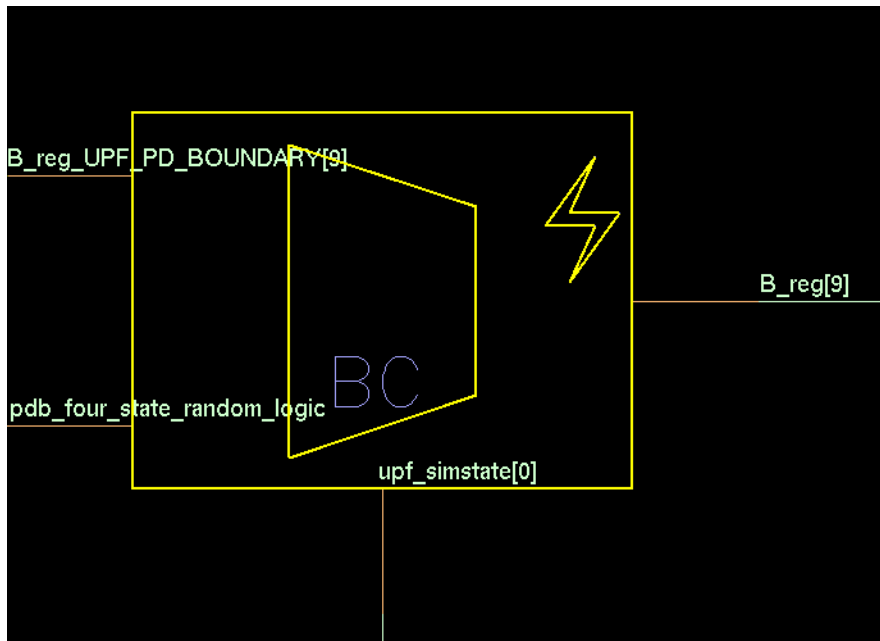
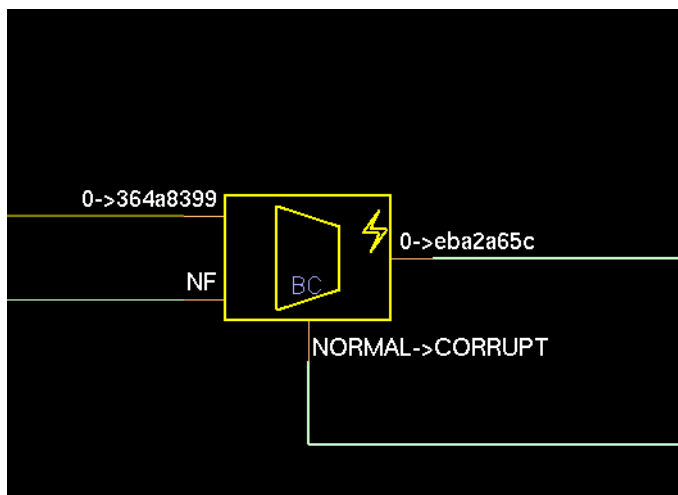


Figure 132 Normal -> Corrupt Boundary Instrument Cell



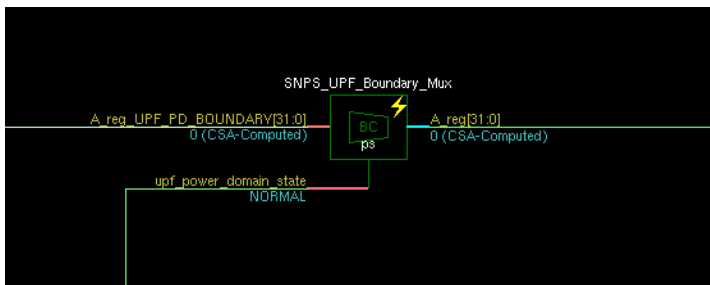
The Drag and Drop (DnD) functionality does not contain `Power_Command`; it is regarded as a usual RTL instance. The DnD functionality between different *nSchema* windows has different behavior, as follows:

- Dragging and Dropping to a full window jumps to a specific scope, whereas, dragging and dropping to a flattened window adds or selects the boundary cell in the flattened window.
- DnD a split boundary cell to *nPowerManage* shows power create domain in the UPF file. A limitation for DnD to *nPowerManage* is that dragging and dropping a merged boundary cell is not supported.
- DnD a split/merged boundary cell to *nWave* adds all the connected signals to *nWave*.

Support for Boundary MUX Cells in Temporal Flow View

Temporal Flow View also supports tracing power boundary MUX cells. The usage of instrument *Boundary Cell (BC)* type is similar to ISO/SPA/SRSN instrument cell types. The instrument cell is dependent on the XML file. The instrument cell color is same as the power domain color; the cell shape is illustrated in the following figure:

Figure 133 Instrumented Boundary Cell



Power Corruption Model in Temporal Flow View

If the value of power corrupted cell is random, its value is obtained from the FSDB file directly. The automatic tracing feature supported by *Temporal Flow View* allows you to trace this value for power net when the power is corrupted. You can use the **Trace This Value** right-click option to trace the corrupted power signal's value instead of using the **Trace Active X** option.

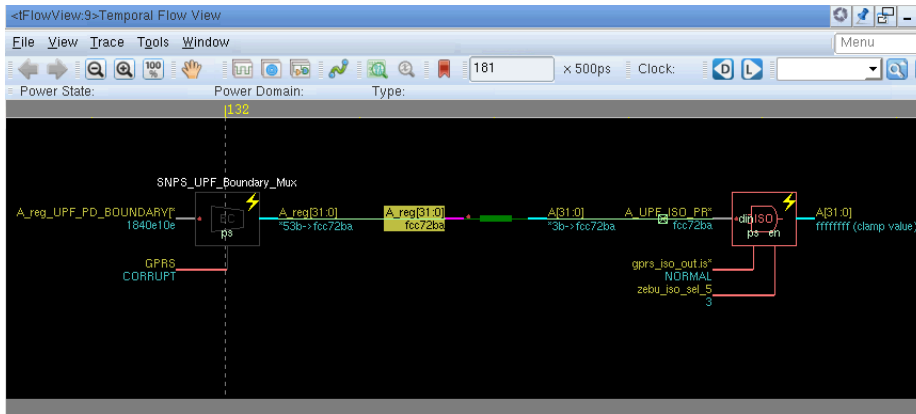
Note:

The **Trace Active X** right-click option is disabled in power-aware *Temporal Flow View* if the signal value is not X.

When tracing the fanout signal for which the power supply set is corrupted, the value is obtained from the FSDB file directly, and trace is performed to the power supply set to detect the root cause of power corruption.

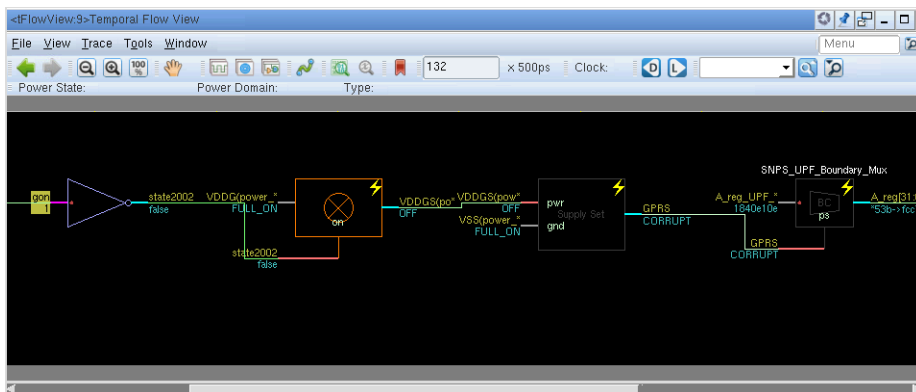
For example, in the following figure, the power of the cell is corrupt.

Figure 134 Corrupted Power Cell



When the **Trace This Value** right-click option is used for the `A_reg` signal, the automatic tracing functionality traces to the corrupted power supply set to debug the root cause of power corruption. In this case, the root cause of power corruption is the power switch which changes from 1 -> 0, as illustrated in the following figure:

Figure 135 Tracing Corrupted Power Supply Set



Note:

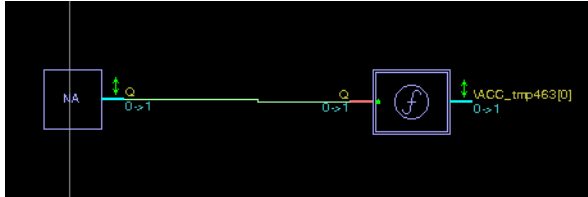
In some scenarios, even though the power model is corrupted, the output may not be X due to which the **Trace Active X** option is not enabled. In such cases, you can use the **Trace This Value** option instead of **Trace Active X**.

Limitations

The following limitations apply to this feature for Temporal Flow View:

- The following UPF types are only supported:
 - Power Port
 - Power Net
 - Power Logic Net
 - i. ack_port of power switch
 - ii. control_port of power switch
 - iii. isolation_signal of isolation
 - iv. save_signal/restore_signal of retention
 - Power Supply
 - Power Switch
 - ISO
 - SRSN
 - SPA
 - Retention (edge trigger only)
- The `connect_logic_net` power logic net and Level Shift UPF types are not supported.
- Single-pin, zero-pin, and level sensitive retention type cells are not supported.

- When tracing is performed in *Temporal Flow View* with partially dumping the FSDB file,
 - If there is no dumping on certain signals, the value is calculated by the register signal. If it cannot be calculated or the signal is a sync signal, such as, register signal, then "NF" is shown in the *Value* pane.
 - When trace reaches a scope that does not contain any dumped signals, the path stops expanding at the boundary and results in the symbol, as illustrated in the following figure:



When the mouse hovers over the symbol, the tooltip message notifies that the trace is stopped because it traced into a scope without any dumped simulation results (which is required for tracing).

- Automatic power-aware tracing feature in *Temporal Flow View* does not support Transition based mode (tracing the power corruption triggering path).

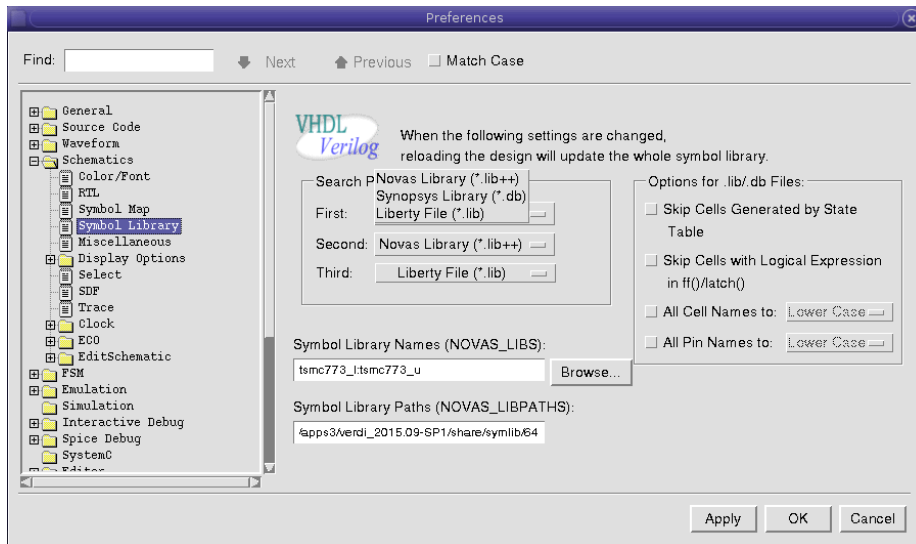
Directly Importing Liberty Files and Displaying Power Cell Information

The Verdi platform supports directly importing Synopsys *Liberty* files. To directly import *Liberty* files and display power cell information, perform the following steps:

1. Specify `NOVAS_LIBS` and `NOVAS_LIBPATHS` environment variables to point to the name and path of your *Liberty* library.
2. Specify the **-powerSymbol** option on the Verdi command line when invoking the Verdi platform.

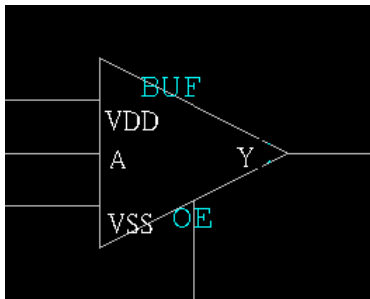
The preferences can be saved on the **Schematics -> Symbol Library** page of the *Preferences* form (invoked with **Tools -> Preferences**). Set the search priority for **Liberty File (*.lib)**, **NOVAS Library (*.lib++)**, and **Synopsys Library (*.db)** in the **First**, **Second** and **Third** fields present in **Search Priority** tab. This is also illustrated in [Figure 136](#).

Figure 136 Liberty File Settings in Preferences Form



After the *Liberty* files with power information are imported, the Verdi platform displays the power cell information. The power and ground pin (*pg_pin*) information can be displayed and the isolation/level shifter/power switch cell can be highlighted in the *nSchema* window. Refer to [Figure 137](#) for an example.

Figure 137 Display *pg_pin* of Power Cells



Debug Support for nwell and pwell Supply Functions

Verdi supports debugging of the nwell and pwell functions of a supply set. When the nwell and pwell functions are referenced in the UPF or explicitly connected to a supply net, they are displayed as child nodes of the supply set in the Hierarchical Power Domain pane, Signal List pane, nTrace, nWave, nSchema, and Temporal Flow View.

The following table shows the icons displayed for nwell and pwell supply functions in the Hierarchical Power Domain pane and Signal List pane:

Icon	Supply Function Name
	nwell
	pwell

The following sections describe this feature in detail:

- [Viewing nwell and pwell Supply Functions in the Power Domain Hierarchy and Signal List Pane](#)
- [Debugging nwell and pwell Supply Functions in nTrace](#)
- [Adding nwell and pwell Supply Functions to nWave](#)
- [Support for nwell and pwell Supply Functions in nSchema](#)
- [Support for nwell and pwell Supply Functions in TFV](#)

Viewing nwell and pwell Supply Functions in the Power Domain Hierarchy and Signal List Pane

When the nwell and pwell supply functions are explicitly connected to a supply net in the design or declared in the UPF, they are displayed as child nodes of the supply set in the power domain hierarchy tree and Signal List pane, as shown in the following figures:

Figure 138 Viewing Explicitly Connected Supply Net as Children Nodes of a Supply Set

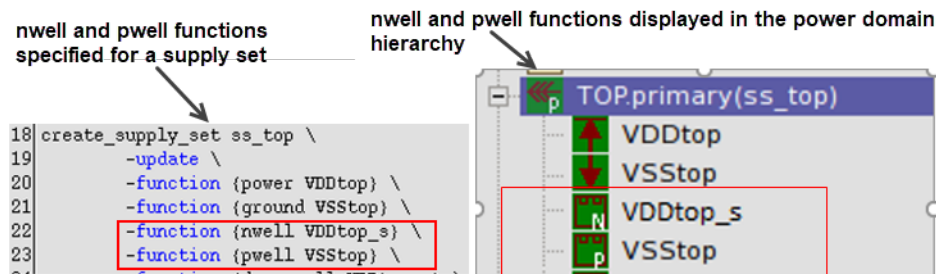


Figure 139 Viewing “supply_set.supply_function”as Children Nodes of a Supply Set

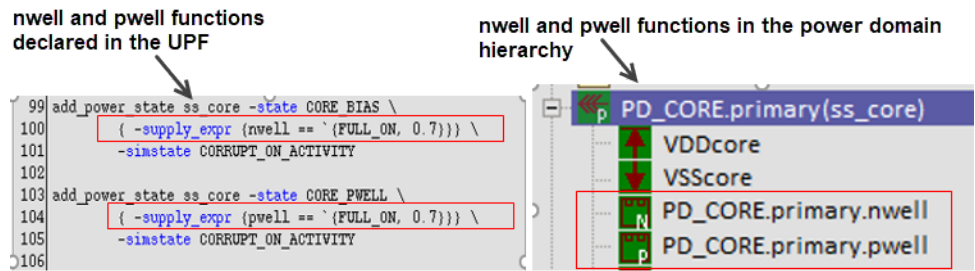


Figure 140 Viewing nwell/pwell Functions (Connected to Supply Set) in the Signal List Pane

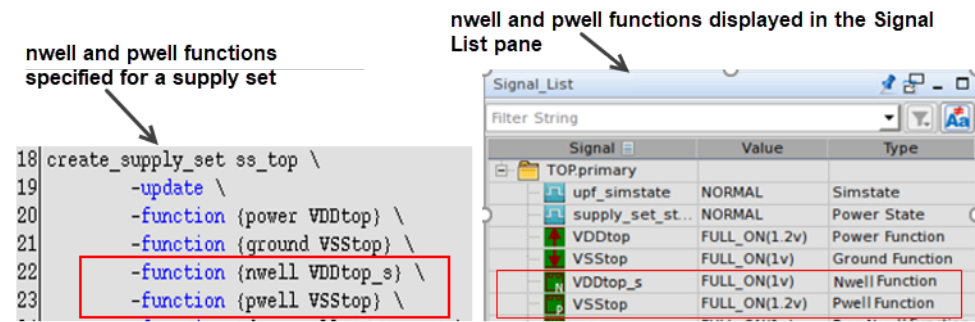
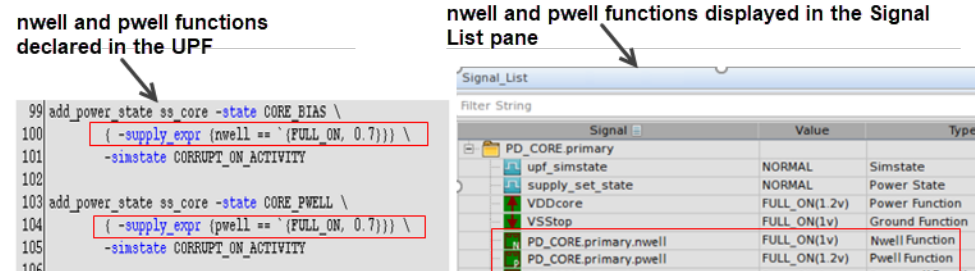


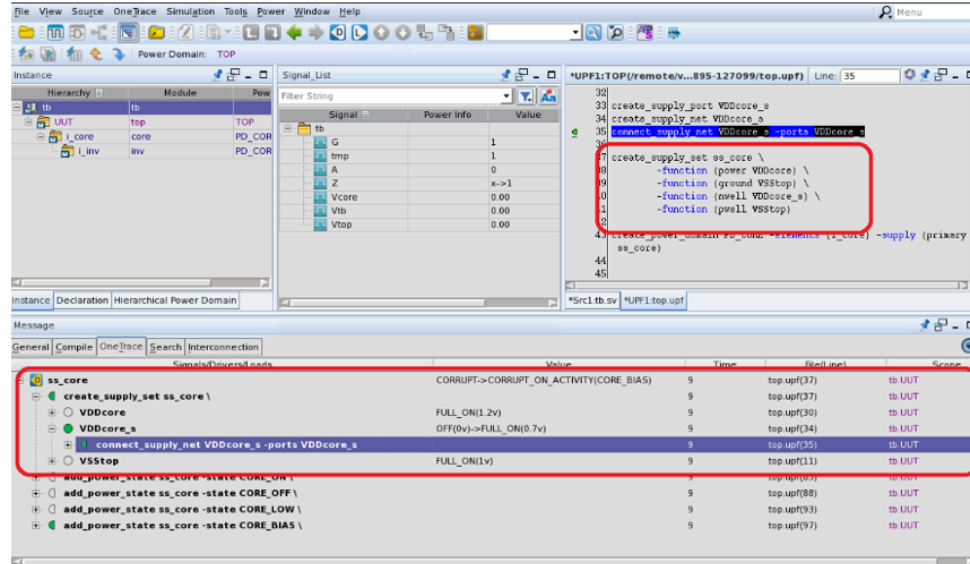
Figure 141 Viewing nwell/pwell Functions (Declared in the UPF) in the Signal List Pane



Debugging nwell and pwell Supply Functions in nTrace

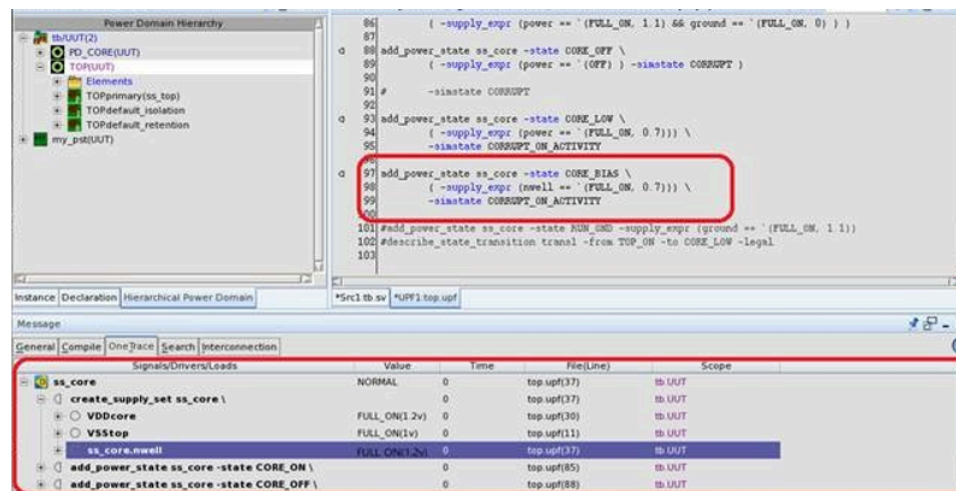
When the nwell and pwell supply functions are explicitly connected to a supply net in the design, they are displayed as contributors of the supply set, for example, as shown in the following figure:

Figure 142 Viewing Explicitly Connected Supply Nets as Contributors of Supply Set



If the nwell and pwell supply functions are referenced in the UPF, Verdi displays the `supply_set.supply_function` signals as contributors, for example, as shown in the following figure. You can expand the “+” icon to perform continue trace.

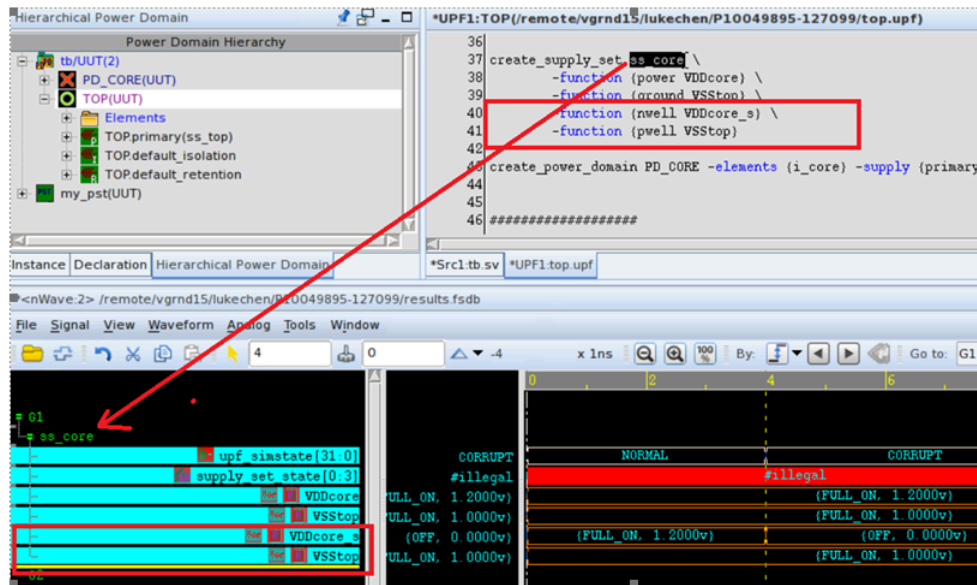
Figure 143 Viewing “supply_set.supply_function” Signals as Contributors of Supply Set



Adding nwell and pwell Supply Functions to nWave

Verdi allows you to drag and drop the Supply Set with nwell and pwell supply functions from Source pane to nWave, as shown in the following figure:

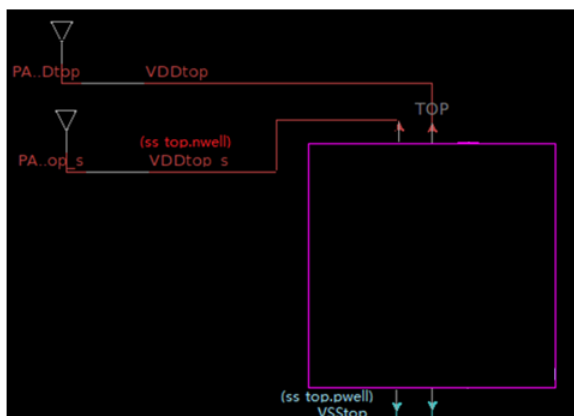
Figure 144 Viewing Supply Functions for Supply Set in nWave



Support for nwell and pwell Supply Functions in nSchema

Verdi displays nwell and pwell supply functions for Supply Set in nSchema as shown in the following figure:

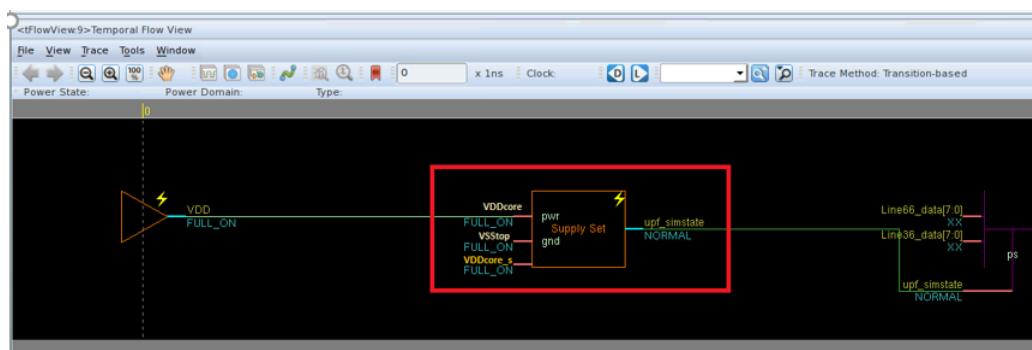
Figure 145 Viewing Supply Functions for Supply Set in nSchema



Support for nwell and pwell Supply Functions in TFV

Verdi creates port for nwell, pwell supply functions and connects corresponding supply set to the port in TFV, as shown in the following figure:

Figure 146 Viewing Supply Functions for Supply Set in TFV



Note:

If a supply net drives multiple ports, TFV is displayed only for one of them.

Displaying Power State Group in Power State Table Frame

Overview

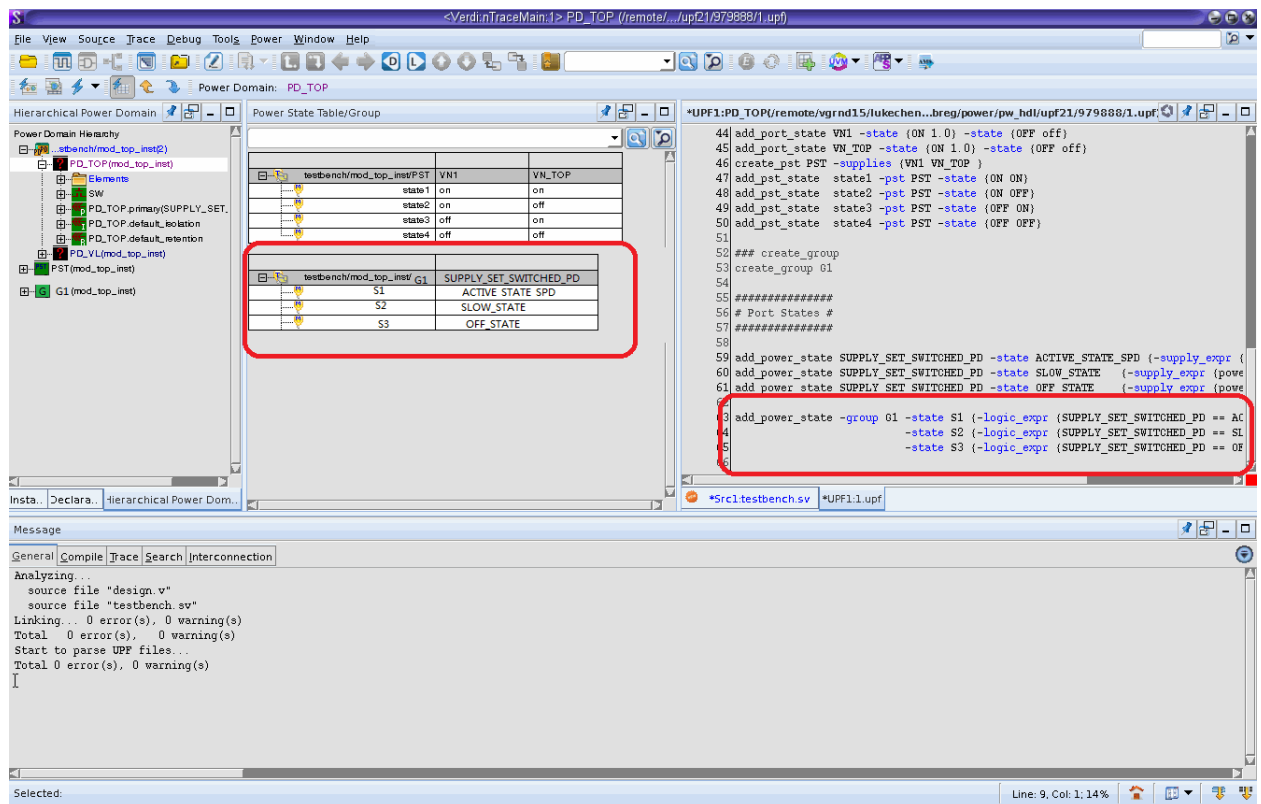
The following enhancements are provided with Verdi Power-Aware Debug:

- The `add_power_state` command is enhanced to define a *Power State Table*. The `create_pst` and `add_pst_state` commands are no longer required to create a *Power State Table*.
- The power states displayed in the *Power State Table* are composed hierarchically to avoid exponential explosion of the size of the Power State Table.
- The *Power State Table* frame and the *Power Domain List* frame displays the related power states in the complete design under a single user-defined group as *Power State Group List*.
- The *Power State Group List* supports dynamic debug functionalities like active annotation, drag and drop, and so on.

Use Model

To view the related power states in a design, create a *Power State Group List* for the entire design. The *Power State Table* frame displays the *Power State Group List* as illustrated in Figure 147.

Figure 147 Power State Group Displayed in the PST Frame



Use the `create_group` command to create a group of related power states and define a group name that can be used in the `add_power_state` command.

Syntax:

```
create_group group_name
```

Use the `add_power_state` command to add power states for the functional nets (*power*, *ground* and so on) and create power state tables and define them hierarchically.

Syntax:

```
add_power_state [-supply|-domain|-group] object_name [-update]
-state {[parent_state_name] state_name [-legal|illegal]}
[-simstate simstate]
```

Chapter 4: Power-Aware Debug Functions

Displaying Power State Group in Power State Table Frame

```
[-supply_expr {Boolean_expression}]
[-logic_expr {Boolean_expression}]
}]*
[-complete]
```

The following is an example to create a group *GROUP1* with *RUN* and *SLEEP* states for *SS1* and *SS2* signals:

```
#Creating power state ON for SS1

add_power_state -supply SS1 \
-state ON {-supply_expr {(power == {FULL_ON 0.8}) && (ground == {FULL_ON 0})}}

#Creating power states ON and OFF for SS2

add_power_state -supply SS2 \
-state ON {-supply_expr {(power == {FULL_ON 0.8}) && (ground == {FULL_ON 0})}}

add_power_state -supply SS2 \
-state OFF {-supply_expr {(power == {OFF}) && (ground == {FULL_ON 0})}}

#Creating group for power states
create_group GROUP1

add_power_state -group GROUP1
-state RUN {-logic_expr {SS1 == ON && SS2 == ON}}
-state SLEEP {-logic_expr {SS1 == ON && SS2 == ON}}
```

The following table shows the power states for *GROUP1* group created for *SS1* and *SS2* signals:

GROUP1	SS1	SS2
RUN	ON	ON
SLEEP	ON	OFF

Supported Features

For each power state group, the member list is displayed in the *Power State Table* frame. The following features are supported for *Power State Group List*:

- Dynamic debug functions (like active annotation, drag-and-drop power state group to waveform, or to debug value) are supported for power state group objects and the member object list.
- Drag and drop action is supported for power state group object and its member object list.
- Create *Power State Group List* by UPF scope, similar to creating *Power State Table List* in *Power Domain* tree is supported. Drag-and-drop action for power state group in power domain tree is also supported.
- Double clicking the *Power State Group* node highlights the related power state group object in *Power State Table* frame and syncs the UPF source code to its declaration.
- Double clicking the group object syncs the source code to its declaration in the UPF source code, and highlights the group object node in the *Power Domain* tree.
- Double clicking the state object syncs the source code to the declaration of its related group in the UPF source code.
- Double clicking the member of *Power State Group* syncs the source code to its reference in related *Power State Group*.
- In *Power State Table* frame, as *Power State Group* can be used as a member of other *Power State Group*, so, for each sub power state group/table object, right-click menu is supported as **Show Definition** to sync source codes to its declaration and highlight the related power state group/table item of definition in the *Power State Table* frame.
- Drag and drop action for *Power State Group* is supported in the *UPF source code* view and *nWave* frame.

Viewing Supply Net Relationship

Verdi allows you to view the relationship between two supply nets/ports. You can use the following frames to view the relationship between any two supply nets/ports:

- [Supply Net Relationship Frame](#)

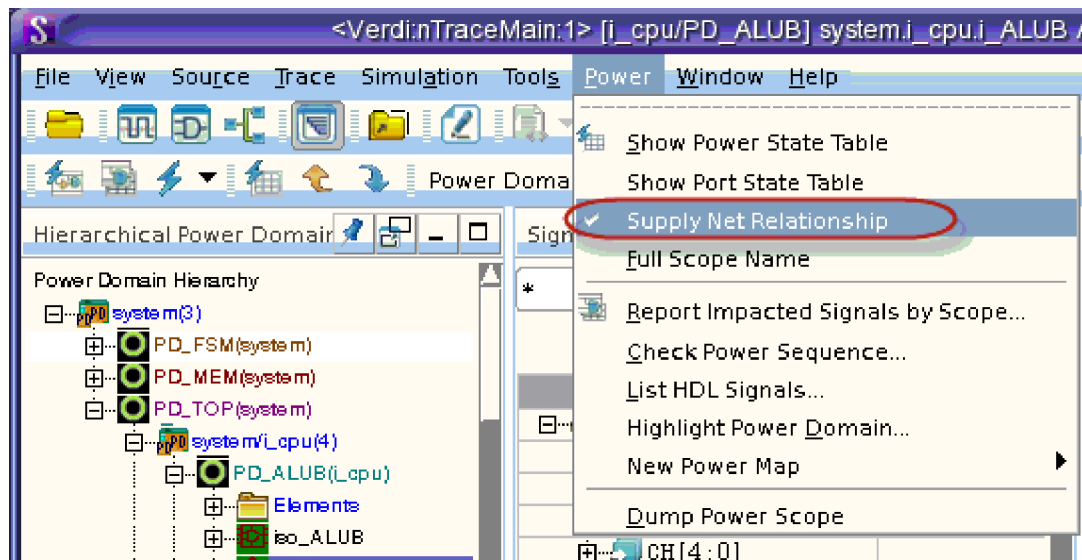
Supply Net Relationship Frame

The *Supply Net Relationship* frame displays the following:

- Relationship between two supply nets/ports
- Merged states of two supply nets/ports

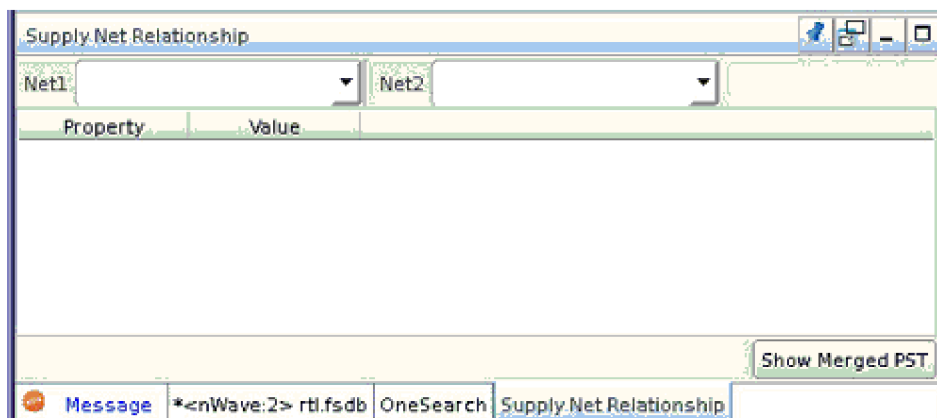
The *Supply Net Relationship* frame is displayed when the **Power -> Supply Net Relationship** command is invoked from the *main* window as shown in the following figure:

Figure 148 Supply Net Relationship Option



The *Supply Net Relationship* frame is docked to the same frame location as the *Message* frame as a new tab (see Figure 148). The *Supply Net Relationship* frame can become a standalone window by clicking the **Undock** icon on the toolbar.

Figure 149 Supply Net Relationship Frame



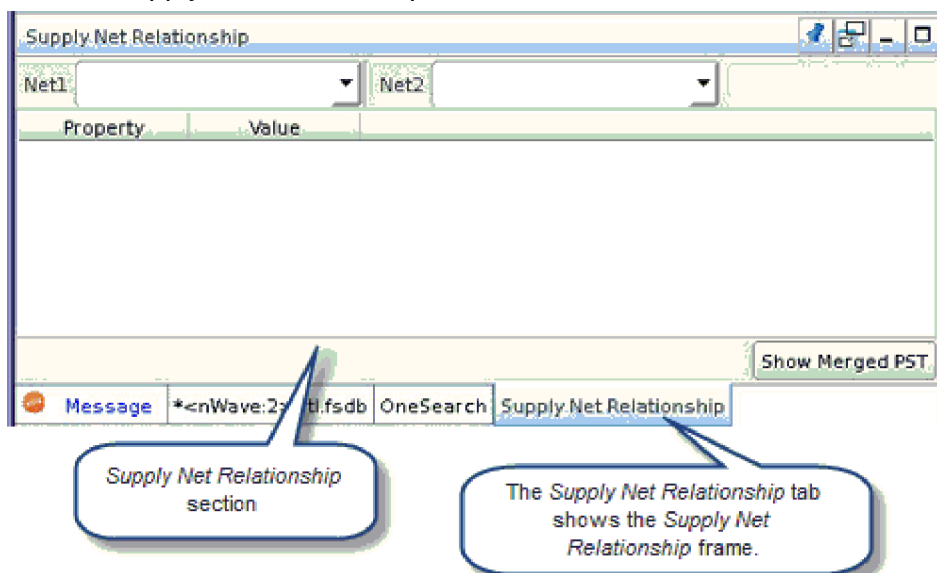
The *Supply Net Relationship* frame contains the following two sections:

- Supply Net Relationship Section
- Merged Power State Table Section

Supply Net Relationship Section

The *Supply Net Relationship* section shows the relationship between two supply nets/ ports. This section is displayed by default in the *Supply Net Relationship* frame. [Figure 150](#) shows the *Supply Net Relationship* section.

Figure 150 Supply Net Relationship Section



The **Net1** and **Net2** text fields allow you to specify supply net/port. You can do either of the following to specify supply net/port in the **Net1** and **Net2** text fields:

- Drag-and-drop a supply net/port from the *Power Domain List* frame, *Power Source Code* frame, or *Signal List* frame to the **Net1** or **Net2** text fields.
- Type the full name of the supply net/port in the **Net1** and **Net2** text fields.

The relationship information is displayed in the **Property** and **Value** columns. The following table shows the types of properties displayed in the **Property** column and the relationship values supported by each property.

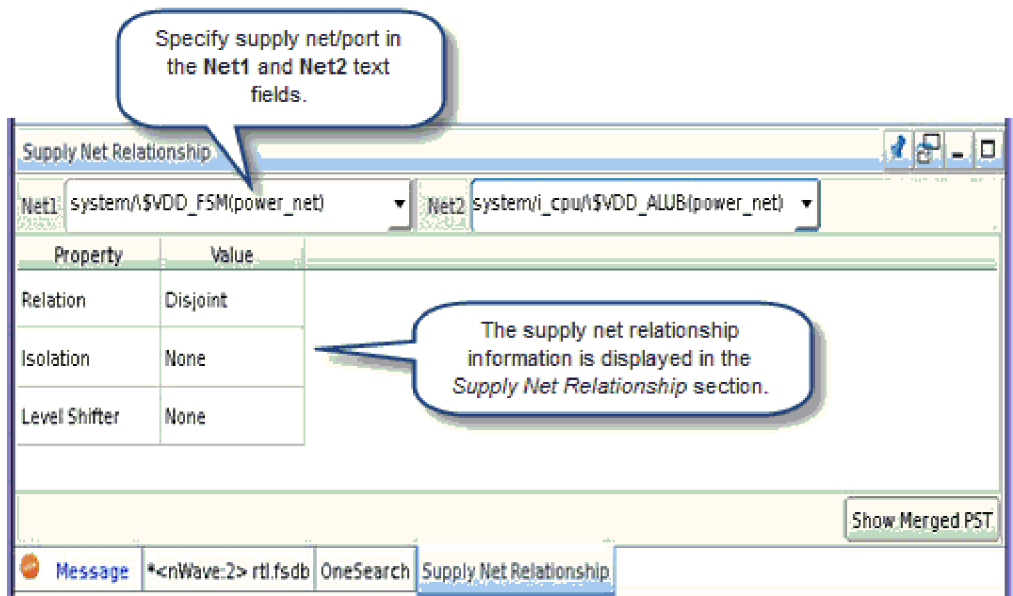
Property	Value	Description
Hierarchical	Common	Both nets/ports are connected
	Disjoint	Nets/Ports are not connected to each other
	Parent-Child	The second net/port is connected to the switched output of the first net/port
	Child-Parent	The first net/port is connected to switched output of the second net/port
Isolation	None	No requirement
	On-to-Off	The first net/port is ON and the second net/port is OFF in the PST
	Off-to-On	The first net/port is OFF and the second net/port is ON in the PST
	Both	A net/port is ON while the other net/port is OFF in the PST
Level-Shifter	None	No requirement
	High-to-Low	First net/port is at a higher voltage than the second net/port in the PST
	Low-to-High	First net/port is at a lower voltage than the second net/port in the PST
	Both	A net/port is at a relatively higher voltage than the other net/port in the PST

To view the supply net relationship, perform the following steps:

1. Specify supply net/port in the **Net1** and **Net2** text fields.
2. Press **Enter**.

Verdi displays the supply net relationship information in the *Supply Net Relationship* section, as shown in [Figure 151](#).

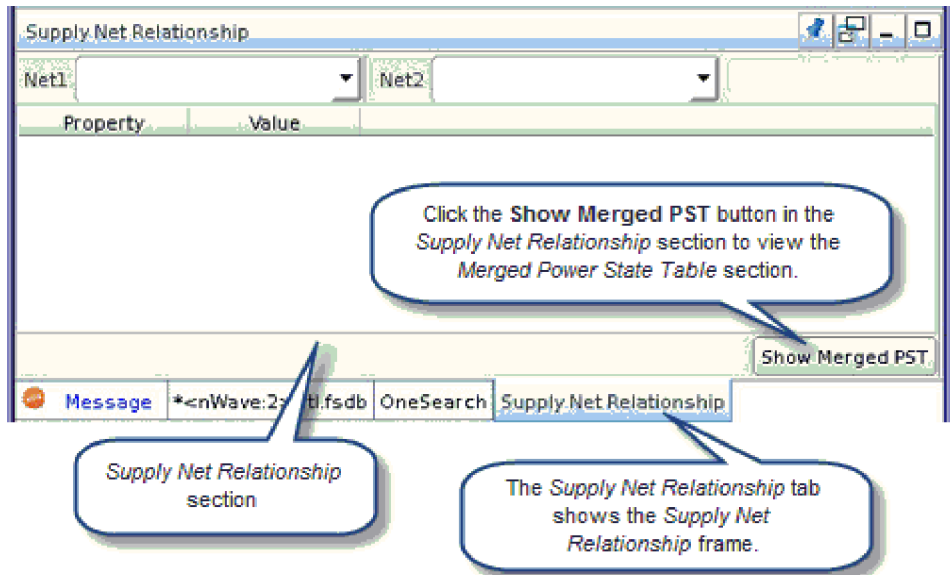
Figure 151 Viewing Supply Net Relationship Information



Merged Power State Table Section

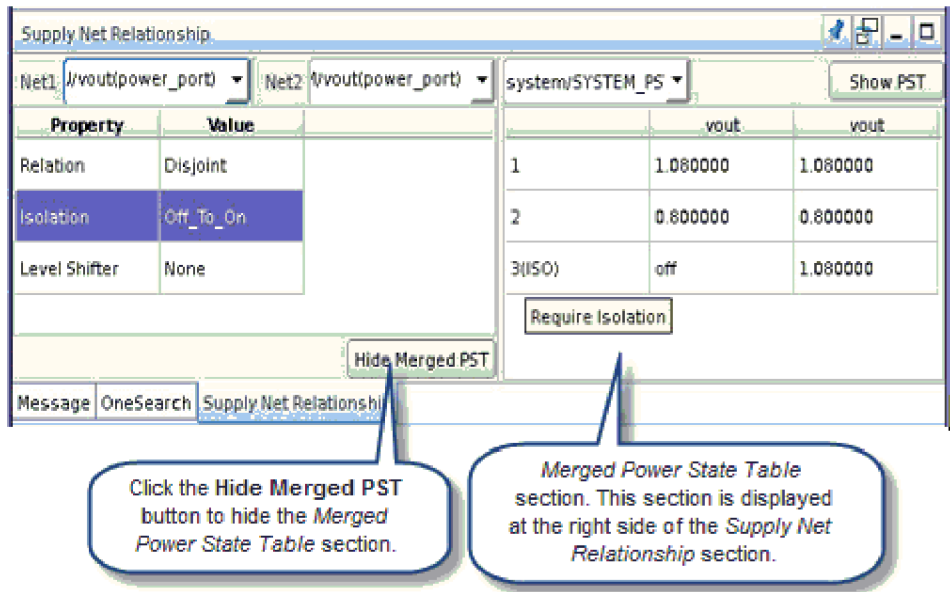
The *Merged Power State Table* section shows the merged states and supply values of the two supply nets/ports selected in the *Supply Net Relationship* section. This section is not displayed by default, you must click the **Show Merged PST** button in the *Supply Net Relationship* section, as shown in [Figure 152](#), to view the *Merged Power State Table* section.

Figure 152 Viewing Merged Power State Table Section



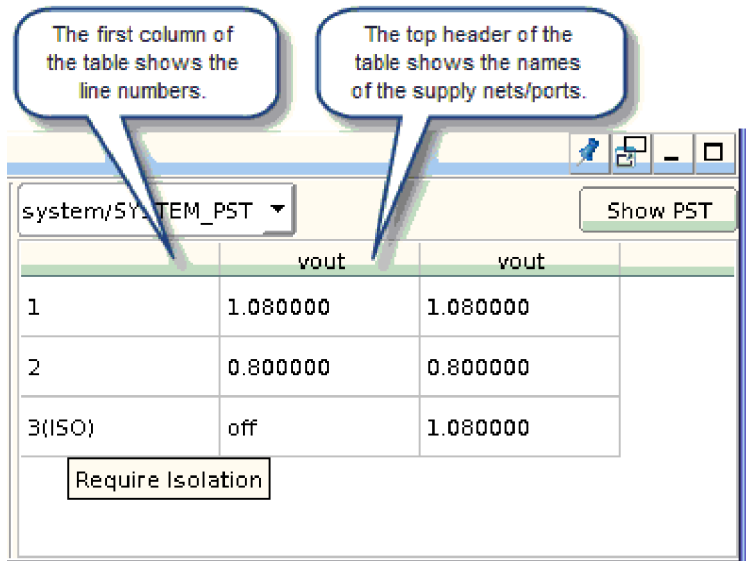
The *Merged Power State Table* section is displayed at the right side of the *Supply Net Relationship* section. The following figure shows the combined view of the *Supply Net Relationship* section and the *Merged Power State Table* section. You can click the **Hide Merged PST** button in the *Supply Net Relationship* section to hide the *Merged Power State Table* section.

Figure 153 Merged Power State Table Section



The *Merged Power State Table* section shows the merged states and supply values in the form of a table. The top header of the table shows the names of the supply nets/ports selected in the *Supply Net Relationship* section. The first column of the table shows the line numbers as shown in [Figure 154](#).

Figure 154 Columns in the Merged Power State Table



If the names of a supply net and supply port are identical, the supply port name is followed by “(Port)”.

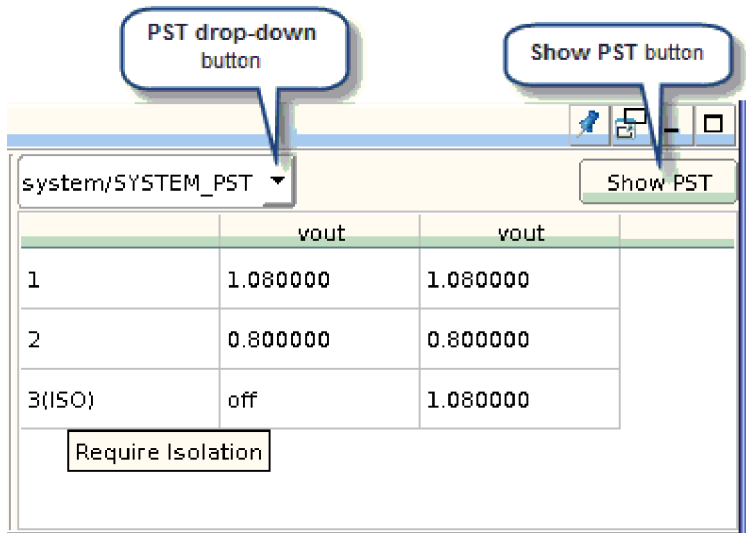
If the merged state requires isolation, the line number of that state is followed by “(ISO)”. The tooltip “Require Isolation” is displayed, as shown in [Figure 154](#), if you hover the mouse pointer on the line number.

Similarly, if a merged state requires level shift, the line number of that state is followed by “(LVS)”. The tooltip “Require Level Shift” is displayed if you hover the mouse pointer on the line number.

If there is any change in the data specified in the *Supply Net Relationship* section, the data is updated in the *Merged Power State Table* section to synchronize with the changes in the *Supply Net Relationship* section.

The **PST drop-down** button located above the top header of the table, as shown in [Figure 155](#), lists all the PSTs in your design which use at least one of the supply nets/ports.

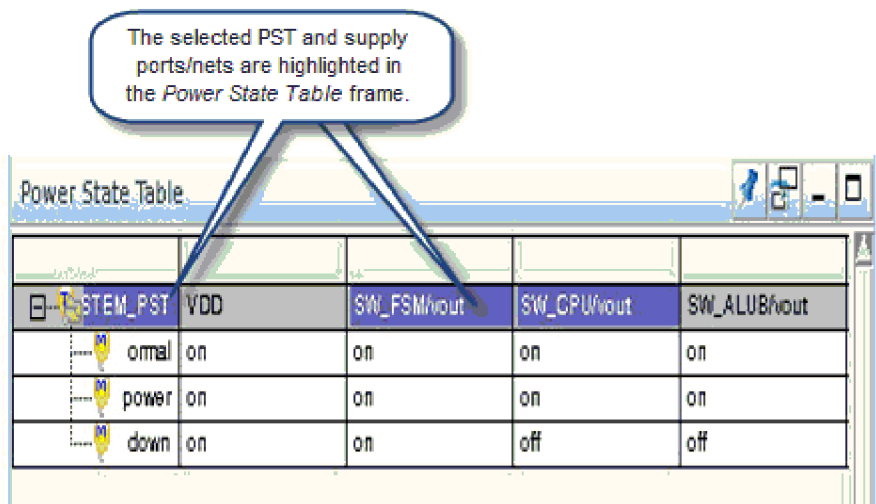
Figure 155 PST drop-down Button and Show PST Button



You can select a PST from the PST drop-down and click the **Show PST** button in the *Merged Power State Table* section to view the power state table of the selected PST in the *Power State Table* frame, as shown in [Figure 156](#).

The selected PST and the headers of the supply nets/ports in the *Merged Power State Table* section are highlighted in the *Power State Table* frame with background color.

Figure 156 Power State Table Frame



Power State Table Frame

You can access the Power State Table frame using the following methods:

- From the *main* window using the **Power -> Show Power State Table** command
- Using the **Show PST** button in the *Merged Power State Table* section.

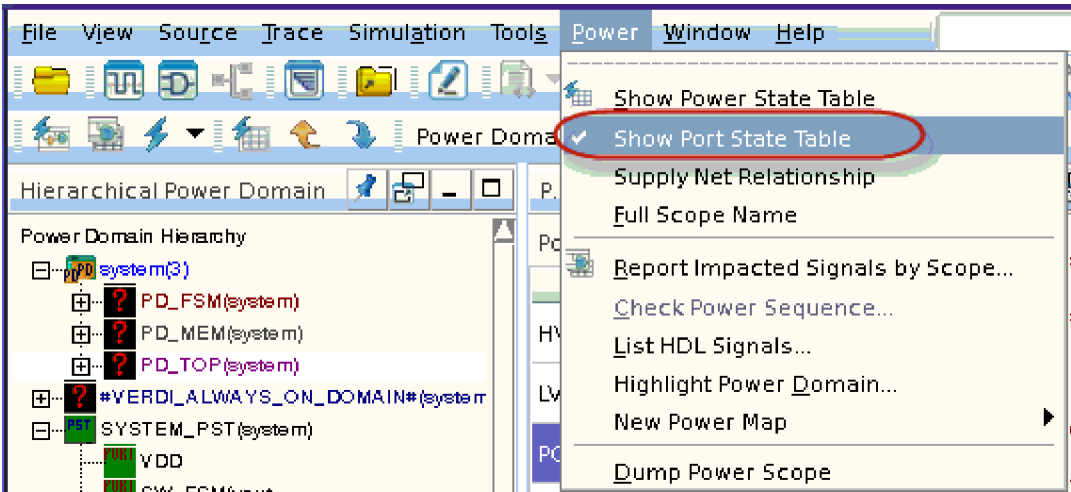
The Power State Table shows only the PST item selected in the **PST drop-down** button. The following table describes the differences between the above two accessing methods:

	From the Main Window	Using the "Show PST" Button
Item Location	Shows the first PST at the first time.	Shows only the selected PST in the PST drop-down button.
Item Highlight	Item is not highlighted in the PST.	Selected PST header is highlighted with background color.
Supplies Highlight	Supplies are not highlighted in the PST.	Headers of the selected supply nets/ports are highlighted with background color.

Viewing the State Names and Supply Values of a Supply Net

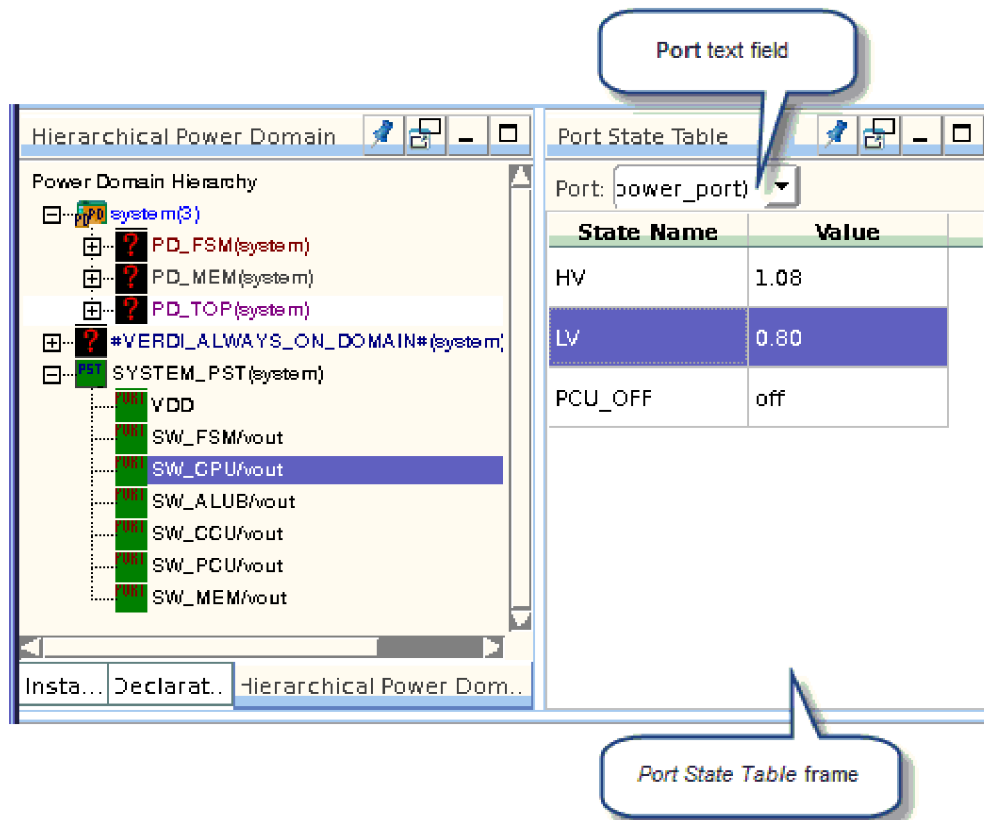
The *Port State Table* frame shows the state names and supply values of a supply net/port. The *Port State Table* frame is displayed when the **Power -> Show Port State Table** command is invoked from the *main* window, as shown in [Figure 157](#).

Figure 157 Show Port State Table Option



The *Port State Table* frame is docked to the same area as the *signal list* frame as a new tab (see [Figure 158](#)).

Figure 158 Port State Table Frame



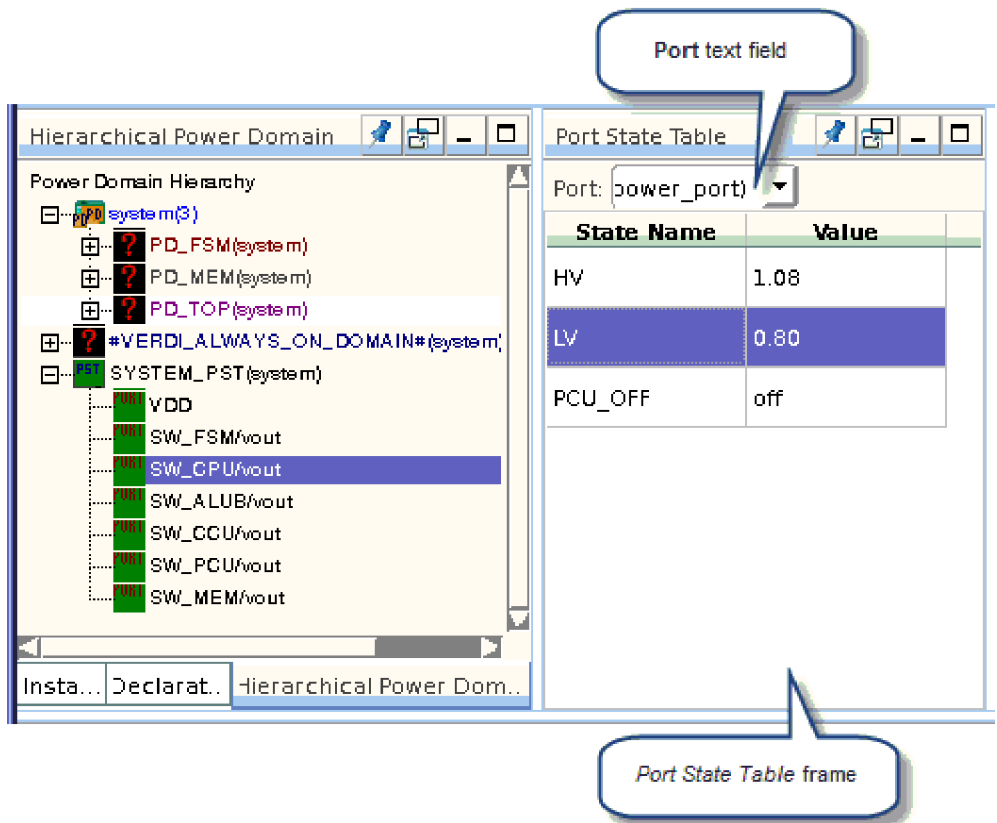
The **Port** text field allows you to specify supply net/port. You can do either of the following to specify supply net/port in the **Port** text field:

- Drag-and-drop a supply net/port from the *Power Domain List* frame or *Power Source Code* frame to the **Port** text field.
- Type the full name of the supply net/port in the **Port** text field.

You can click the drop-down button of the **Port** text field to view the previously mentioned supply nets/ports.

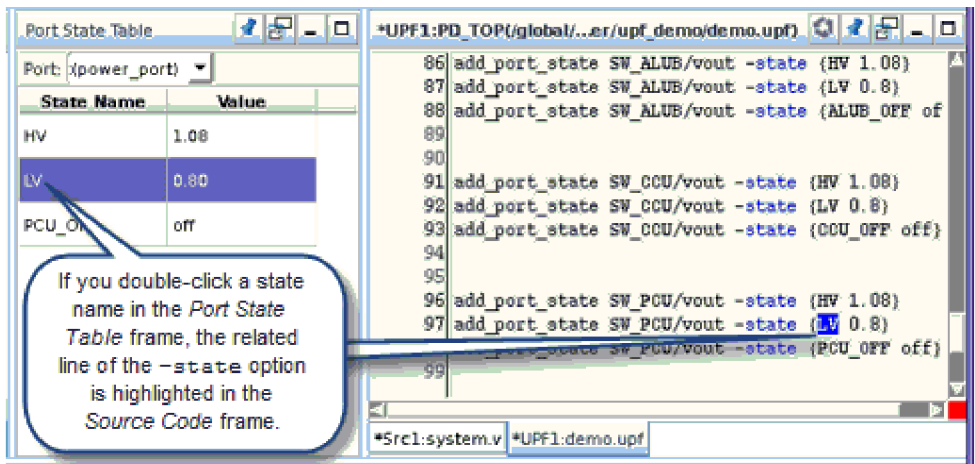
The state names and the values of the specified supply net/port are displayed in the **State Name** and **Value** columns as shown in [Figure 159](#).

Figure 159 State Name and Value Columns of the Port State Table Frame



If you double-click a state name (for example, LV) in the *Port State Table* frame, the related line of the `-state` option is highlighted in the *Source Code* frame, as shown in Figure 160.

Figure 160 Debugging State Names of a Supply Net/Port



Dumping Low Power Debug DB XML File During Compilation In UFE Flow

Verdi allows you to debug low power objects instrumented by VCS NLP. You can use the low power debug DB, an XML format file, dumped by VCS NLP to handle all instrumentation scenarios.

You can use the `-kdb` option at compile time to dump low power debug DB file in the UFE flow.

Following are the benefits of this feature:

- Allows you to dump low power instrument scenarios/objects with specific XML format for every partition and reuse same information from same partition.
- Reduces the memory usage, thereby improving the compile-time performance.

Use Model

The following is the syntax to dump DB XML file for single compile:

```
% vcs <design file> -upf <upf file> -kdb -debug_access+all <compile options>
```

The following is the syntax to dump DB XML file for partition compile:

```
% vcs <design file> -upf <upf file> -partcomp -kdb -debug_access+all <compile options>
```

Example:

```
% vlogan dut.v testbench.v design.v -sverilog -kdb  
% vcs topcfg -q -upf top.upf -partcomp -kdb -lca -debug_access+all  
-power=dumplpconnect
```

5

Demo

The following demo illustrates how to import a UPF power file into the Verdi platform and to use power-related panes to browse the power intent.

1. Copy the `upf_demo` folder located at the following path to your local directory:

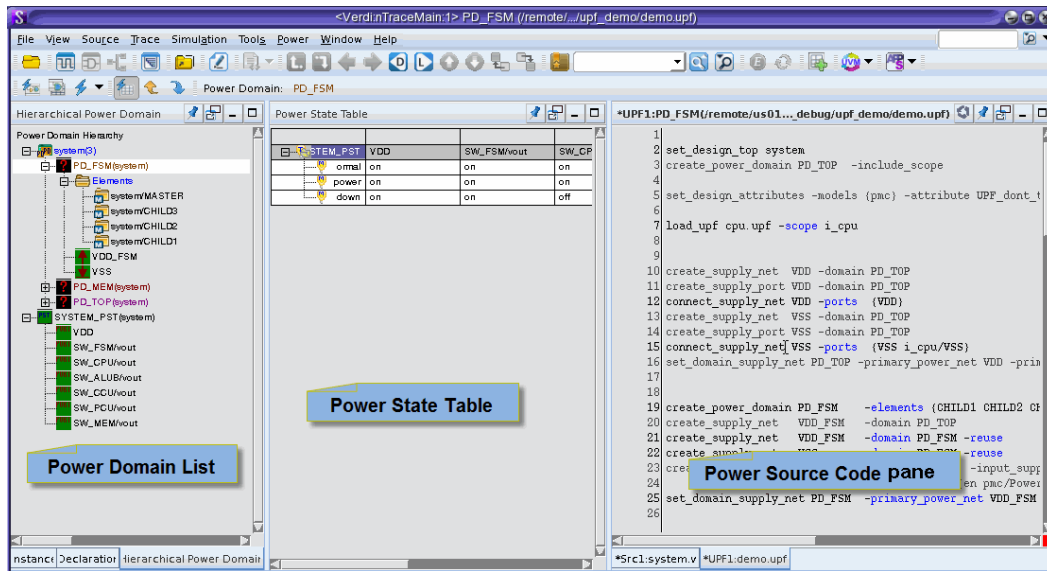
```
$VERDI_HOME/demo/power/upf_demo
```

2. Load the design and the FSDB dump file into the Verdi platform using the following command:

```
verdi -f run.f -ssf rtl.fsdb &
```

3. Load the UPF power file into the Verdi platform by invoking the **File -> Import CPF/UPF Files** option to open the **Import CPF/UPF Files** form. Select *UPF1.0* in the **Language** field, double-click the `demo.upf` file, and click the **OK** button. After loading is complete, power-related panes are opened and docked in the main window.
4. The *Power Domain List* pane shows all power domains with every domain marked with a different color. All power definitions are shown under the power domain hierarchically. The *Power State/Mode Table* pane is added to the middle pane. A power tab is added to the source code pane. When you double-click on different items in the left pane, the content in the power tab changes accordingly.
5. Double-click the `PD_FSM` node in the *Power Domain List* pane. Browse the components of the `PD_FSM` power domain.

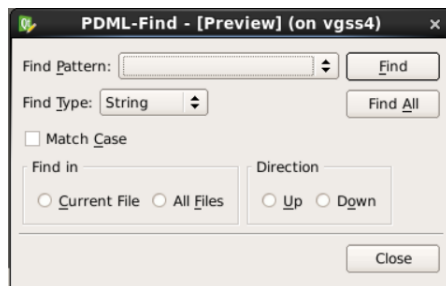
The Verdi platform with the Power-Aware Debug support can automatically extract power state tables defined in the UPF and show the states in a table format.



1. In the *Power State Table* pane, double-click **VDD** and the source code jumps to **VDD** in the *Power Source Code* pane.

2. Search capability for Power-Aware Debug

a. In the main window, invoke the **Source -> Find String** option. The *PDML-Find* form is displayed.



b. Select **String** in the **Find Type** field, and type in *isolation*. Click the **Find All** button.

c. Search results are listed in the *Search* tab of the *Message* pane. Double-click any line to jump to the corresponding source code in the *source code* pane.

3. Trace driver/load of a signal

a. In the *Power Domain Hierarchy* pane, double-click the **PD_MEM** node. The cursor jumps to the respective line in the source code in the *Power Source Code* pane.

b. In the *Power Source Code* pane, right-click the **VDD_MEM** signal and select the **Trace Power Driver** option. Power drivers are listed in the *Trace* tab of the *Message* pane.

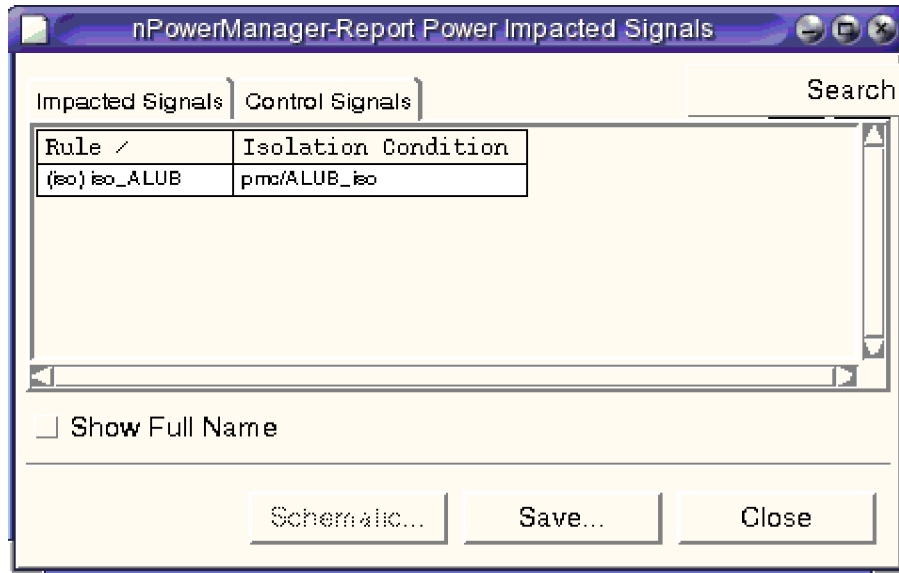
4. Report Impacted Signals by invoking the **Power -> Report Impacted Signals by Scope** option to open the *Scope Lists* form. Use the **Add** button to add the scopes. The Report Power Impacted Signals form opens up, as illustrated in the following figure:

Rule /	Signal	From Instance	From Domain	To Instance	To Domain	Value
(non_cover) iso	systemmEn_A	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_AB	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_AC	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_AD	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_B	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_BC	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_BD	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_C	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_CD	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmEn_D	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmFirstDataOutRdy	systemmMASTER	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmMux1_Se[1:0]	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) iso	systemmMux2_Se[1:0]	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_A	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_AB	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_AC	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_AD	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_B	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_BC	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_BD	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_C	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_CD	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmEn_D	systemmCHILD1	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmFirstDataOutRdy	systemmMASTER	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmMux1_Se[1:0]	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmMux2_Se[1:0]	systemmCHILD2	PD_FSM	system	PD_TOP	NF
(non_cover) lvs	systemmMux3_Se[1:0]	systemmCHILD3	PD_FSM	system	PD_TOP	NF

☐ Show Full Name ☐ Cell Information

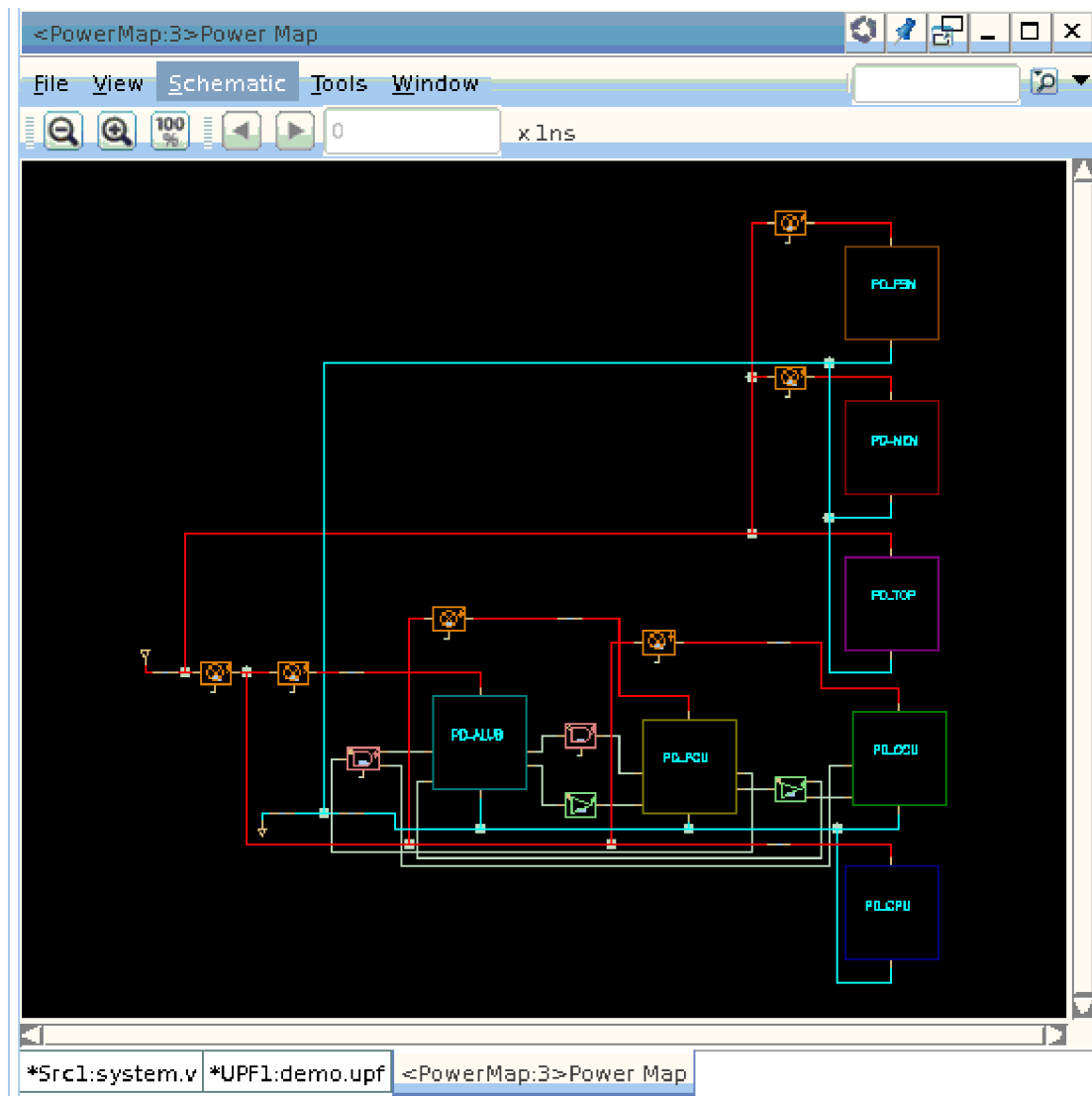
Schematic... Save... Close

- a. Select the **Schematic** button to open an *nSchema* pane displaying the connection of the selected signal.
- b. Select the **Control Signals** tab to browse the control signals of retention and isolation rules.



You can bring up a schematic view of the power design in the *Power Map* pane using the following steps:

1. From the main menu, select **Power -> New Power Map -> Full Power Map** to invoke the *Power Map* pane.
2. Observe that all power domains are shown as rectangles. Drag and drop a domain to the *Power Source Code* pane to browse its definition.
3. Observe that the power switch, isolation and retention rules are presented as cells connecting to the related power domains. Drag and drop an isolation rule to the *Power Source Code* pane to browse its definition.



4. Select the red dotted line between the domains. The red dotted line represents a non-covered isolation net.
5. To view the non-covered isolation net signals, select **Schematic -> Report Selected Impacted Signals**. These signals are suggested by the Verdi platform to be guarded with an isolation rule for protection.

Go to the *nTrace* window and perform the following steps to practice visualizing power information in the window:

1. In the *Design Browser* pane, the instances have different colors. These colors are assigned based on the power domain specified in the UPF. You can change the color by performing the following steps:
 - a. Invoke the **Power -> Highlight Power Domain** option to display the *Highlight Power Domain* form.
 - b. Select `PD_TOP` and change it to a different color, and then press the **OK** button.

A tip window is displayed for showing power information. This helps you to quickly understand the power intent of any module or signal.

2. Put the mouse cursor on `i_ALUB` (under `system/i_cpu`) in the *design hierarchy* pane to view the power domain and state condition.
3. Double-click `i_ALUB` to view the source code which has retention annotation on some of the signals.
4. Double-click `i_CCU` (under `system/i_cpu`) to view more retention annotation on the signals.
5. Put the mouse cursor on one of the retention signals to view the tip with save/restore, and shut-off condition.
6. Select `i_pram` (under `system`) to view level shifter and isolation annotation.
7. Put the mouse cursor on one of the signals to view the tip window with isolation and level-shifter rules.

The power content can be cross probed between panes by performing the following steps:

8. Select `i_pram` in the *Design Browser* pane and drag-and-drop to the *Power Source Code* pane.
9. Go to the *Power Source Code* pane to view the source code of `i_pram` power intent.
10. To view the rules of level shifter of this domain, double-click the `ls_CPU_to_MEM` node under the *Level Shifter* node in the *Power Domain Details* pane.
11. The level shifter rule is applied to the inputs of the memory. Therefore, the source code in *nTrace* has an "I" annotation on all the inputs of `i_pram`.

Go to the *nSchema* window and perform the following steps to practice visualizing power information in the schematic window:

1. In the *Design Browser* pane, double-click the system node and select the **New Schematic**  icon.

In the created *nSchema* window, blocks are highlighted with different power domain colors to indicate to which power domain each block belongs to. It is a quick way to see the number of power domains at the top-level view.

2. Double-click the *CPU* block and see that each of the three blocks below it belong to a different power domain.
3. Select the *ALUB* block to view the Power Domain information on the toolbar.
4. Double-click the *ALUB* block to highlight the instances below. All the instances have the same power domain as their parent scope. The signals highlighted in red are retention signals.
5. Select a retention signal, and notice that the information of Power Signal Type is shown on the toolbar. If the *Show Tip* option in the **Schematics -> Select** page of the *Preferences* form is turned *on*, hovering the mouse cursor on that information displays a tip for Restore and Save conditions.
6. Turn *on* the **Schematic -> Active Annotation** option; to see that the power state is annotated on schematic symbols.
7. Type in 6850 in the *Cursor Time* field on the toolbar, the schematic symbol with state *off* is grayed out.
8. To change the highlight color for power objects in the *Preferences* form, invoke the *Preferences* form using the **Tools -> Preferences** option. Select the **Power Aware** folder in the left pane to view the configuration page for *Power Aware Objects*. Select the *Retention Object*, set the color to green, and click the **Apply** button.
9. Toggle off the **Highlight Power Aware Design Object(s)** option in the *Preferences* form and click the *Apply* button. Then, the color highlight for retention signals in the *nSchema/nTrace* window is removed. Similarly, toggle off the **Highlight Power Domain** option to remove the color highlight for the power domains in all panes. Now, toggle *on* both **Highlight Power Aware Design Object(s)** and **Highlight Power Domain** options and click the *Apply* button to resume the highlight. Alternatively, select **View -> Highlight Power Aware Object** and toggle on and off the **Power Domain Color** or/and **Power Aware Object Color** options.

Go to the *nWave* window and perform the following steps to practice visualizing power information in the waveform window.

1. Click the *Get Signals* icon to open the *Get Signals* form.
2. Click the **system** -> **i_cpu** -> **i_CCU** node, select the signals, and click the *OK* button. The signal pane shows the selected signals.
3. Drag-and-drop a node from the *Design Browser* pane to the *nWave* window. In the *nWave* signal pane, added Isolation/Level-shifted signals are highlighted in yellow and are annotated with the *Iso/Lvs* postfix.

To practice with Active Trace and learn how power information is visualized in the active trace results, perform the following steps:

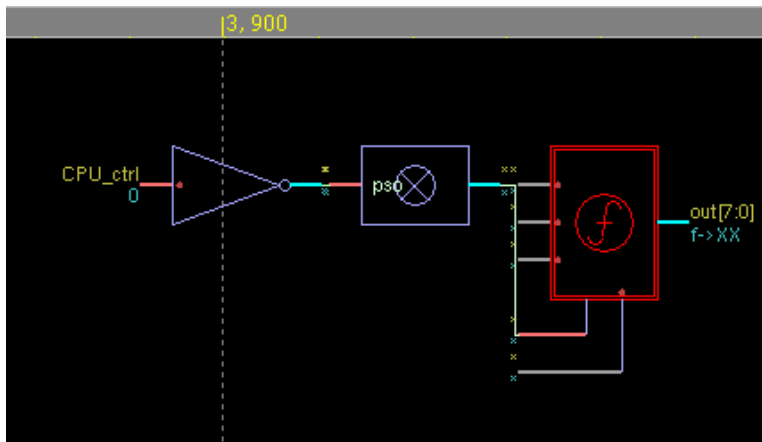
1. Drag-and-drop the `i_ALUB` scope from the *Design Browser* pane to the *nWave* window. Select the `ALU[7:0]` signal in the *nWave* signal pane. Click the transition 3 -> 55 at time 951.
2. Double-click the transition to show the active driver in the *Design Source Code* pane.
3. Turn on the active annotation (**Source -> Active Annotation**) in the main *nTrace* window.
4. Do active trace on the signal, *a*, by double-clicking on it.

The **Power Domain** field on the toolbar shows the power domain name of the signal *a*. In the *Message* pane, each trace result is added with its corresponding power domain name and the name is highlighted with the relevant power domain color.

Go to the *Temporal Flow View* pane and perform the following steps to see how power information is visualized in the *Temporal Flow View* pane:

1. Select the `ALU[7:0]` signal in the *nWave* signal pane and click the transition `f -> XX`, time 3900. Choose the **Temporal Flow Window -> New Temporal Flow Window** option from the right click menu and click the **OK** button in the *Create Temporal Flow View* form.
2. In the created *Temporal Flow View* pane, select the `out[7:0]` signal and select **Trace This Value (Trace -> Trace This Value)** in the invoked *TFV* window to unroll the path.

The unrolled path shows that the cause of the value change is power isolation.



3. Drag-and-drop any pin of the power switch cell to the *Power Source Code* pane to show the related power rule.

To debug power sequences, perform the following steps:

1. Go to the *nWave* window and zoom 100%.

Observe that there are some *unknowns* (X's) in the waveform.

2. Select **View -> Power -> Mask Power Off** to enable Verdi to automatically identify the Xs caused by power off and mark them in a shade mask.
3. Drag and drop *i_pram* and *i_ALUB* scopes from the *Design Browser* pane to the *nWave* window. See the waveform around time 3900ns, the `system.i_pram.data[7:0]` signal has X, but does not have mask as it is not caused by power off.
4. Right-click `system.i_pram.data[7:0]` in the *nWave* signal pane to invoke the **Power -> Add Power Domain** option. This option traces the power domain and adds it under the `data[7:0]` signal. As the power is on, X is not caused by power off.

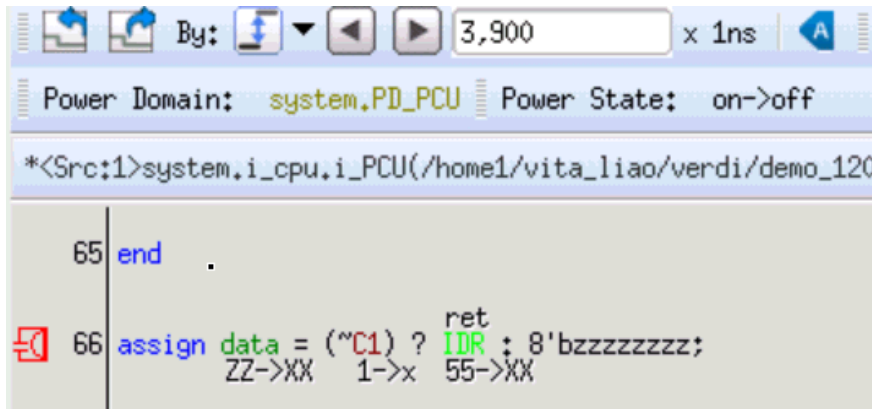
Trace the active driver of X to find out why `data[7:0]` is X.

5. Select `data[7:0]` and invoke the **Tools -> List X** option to open the *Source File to List X* form. Select the **Use Current Simulation Results** option in the *Source File Selection* field. The other two options do not contain power information. Select the **Selected SignalsPD** option in the *List* field.

After pressing the **Next** button in the *Source File to List X* form, the *List X* form displays all *unknowns*.

6. Click the **Go to next time table** button in the upper right to change time to 3900. Click the **Trace Active X** button. Click **Yes**, **OK** and **Trace** buttons in the following three forms, the *Trace Active X Results* pane is displayed.

7. In the pane, it shows `X` is caused by power off. Select the **Action -> Show source code on nTrace** option in the form to continue tracing.
8. Enable the **Source -> Active Annotation** option in the main window. In the source code, the driving signals `C1` and `IDR` are changed to `X` as their power domain (`system.PD_PCU`) is switched off at the time 3900.



`X` was caused by missing isolation rules (between `PD_MEM` and `PD_PCU` power domains). This means there should be isolation cells on `C1` and `IDR` so it does not corrupt the signals in power on the module.

9. Let's look at other unknowns, say `system.i_cpu.i_ALUB.S1`, in *nWave* window and see what causes it to be `X`.
10. Right-click `s1` in the *nWave* signal pane and select the **Power -> Add Power Domain** option. This adds the `PD_ALUB` power domain to the waveform. This is the power domain `s1` is in. You can quickly see the power state of the `s1` power domain.
11. Go back to `s1` in *nWave* and find out what causes `X`. Select `s1` and double-click at time 6850 to locate the active driver.
12. Go to the *Design Source Code* pane to view active drivers. Select `CH[0]` and do active trace (double-click) to locate the next active driver.
13. Show `CH` has retention annotation and it is in a different power domain. (Review the power domain on the toolbar.)
14. Hover the mouse cursor on `CH` to view save/restore condition. Drag-and-drop `CH` to *nWave*.

Let's find out the reason for `CH[4:0]` to remain `X` after the power is turned *on*.

15. Right-click `CH[4:0]` and select the **Power -> Add Power Domain** option.
16. Trace control signals for the retention rule. Right-click `CH[4:0]` and select the **Power -> Retention Rules -> Add Ret Controls** option. This option automatically traces save and restore signals and adds them to *nWave*.

17. Let's see the source code of the UPF file. right-click `CH[4:0]` and select **Power -> Retention Rules -> Locate Ret. Command** option. This option jumps to the *Power Source Code* pane and automatically locate the rule.
18. Turn on **Source -> Active Annotation**. The power annotation is shown in the *Power Source Code* pane.
19. The power state is also shown in the *Power Mode Table* pane. The Verdi platform automatically identifies power states at any given simulation time.

Go back to *nWave*, check the signals added in Step 16. The `CPU_restore` signal tries to restore before the power is turned on again.

20. Let's check the power domain's shut-off rule. Go to the *Power Domain Details* pane and select the **Shut off Condition** node under the `PD_CCU` node to check the shut off condition.
21. Drag-and-drop the shut off the control port `CPU_ctrl` signal to *nWave*.
22. `CPU_ctrl` comes before `CPU_save`, hence the shut-off starts before data is saved. This leads to an unknown value being restored when power is turned on again. This concludes that the power sequence is wrong.

6

Appendix A: Power Map Validation

Currently, the Power Map analyzes the power mode (or state table) definition in UPF to determine if a possibility exists that the *from* domain (the domain from where a signal is coming from) is *off* while the *to* domain (the domain the signal is going to) is not. If this is the case, unexpected results could propagate through such signals from the *from* domain to the *to* domain.

For example, there are three power domains A, B, and C; two power modes: 1 and 2 in total.

	Domain A	Domain B	Domain C
Mode 1	On	On	Off
Mode 2	On	Off	Off

When domain B is *off*, domain C is always *off* (both in mode 1 and mode 2). When domain C is *off*, domain B could be *on* (mode 1) or *off* (mode 2). So, from domain C to domain B, it is required to insert an isolation cell. However, from domain B to domain C, it is not necessary to insert an isolation cell.

To skip analyze power mode, turn *off* the **Check Power Mode** option (default: *on*) in the *Preferences* form.